Appunti di Algoritmi e Strutture Dati - 1 parte

Alessio Novel

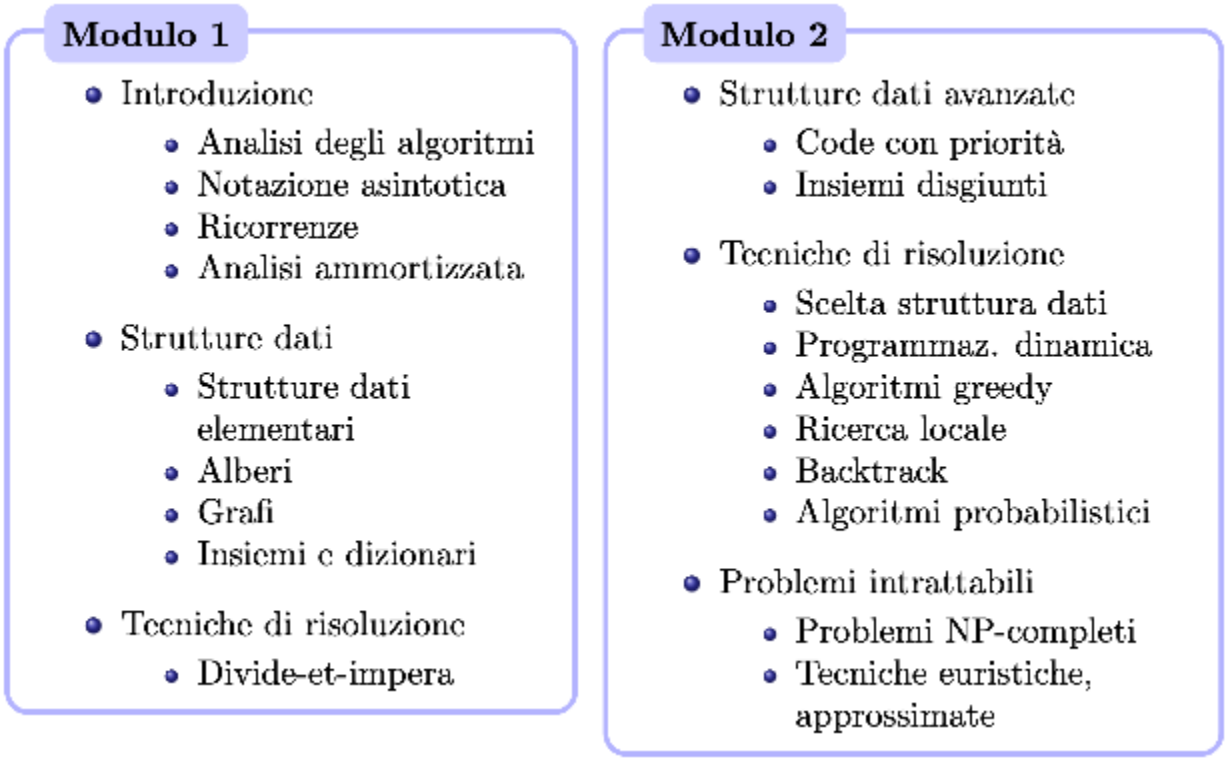
Introduzione....4
Panoramica del corso....4
Introduzione agli algoritmi....4
Descrizione di algoritmi....5
Valutazione di algoritmi....7
Analisi di algoritmi....11
Introduzione....11
Modelli di calcolo....11
Ordini di complessità....15
Notazione asintotica....15
Complessità algoritmi vs Complessità problemi....19
Tipologia dell’input....26
Problema dell’ordinamento....26
Analisi delle funzioni di costo....31
Proprietà della notazione asintotica....31
Ricorrenze....36
Analisi per livelli....37
Analisi per tentativi....42
Errore nell'intuizione....45
Difficoltà matematiche nel passo induttivo....47
Difficoltà matematiche nel caso base....50
Metodo dell’esperto....53
Ricorrenze lineari con partizione bilanciata....54
Ricorrenze lineari di ordine costante....59
Esempi di applicazione....60
Tipo di dato e strutture di dati....61
Strutture dati astratte....61
Sequenza....62
Insieme....63
Dizionario....64
Albero....65
Grafo....65
Commenti....65
Implementazione strutture dati elementari....66
Lista....66
Pila....67
Coda....68

Analisi ammortizzata....69
Esempio: Contatore Binario....70
Metodo dell’aggregazione....70
Metodo degli accantonamenti....73
Metodo del potenziale....74
Esempio: Vettore dinamico....75
Somma: raddoppiamento....77
Somma: incremento....78
Cancellazione....80
Alberi....84
Albero binario....85
Albero generico....89
Memorizzazione....91
Alberi binari di ricerca....93
Ricerca....94
Minimo-massimo....96
Successore-predecessore....96
Inserimento....97
Cancellazione....98
Alberi binari di ricerca bilanciati....100
Rotazioni....104
Inserimento....105
Altezza....108
Cancellazione....110
Grafi....111
Definizioni....113
Specifica....116
Memorizzazione di grafi....117
Visita di un grafo....121
Breadth-first search....123
Depth-first-search....126
Hashing....146
Introduzione....146
Funzioni hash....147
Estrazione....149
XOR....150
Metodo della divisione....151
Metodo della moltiplicazione (Knuth)....152
Gestione collisioni....153
Liste/vettori di trabocco....153
Indirizzamento aperto....155
Insiemi e dizionari....162
Divide-et-impera....166
Introduzione....166

Torre di Hanoi..... 168
Quicksort..... 170
Algoritmo di Strassen..... 173
Conclusioni..... 175
Esercizio..... 175
Strutture dati speciali..... 177
Code con priorità..... 178
Vettore heap..... 180
HeapSort..... 183
Implementazione code con priorità..... 191

Introduzione

Panoramica del corso



Scegliere un algoritmo efficiente è importantissimo, ad esempio basta aumentare di poco la lunghezza di un array che un algoritmo fatto male inizia ad aumentare il tempo di esecuzione in maniera esponenziale.
In questo corso vedremo una panoramica sui problemi fondamentali e le loro soluzioni algoritmiche, così da ampliare il bagaglio di conoscenze che servirà per risolvere nuovi problemi. E poi studieremo delle tecniche standard che si usano per risolvere un'ampia gamma di problemi complessi.
Un algoritmo deve essere guardato per credere che sia corretto, il miglior modo per capire come funziona (e se funziona) un algoritmo è provarlo.

Introduzione agli algoritmi

Dati un dominio di input e un dominio di output, un **problema computazionale** è rappresentato dalla **relazione matematica** che associa ogni elemento del dominio di input ad uno o più elementi del dominio di output.
Dato un problema computazionale, un **algoritmo** è un procedimento effettivo, espresso tramite un insieme di passi elementari (cioè passi che la macchina elementare può fare) ben specificati in un sistema formale di calcolo, che risolve il problema computazionale in tempo finito.

Esiste una distinzione ben precisa tra un **problema computazionale** e gli **algoritmi** che lo risolvono: mentre l'enunciato specifica quale relazione esista tra l'ingresso e l'uscita, l'algoritmo descrive la sequenza di azioni che bisogna eseguire per ottenere l'uscita.

Il nome **algoritmo** deriva da un matematico, astrologo, astronomo e geografo uzbeko: al-Khwarizmi. Ha anche introdotto i numeri indiani (chiamati arabi) nel mondo occidentale.

Prendiamo come esempio il seguente problema computazionale:
Il minimo di un insieme S è l'elemento di S che è minore o uguale ad ogni elemento di S:

$$min(S) = a \Leftrightarrow \exists a \in S \ : \ \forall b \in S, \ a \leq b$$

A questo problema possiamo associare il seguente "algoritmo":
Per trovare il minimo di un insieme, confronta ogni elemento con tutti gli altri; l'elemento che è minore di tutti è il minimo
Ora facciamo alcune osservazioni sull' "algoritmo" che abbiamo trovato:

- **Descrizione dell'algoritmo**
  Abbiamo fatto uso di un linguaggio naturale/impreciso e di una terminologia non ben definita ("con tutti gli altri"). **Dobbiamo usare un linguaggio più formale**
- **Valutazione**
  Possiamo ritenerci soddisfatti dell'algoritmo? Oppure ne esiste uno migliore? Capiremo più avanti cosa significa migliore ma notiamo subito che confrontare ogni elemento con tutti gli altri, anche con quelli che abbiamo già scartato, non ha molto senso
- **Progettazione di algoritmi**
  In questo caso il problema era molto semplice e non richiedeva sforzi di progettazione, ma spesso non è così

Descrizione di algoritmi

Dobbiamo usare una descrizione formale, che è indipendente dal linguaggio di programmazione che useremo per metterlo in pratica, quindi faremo uso di **pseudo codice**

Bisogna fare particolare attenzione al livello di dettaglio, ad esempio in una ricetta di canederli leggo: "...amalgamate il tutto e fate riposare un quarto d'ora…". Cosa significa "amalgamare", cosa significa "far riposare"? Perchè non ho scritto direttamente "prepara i canederli"?
Questo perché come abbiamo detto prima gli algoritmi si basano sulle operazioni elementari della macchina che dovrà eseguirli, e assumendo che la "macchina" sia lo chef, è corretto scrivere la ricetta assumendo certi concetti come operazioni elementari.

```
int min(int[] S, int n)
for i = 1 to n do
    boolean isMin = true
    for j = 1 to n do
        if i ≠ j and S[j] < S[i]
         then
            isMin = false
    if isMin then
        return S[i]
```



Dobbiamo utilizzare **azioni elementari**, ovvero azioni le quali applicate su un dato diano un risultato certo, unico e ripetibile e sia indipendente da terzi fattori.

Pseudo-codice

- $a = b$
- $a \leftrightarrow b \ \equiv$
  $tmp = a; a = b; b = tmp$
- $T[\,]\ A = \textbf{new}\ T[1 \ldots n]$
- $T[\,][\,]\ B = \textbf{new}\ T[1 \ldots n][1 \ldots m]$
- **int**, **float**, **boolean**, **int**
- **and**, **or**, **not**
- $==, \neq, \leq, \geq$
- $+, -, \cdot, /, \lfloor x \rfloor, \lceil x \rceil, \log, x^2, \ldots$
- iif$(condizione, v_1, v_2)$
- **if** *condizione* **then** istruzione
- **if** *condizione* **then** istruzione1 **else** istruzionc2
- **while** *condizione* **do** istruzione
- **foreach** *elemento* $\in$ *insieme* **do** istruzione
- **return**
- % *commento*

- **for** $indice = estremoInf$ **to** $estremoSup$ **do** istruzione

  int $indice = estremoInf$
  while $indice \leq estremoSup$ **do**
  $istruzione$
  $indice = indice + 1$

- **for** $indice = estremoSup$ **downto** $estremoInf$ **do** istruzione

  int $indice = estremoSup$
  while $indice \geq estremoInf$ **do**
  $istruzione$
  $indice = indice - 1$

- RETTANGOLO $r =$ **new** RETTANGOLO
- $r.altezza = 10$
- **delete** r
- $r =$ **nil**

RETTANGOLO
int *lunghezza*
int *altezza*

Valutazione di algoritmi

Per valutare un algoritmo prendiamo in considerazione i due fattori più importanti, ovvero:

- **Correttezza**
  Dobbiamo farlo con una dimostrazione matematica (notiamo che alcuni problemi possono venir risolti solo in modo approssimato, mentre altri non possono proprio venir risolti)
- **Efficienza**
  Dobbiamo stabilire come valutare l'efficienza di un programma.
  Certi problemi non possono essere risolti in maniera "efficiente"

Quando parliamo di **efficienza** non andiamo a guardare il tempo effettivo, o meglio non in maniera diretta, questo perché il tempo dipende moltissimo dalla macchina e dal tipo dei dati che abbiamo. Un confronto più corretto e generale è quello di contare il numero di **operazioni elementari rilevanti** che vengono fatte nell'algoritmo.
Infatti per valutare la **complessità** di un algoritmo analizzeremo le **risorse** impiegate da esso per risolvere il problema, in funzione della **dimensione** e della **tipologia** dell'input.
Le risorse sono:

- Il **tempo**
  Come abbiamo detto possiamo considerare quello misurato con il cronometro, ma ci è di poco aiuto, in quanto ci sono tanti fattori (bravura del programmatore, linguaggio di programmazione utilizzato, codice generato del compilatore, processore e memoria, sistema operativo, etc) che vanno ad influenzarlo.
  Quindi in genere si usa una rappresentazione più astratta del tempo, ad esempio considerando il numero di operazioni rilevanti o il numero di operazioni elementari

- Lo **spazio**
  La quantità di memoria utilizzata
- La **banda**
  La quantità di bit spediti (nel caso di algoritmi distribuiti)

Tornando al nostro esempio del minimo, come tempo possiamo considerare il numero di operazioni di confronto <

Valutazione algoritmi – Minimo

Contiamo il numero di confronti per il problema del minimo

```
int min(int[] S, int n)
for i = 1 to n do
    boolean isMin = true
    for j = 1 to n do
        if i ≠ j and S[j] < S[i] then
            isMin = false
    if isMin then
        return S[i]
```

Algoritmo "naïf": $n^2 - n$

Si può fare meglio di così?

Otteniamo n^2 - n perchè abbiamo due cicli annidati che eseguono l'operazione <, però non verrà eseguita n volte, ovvero nei casi in cui $i = j$.
Possiamo però migliorare (come abbiamo visto prima):

Valutazione algoritmi – Un algoritmo migliore

Contiamo il numero di confronti per il problema del minimo

```
int min(int[] S, int n)
% Partial minimum
int min = S[1]
for i = 2 to n do
    if S[i] < min then
        % Update partial minimum
        min = S[i]
return min
```

Algoritmo "naïf": $n^2 - n$

Algoritmo efficiente: $n-1$

Vediamo ora la **correttezza**, aggiungendo qualche nozione molto utile per la verifica di questo fattore.

Invariante: Condizione sempre vera in un certo punto del programma

Invariante di ciclo: Una condizione sempre vera all'inizio dell'iterazione di un ciclo

Invariante di classe: Una condizione sempre vera al termine dell'esecuzione di un metodo della classe.

Il concetto di invariante di ciclo ci aiuta a dimostrare la correttezza di un algoritmo iterativo:

- Inizializzazione (caso base):
  La condizione è vera alla prima iterazione di un ciclo
- Conservazione (passo induttivo):
  Se la condizione è vera prima di un'iterazione del ciclo, allora rimane vera al termine (quindi prima della successiva iterazione)
- Conclusione:
  Quando il ciclo termina, l'invariante deve rappresentare la "correttezza" dell'algoritmo

1 Esercizio: Dimostrare che l'invariante di min() è rispettata

L'invariante da dimostrare è la seguente: All'inizio di ogni iterazione del ciclo **for**, la variabile *min* contiene il minimo parziale degli elementi $S[1 \dots i-1]$.

- *Inizializzazione*: All'inizio della prima iterazione, il valore della variabile i è pari a 2. Essendo *min* inizializzato ad $S[1]$, esso è banalmente il minimo del sottovettore $S[1 \dots 1]$.
- *Conservazione*: All'inizio di una iterazione, *min* contiene il valore del minimo di $S[1 \dots i-1]$. Al termine, i viene incrementata di 1 e *min* viene confrontanto con $S[i]$, ed eventualmente aggiornato con tale valore, se più basso. Quindi all'inizio del ciclo successivo *min* contiene il minimo di $S[1 \dots i]$, come richiesto.
- *Conclusione*: al termine del ciclo, il valore della variabile i è pari a $n+1$. Quindi *min* contiene il minimo del vettore $S[1 \dots n]$, ovvero il valore che volevamo ottenere.

Vediamo anche un esempio di verifica di correttezza di un algoritmo ricorsivo, grazie all'**induzione**:

Valutazione algoritmi – Correttezza

La dimostrazione per induzione è utile anche per gli algoritmi ricorsivi

```
int binarySearch(int[] S, int v, int i, int j)
if i > j then
    return 0
else
    int m = ⌊(i + j)/2⌋
    if S[m] == v then
        return m
    else if S[m] < v then
        return binarySearch(S, v, m + 1, j)
    else
        return binarySearch(S, v, i, m − 1)
```

Per induzione sulla dimensione n dell'input

- Caso base: $n = 0$ $(i > j)$
- Ipotesi induttiva: vero per tutti gli $n' < n$
- Passo induttivo: dimostrare che è vero per n

2 Esercizio: Dimostrare che binarySearch() è corretta.

La correttezza si dimostra per induzione sulla dimensione n del vettore.

- Passo base: Se $n = 0$, ovvero se $i > j$, il valore cercato non è presente e correttamente l'algoritmo ritorna 0.
- Ipotesi induttiva: vogliamo dimostrare che l'algoritmo è corretto per la dimensione n, e supponiamo di aver dimostrato che l'algoritmo è corretto per tutte le dimensioni $n' < n$.
- Passo induttivo: Sia m l'elemento mediano; se $S[m] = v$, l'elemento è stato trovato e correttamente l'algoritmo ritorna l'indice m. Se invece $S[m] \neq v$, il valore deve necessariamente trovarsi negli elementi $S[i \dots m-1]$ (se minore) o $S[m+1 \dots j]$ (se maggiore). Tali sottovettori hanno dimensione minore di n, e quindi la ricerca dell'elemento in essi dà origine ad un risultato corretto.

Ci sono anche altre proprietà della correttezza come ad esempio semplicità, modularità, espandibilità, robustezza, sicurezza eccetera, i quali sono secondari per un corso come questo che si concentra sugli algoritmi, ma sono fondamentali per quanto riguardano l'ingegneria del software

Analisi di algoritmi

Introduzione

L'obiettivo è riuscire a stimare la complessità di un algoritmo in **tempo**, per poter:

- stimare il tempo impiegato per un dato input
- stimare il più grande input gestibile in tempi ragionevoli
- confrontare l'efficienza di algoritmi diversi
- ottimizzare le parti più importanti

Modelli di calcolo

La complessità è una funzione che va dalla **dimensione dell'input** n al **tempo**.
Però dobbiamo definire bene cosa sia la dimensione dell'input e definire come misurare il tempo (come abbiamo già detto).

Bisogna fare attenzione a definire bene cosa sia n, perché potrebbe essere il numero di elementi dell'input (**criterio di costo uniforme**, ad esempio ricerca del minimo in un vettore di n elementi) oppure il numero di bit necessari per rappresentarlo (**criterio di costo logaritmico**, ad esempio moltiplicazione di numeri binari lunghi n bit).
Nella maggior parte dei casi però possiamo assumere che gli "elementi" siano rappresentati da un numero costante di bit, e quindi le due misure coincidono a meno di una costante moltiplicativa.

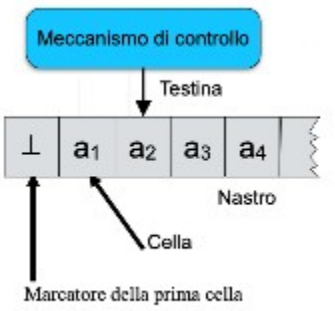
Il costo delle operazioni è principalmente valutato nel **caso pessimo**.

Come già detto, il tempo verrà misurato in **numero di operazioni elementari**, ovvero quelle operazioni che possono essere eseguite in un tempo **costante** dal processore.
Tra le operazioni elementari abbiamo alcune operazioni comuni e altre che sembrano non elementari ma che in realtà lo sono (come il coseno), ma non bisogna assumere come elementari operazioni che non lo sono, come ad esempio il minimo, che non è elementare perchè non è implementato in hardware ed è composto da più operazioni elementari (è una funzione).

Un **modello di calcolo** è una rappresentazione astratta di un calcolatore che deve avere le seguenti caratteristiche:

- Astrazione (nascondere i dettagli)
- Realismo (riflettere la situazione reale)
- Potenza matematica (trarre conclusioni “formali” sul costo)

Un esempio di modello di calcolo, forse il più famoso, è la **macchina di Turing**, la quale è una macchina ideale che manipola i dati contenuti su un nastro di lunghezza infinita, secondo un insieme prefissato di regole.



Ad ogni passo, la Macchina di Turing:

- legge il simbolo sotto la testina
- modifica il proprio stato interno
- scrive un nuovo simbolo nella cella
- muove la testina a destra o a sinistra

Questo modello è fondamentale nello studio della calcolabilità, ma è di un livello troppo basso per i nostri scopi.

Un altro modello di calcolo, che utilizzeremo noi, è la **RAM** (Random Access Machine) che ha le seguenti caratteristiche:

- Memoria
  Quantità infinita di celle di dimensione finita
  Accesso in tempo costante (indipendente dalla posizione)
- Processore (singolo)
  Set di istruzioni elementari simile a quelli reali:
  - somme, sottrazioni, moltiplicazioni, operazioni logiche, etc.
  - istruzioni di controllo (salti, salti condizionati)
- Costo delle istruzioni elementari
  Uniforme (ininfluente ai fini della valutazione come vedremo)

Abbiamo bisogno di una misura “astratta”, perché misurare un algoritmo in base al numero dei secondi impiegati da un calcolatore elettronico per eseguire una procedura che lo descriva non va bene, visto che, come abbiamo già detto, dipende da molti fattori.

Visto che i problemi da risolvere hanno una dimensione che dipende dalla grandezza dei dati in ingresso, esprimeremo il tempo di calcolo in funzione della dimensione dei dati in ingresso.
Le operazioni elementari sono:

- Aritmetiche
- Logiche
- Di confronto
- Di assegnamento

Talvolta è utile non considerare tutte le operazioni ma solo quelle dominanti. Il costo delle operazioni è principalmente valutato nel **caso pessimo**, qualche volta nel **caso medio** ma talvolta è utile considerare anche i **casi migliori**.

Tempo di calcolo min()

- Ogni istruzione richiede un tempo costante per essere eseguita
- La costante è potenzialmente diversa da istruzione a istruzione
- Ogni istruzione viene eseguita un certo # di volte, dipendente da n

ITEM min(ITEM[] A, **int** n)	Costo	# Volte
ITEM $min = A[1]$	c_1	1
for $i = 2$ **to** n **do**	c_2	n
if $A[i] < min$ **then**	c_3	$n-1$
$min = A[i]$	c_4	$n-1$
return min	c_5	1

$$T(n) = c_1 + c_2 n + c_3(n-1) + c_4(n-1) + c_5$$
$$= (c_2 + c_3 + c_4)n + (c_1 + c_5 - c_3 - c_4) = an + b$$

In generale, anche quando il caso pessimo è molto raro che si verifichi, è giusto considerare quello per calcolare il tempo così da avere il **tempo massimo**, ovvero il tempo di esecuzione che l'algoritmo non supererà mai.

Vediamo ora il tempo di calcolo di un altro algoritmo:

Tempo di calcolo binarySearch()

Il vettore viene suddiviso in due parti: Parte SX: $\lfloor (n-1)/2 \rfloor$
Parte DX: $\lfloor n/2 \rfloor$

int binarySearch(ITEM[] A, ITEM v, **int** i, **int** j)

	Costo	# $(i > j)$	# $(i \leq j)$
if $i > j$ **then**	c_1	1	**1**
return 0	c_2	1	0
else			
int $m = \lfloor (i+j)/2 \rfloor$	c_3	0	**1**
if $A[m] = v$ **then**	c_4	0	**1**
return m	c_5	0	0
else if $A[m] < v$ **then**	c_6	0	**1**
return binarySearch$(A, v, m+1, j)$	$c_7 + T(\lfloor n/2 \rfloor)$	0	0/1
else			
return binarySearch$(A, v, i, m-1)$	$c_7 + T(\lfloor (n-1)/2 \rfloor)$	0	1/**0**

Per semplicità assumiamo n come potenza di 2, quindi $n = 2^k$.
Facciamo l'assunzione di avere il caso pessimo, ovvero che l'elemento cercato non è presente.
Ad ogni passo scegliamo sempre che la parte destra ha dimensione $\frac{n}{2}$, e scegliamo sempre quella come caso pessimo.

- Due casi:

$$i > j \quad (n = 0) \qquad T(n) = c_1 + c_2 = c$$

$$i \leq j \quad (n > 0) \qquad T(n) = T(n/2) + c_1 + c_3 + c_4 + c_6 + c_7 = T(n/2) + d$$

- Relazione di ricorrenza:

$$T(n) = \begin{cases} c & \text{se } n = 0 \\ T(n/2) + d & \text{se } n \geq 1 \end{cases}$$

Abbiamo ottenuto una **relazione di ricorrenza** (vedremo più avanti come risolverle bene). Per ottenere il tempo di calcolo procediamo tramite espansione:

Soluzione della relazione di ricorrenza tramite espansione

$$
\begin{aligned}
T(n) &= T(n/2) + d \\
&= T(n/4) + 2d \\
&= T(n/8) + 3d \\
&\dots \\
&= T(1) + kd \\
&= T(0) + (k+1)d \\
&= kd + (c+d) \\
&= d \log n + e.
\end{aligned}
$$

$n = 2^k \Rightarrow k = \log n$

Ordini di complessità

Calcolando il tempo otteniamo delle **complessità**.
Per capire quanto sia efficiente un algoritmo possiamo usare questa tabella che mostra quali complessità sono migliori e quali peggiori.

Classi di complessità

$f(n)$	$n = 10^1$	$n = 10^2$	$n = 10^3$	$n = 10^4$	**Tipo**
$\log n$	3	6	9	13	logaritmico
$\sqrt{n}$	3	10	31	100	sublineare
n	10	100	1000	10000	lineare
$n \log n$	30	664	9965	132877	loglineare
n^2	10^2	10^4	10^6	10^8	quadratico
n^3	10^3	10^6	10^9	10^{12}	cubico
2^n	1024	10^{30}	10^{300}	10^{3000}	esponenziale

Notazione asintotica

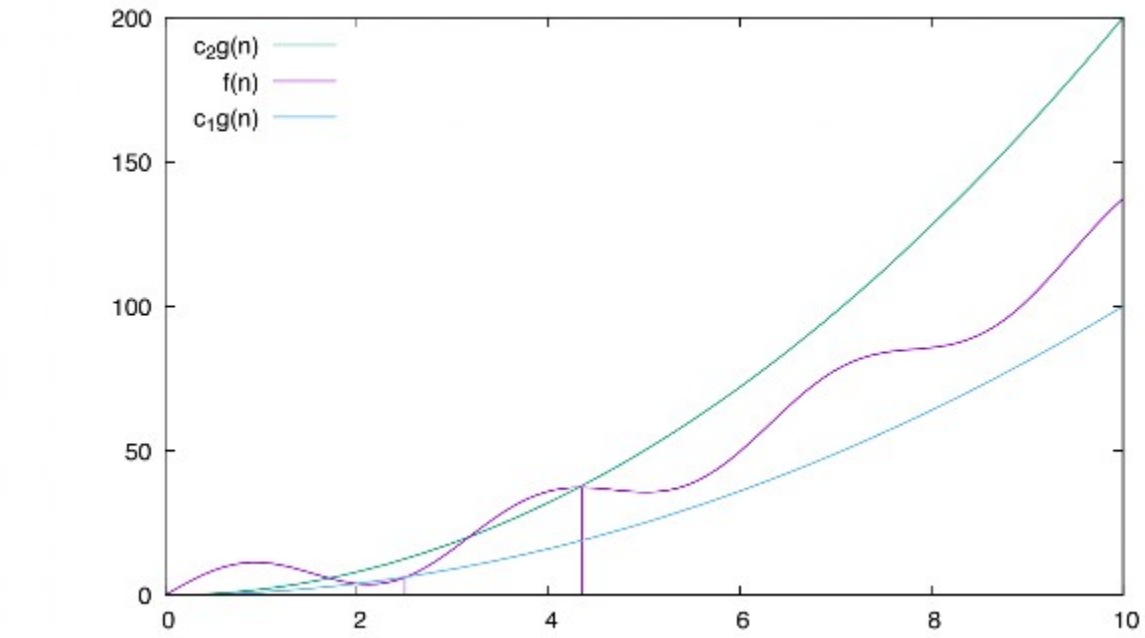
Nel valutare il tempo di calcolo $T(n)$ di una procedura è difficile quantificare con esattezza le costanti coinvolte. Facciamo uso quindi di una valutazione del numero di operazioni in **ordini di grandezza**, cioè esprimendolo come limitazione della funzione $T(n)$ al tendere all'infinito della dimensione n.
Usiamo quindi tre notazioni per gli ordini di grandezza, sia $g(n)$ una funzione di costo:

- indichiamo con $O(g(n))$ l'insieme delle funzioni $f(n)$ tali che:
  $\exists c > 0,\ \exists m \geq 0\ :\ f(n) \leq c \cdot g(n),\ \forall n \geq m$

Scriveremo quindi $f(n)=O(g(n))$ e si legge "$f(n)$ *è O grande di* $g(n)$"
Diremo che $g(n)$ è un **limite asintotico superiore** per $f(n)$, ovvero che $f(n)$ cresce al più come $g(n)$

- indichiamo con $\Omega(g(n))$ l'insieme delle funzioni $f(n)$ tali che:
  $\exists c>0,\ \exists m\geq 0\ :\ f(n)\geq c\cdot g(n),\ \forall n\geq m$
  Scriveremo quindi $f(n)=\Omega(g(n))$ e si legge "$f(n)$ *è Omega grande di* $g(n)$"
  Diremo che $g(n)$ è un **limite asintotico inferiore** per $f(n)$, ovvero che $f(n)$ cresce almeno quanto $g(n)$
- indichiamo con $\Theta(g(n))$ l'insieme delle funzioni $f(n)$ tali che:
  $\exists c_1>0, \exists c_2>0\ ,\ \exists m\geq 0\ :\ c_1\cdot g(n)\leq f(n)\leq c_2\cdot g(n),\ \forall n\geq m$
  Scriveremo quindi $f(n)=\Theta(g(n))$ e si legge "$f(n)$ *è Theta di* $g(n)$"
  Diremo che $f(n)$ cresce esattamente come $g(n)$

Vale $f(n)=\Theta(g(n))$ se e solo se $f(n)=O(g(n))$ e $f(n)=\Omega(g(n))$



Vediamo un esempio dove verifichiamo che una funzione $f(n)$ è O grande di un altra:

$$f(n) = 10n^3 + 2n^2 + 7 \stackrel{?}{=} O(n^3)$$

Dobbiamo provare che $\exists c > 0, \exists m \geq 0 : f(n) \leq cn^3, \forall n \geq m$

$$\begin{aligned} f(n) &= 10n^3 + 2n^2 + 7 \\ &\leq 10n^3 + 2n^3 + 7 && \forall n \geq 1 \\ &\leq 10n^3 + 2n^3 + 7n^3 && \forall n \geq 1 \\ &= 19n^3 \\ &\stackrel{?}{\leq} cn^3 \end{aligned}$$

che è vera per ogni $c \geq 19$ e per ogni $n \geq 1$, quindi $m = 1$.

Ma questo non è l'unico modo in cui possiamo procedere, infatti avremmo anche potuto fare cosi:

$$f(n) = 10n^3 + 2n^2 + 7 \stackrel{?}{=} O(n^3)$$

Dobbiamo provare che $\exists c > 0, \exists m \geq 0 : f(n) \leq cn^3, \forall n \geq m$

$$\begin{aligned} f(n) &= 10n^3 + 2n^2 + 7 \\ &\leq 10n^3 + 2n^3 + 7 && \forall n \geq 1 \\ &\leq 10n^3 + 2n^3 + n^3 && \forall n \geq \sqrt[3]{7} \\ &= 13n^3 \\ &\stackrel{?}{\leq} cn^3 \end{aligned}$$

che è vera per ogni $c \geq 13$ e per ogni $n \geq \sqrt[3]{7}$, quindi usiamo $m = 2$

Vediamo un esempio dove verifichiamo se una funzione è Theta di un altra:

$$f(n) = 3n^2 + 7n \stackrel{?}{=} \Theta(n^2)$$

Limite inferiore: $\exists c_1 > 0, \exists m_1 \geq 0 : f(n) \geq c_1 n^2, \forall n \geq m_1$

$$\begin{aligned} f(n) &= 3n^2 + 7n \\ &\geq 3n^2 && \text{Per } n \geq 0 \\ &\stackrel{?}{\geq} c_1 n^2 \end{aligned}$$

che è vera per ogni $c_1 \leq 3$ e per ogni $n \geq 0$, quindi $m_1 = 0$

$$f(n) = 3n^2 + 7n \stackrel{?}{=} \Theta(n^2)$$

Limite superiore: $\exists c_2 > 0, \exists m_2 \geq 0 : f(n) \leq c_2 n^2, \forall n \geq m_2$

$$\begin{aligned} f(n) &= 3n^2 + 7n \\ &\leq 3n^2 + 7n^2 && \text{Per } n \geq 1 \\ &= 10n^2 \\ &\stackrel{?}{\leq} c_2 n^2 \end{aligned}$$

che è vera per ogni $c_2 \geq 10$ e per ogni $n \geq 1$, quindi $m_2 = 1$

$$f(n) = 3n^2 + 7n \stackrel{?}{=} \Theta(n^2)$$

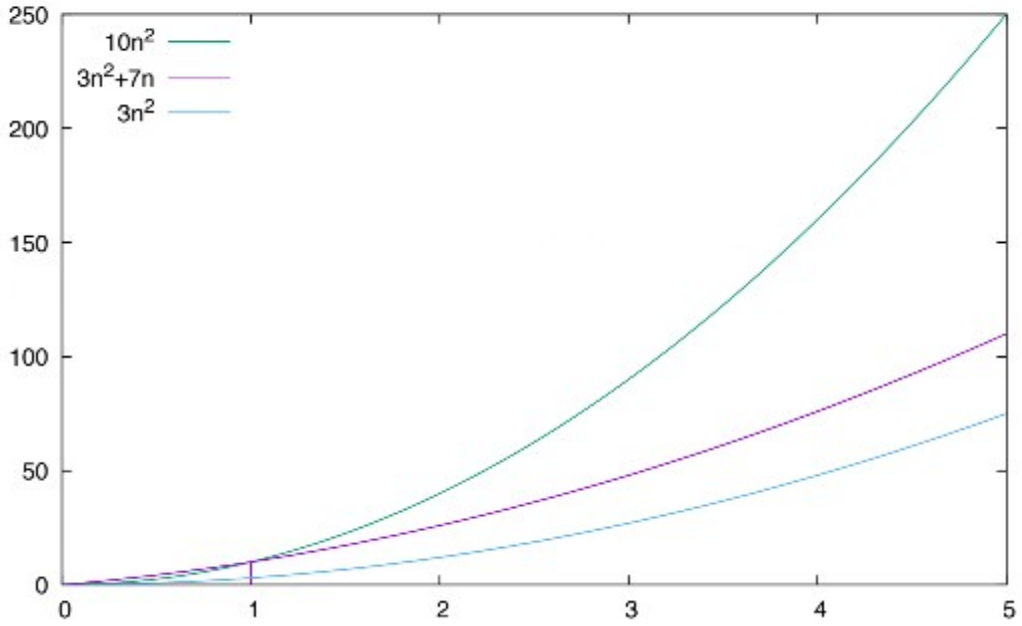
Notazione Θ:

$$\exists c_1 > 0, \exists c_2 > 0, \exists m \geq 0 : c_1 n^2 \leq f(n) \leq c_2 n^2, \forall n \geq m$$

Con questi parametri:

$$\begin{aligned} c_1 &= 3 \\ c_2 &= 10 \\ m &= \max\{m_1, m_2\} = \max\{0, 1\} = 1 \end{aligned}$$

Graficamente



Possiamo facilmente verificare che ogni funzione con grado massimo x (tipo n^x) sia O grande di ogni funzione con grado massimo k con $x \leq k$

$$n^2 \stackrel{?}{=} O(n^3)$$

Dobbiamo dimostrare che $\exists c > 0, \exists m > 0 : n^2 \leq cn^3, \forall n \geq m$

- Otteniamo questo: $n^2 \leq cn^3 \Leftrightarrow c \geq \frac{1}{n}$
- La funzione $1/n$ è monotona decrescente per $n > 0$.
- In altre parole, possiamo prendere un qualunque valore m (e.g., $m = 1$), e prendere un costante $c \geq 1/m$, come ad esempio $c = 1$.

Complessità algoritmi vs Complessità problemi

Come obiettivo abbiamo:

- Capire quando è impossibile fare di meglio
- Capire quando si può migliorare
- Capire il rapporto tra un problema computazionale e l'algoritmo
- Capire quale è il ruolo del modello di calcolo

In certi casi, anche se il problema è complesso, l'algoritmo finale potrebbe essere molto semplice o molto diverso dalla soluzione "normale" del problema, come ad esempio:

Moltiplicare numeri complessi

Moltiplicazione numeri complessi

- $(a + bi)(c + di) = [ac - bd] + [ad + bc]i$
- Input: a, b, c, d
- Output: $ac - bd$, $ad + bc$

Domande

Considerate un modello di calcolo dove la moltiplicazione costa 1, le addizioni/sottrazioni costano 0.01,

- Quanto costa l'algoritmo dettato dalla definizione?
- Potete fare meglio di così?
- Qual è il ruolo del modello di calcolo?

1 Domanda: Moltiplicare numeri complessi

1. **Quanto costa l'algoritmo "banale" dettato dalla definizione?**
   4.02
2. **Potete fare di meglio?**
   Soluzione di Gauss, datata 1805.
   Input: a, b, c, d, Output: $A1 = ac - bd$, $A2 = ad + bc$

$$\begin{aligned} m1 &= ac \\ m2 &= bd \\ A1 &= m1 - m2 = ac - bd \\ m3 &= (a + b)(c + d) = ac + ad + bc + bd \\ A2 &= m3 - m1 - m2 = ad + bc \end{aligned}$$

   In questo caso, il costo è 3.05 (3 moltiplicazioni e 5 addizioni).

3. **Qual è il ruolo del modello di calcolo?**
   Detto m il costo di una moltiplicazione e a il costo di una disequazione, si ottiene

$$3m + 5a \leq 4m + 2a$$

   disequazione che è vera solo quando $m \geq 3a$.
   Si tenga conto che al tempo, il termine "Computer" significava "Colui che calcola" ed era associato agli esseri umani (fin dal 1640); il termine ha iniziato ad essere associato alle "macchine calcolatrici" nel 1897, per poi venire associato a macchine programmabili solo nel 1945.
   Questo giustifica lo sforzo di Gauss per ridurre il numero di calcoli. Tenete conto che esistevano macchine calcolatrici per l'addizione a partire dal 1645 (la pascalina di Pascal) e per la moltiplicazione a partire dal 1672 (la macchina di Leibniz), ma la prima produzione di massa avvenne solo a partire dal 1820.

Vediamo ora un esempio dove non si può trovare un metodo più efficiente di quello "normale":

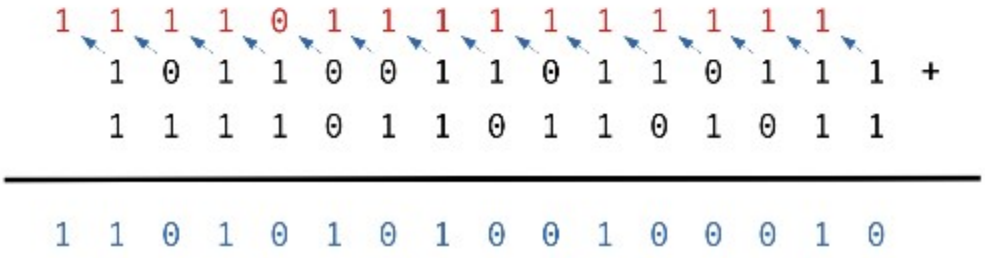
Sommare numeri binari

Algoritmo elementare della somma – sum()

- richiede di esaminare tutti gli n bit
- costo totale $cn = O(n)$
  ($c \equiv$ costo per sommare tre bit e generare riporto)

Domanda

Esiste un metodo più efficiente?



2 Domanda: Sommare numeri binari

Dimostrazione per assurdo. Supponiamo di avere un algoritmo "misterioso" che non esamina tutti i bit. Si esegua l'algoritmo su una coppia di numeri x e y. Deve esistere qualche bit in posizione i che non viene esaminato in uno dei due numeri. Supponiamo che l'algoritmo dia una risposta esatta (altrimenti non sarebbe corretto). Cambiamo il valore del bit nella posizione i e diamo all'algoritmo la nuova coppia di numeri. Risponde con lo stesso valore di prima, perché quel bit non viene esaminato, dando quindi una risposta sbagliata perché la somma è cambiata. Assurdo.
In altre parole, qualunque algoritmo che effettui una somma deve esaminare tutti i bit, e quindi non può sommare in modo sublineare. Il meccanismo di somma della scuola elementare è ottimale.

Finora abbiamo parlato di complessità di algoritmi, parliamo ora di complessità di **problemi**.

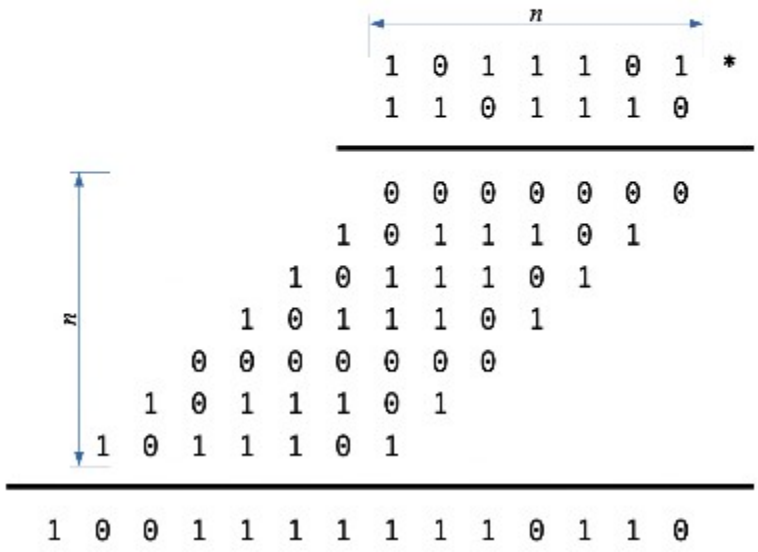
Un problema ha complessità $O(f(n))$ se **esiste almeno un algoritmo** (che lo risolva) che ha complessità $O(f(n))$.
Ad esempio il problema della somma dei numeri binari ha complessità $O(n)$.

Un problema ha complessità $\Omega(f(n))$ se **tutti i possibili algoritmi** (che lo risolvono) hanno complessità $\Omega(f(n))$.
Ad esempio il problema della somma dei numeri binari ha complessità $\Omega(n)$.

Non bisogna fare l'errore di confrontare gli algoritmi per dire che un problema è più costoso di un altro, si può fare solo quando gli algoritmi sono i più efficienti possibili (dimostrando che non esistono algoritmi più efficienti), ad esempio prendiamo questo algoritmo per la moltiplicazione di numeri binari:

Algoritmo elementare del prodotto – prod()

- moltiplicazione di ogni bit con ogni altro bit
- costo totale $cn^2 = O(n^2)$



Confronto della complessità computazionale

- Somma : $T_{sum}(n) = O(n)$
- Prodotto : $T_{prod}(n) = O(n^2)$

Si potrebbe concludere che...

- Il problema della moltiplicazione è inerentemente più costoso del problema dell'addizione
- Conferma la nostra esperienza

Così però abbiamo dimostrato solo che l'**algoritmo** della somma delle elementari è più efficiente dell'**algoritmo** del prodotto delle elementari. Però non sappiamo se esiste una soluzione in tempo sub-quadratico per il **problema** del prodotto. L'errore che abbiamo fatto è quello di **confrontare gli algoritmi**, quando in realtà dobbiamo **confrontare i problemi**, infatti proviamo a ragionare nella seguente maniera:

Divide-et-impera

- Divide: dividi il problema in sottoproblemi di dimensioni inferiori
- Impera: risolvi i sottoproblemi in maniera ricorsiva
- Combina: unisci le soluzioni dei sottoproblemi in modo da ottenere la risposta del problema principale

Moltiplicazione divide-et-impera

$$X = a \cdot 2^{n/2} + b$$
$$Y = c \cdot 2^{n/2} + d$$
$$XY = ac \cdot 2^n + (ad + bc) \cdot 2^{n/2} + bd$$

X	a	b
Y	c	d

---

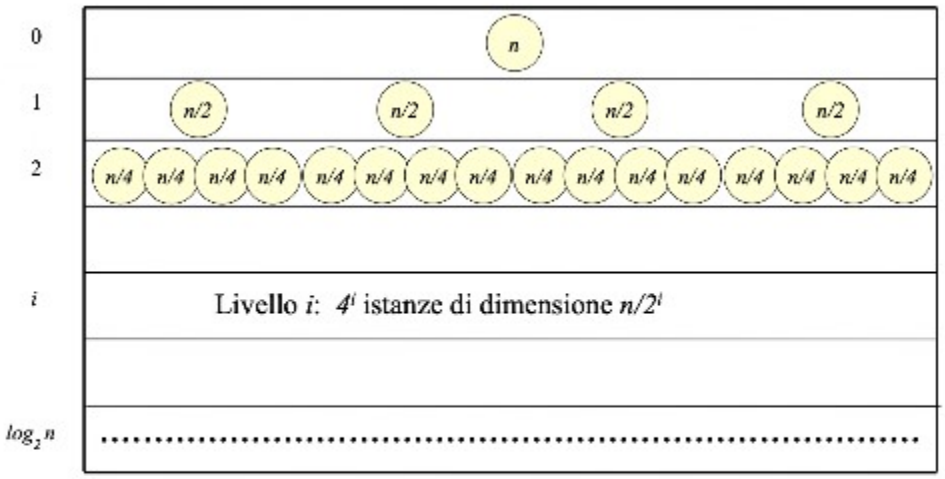
boolean[] pdi(**boolean**[] X, **boolean**[] Y, **int** n)

---

if $n == 1$ **then**
  return $X[1] \cdot Y[1]$
else
  spezza X in $a; b$ e Y in $c; d$
  return $\text{pdi}(a, c, n/2) \cdot 2^n + (\text{pdi}(a, d, n/2) +$
    $\text{pdi}(b, c, n/2)) \cdot 2^{n/2} + \text{pdi}(b, d, n/2)$

$$T(n) = \begin{cases} c_1 & n = 1 \\ 4T(n/2) + c_2 \cdot n & n > 1 \end{cases}$$

Nota: Moltiplicare per $2^t \equiv$ shift di t posizioni, in tempo lineare



$c_2 \cdot 1 \cdot n$

$c_2 \cdot 4 \cdot n/2$

$c_2 \cdot 4^2 \cdot n/2^2$

$c_2 \cdot 4^i \cdot n/2^i$

$T(1) \cdot 4^{\log n}$

$$T(n) = \begin{cases} c_1 & n = 1 \\ 4T(n/2) + c_2 \cdot n & n > 1 \end{cases}$$

$= c_1 \cdot n^{\log 4}$

$= c_1 \cdot n^2$

Notiamo che la complessità resta uguale (ovvero quadratica), e in più le costanti moltiplicative sono più alte. Quindi non ha senso usare questo algoritmo se è addirittura più lento, possiamo però osservare che utilizzando il trucchetto di prima della moltiplicazione dei numeri complessi possiamo risparmiare una chiamata:

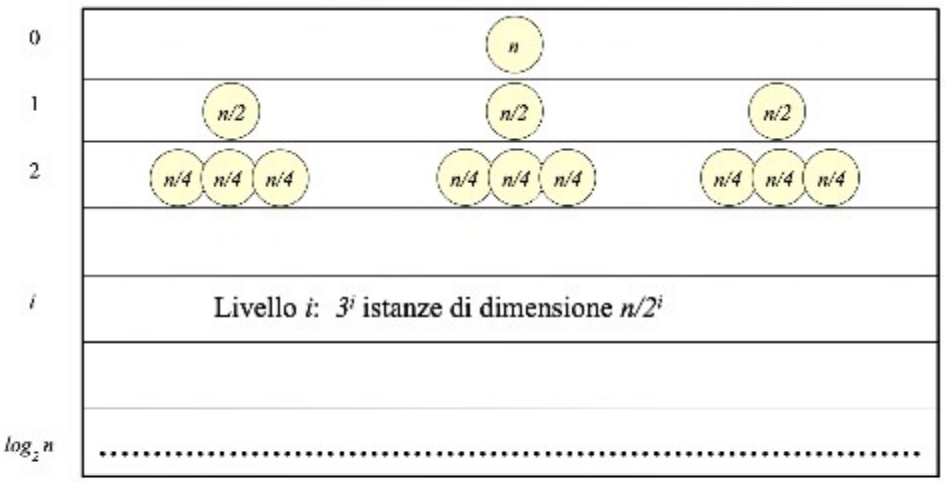
Moltiplicazione di Karatsuba (1962)

$$A_1 = a \times c$$
$$A_3 = b \times d$$
$$m = (a + b) \times (c + d) = ac + ad + bc + bd$$
$$A_2 = m - A_1 - A_3 = ad + bc$$



boolean [] KARATSUBA(**boolean**[] X, **boolean**[] Y, **int** n)

if $n == 1$ **then**
  return $X[1] \cdot Y[1]$
else
  spezza X in $a; b$ e Y in $c; d$
  boolean[] $A_1 =$ KARATSUBA$(a, c, n/2)$
  boolean[] $A_3 =$ KARATSUBA$(b, d, n/2)$
  boolean[] $m =$ KARATSUBA$(a + b, c + d, n/2)$
  boolean[] $A_2 = m - A1 - A3$
  return $A_1 \cdot 2^n + A_2 \cdot 2^{n/2} + A_3$



$$T(n) = \begin{cases} c_1 & n = 1 \\ 3T(n/2) + c_2 \cdot n & n > 1 \end{cases} \quad \begin{matrix} = c_1 \cdot n^{\log 3} \\ = c_1 \cdot n^{1.58....} \end{matrix}$$

Confronto della complessità computazionale

- Prodotto : $T_{prod}(n) = O(n^2)$ Es. $T_{prod}(10^6) = 10^{12}$
- Prodotto : $T_{kara}(n) = O(n^{1.58...})$ Es. $T_{kara}(10^6) = 3 \cdot 10^9$

Possiamo concludere che l'algoritmo naif (il più facile/semplice) non è sempre il migliore, e che può esistere spazio di miglioramento a meno che non sia possibile **dimostrare il contrario**.
Ad esempio per il prodotto binario abbiamo che il problema ha complessità $\Omega(n \log n)$

Complessità in tempo di un algoritmo

La quantità di tempo richiesta per input di dimensione n

- $O(f(n))$: Per tutti gli input, l'algoritmo costa al più $f(n)$
- $\Omega(f(n))$: Per tutti gli input, l'algoritmo costa almeno $f(n)$
- $\Theta(f(n))$: L'algoritmo richiede $\Theta(f(n))$ per tutti gli input

Complessità in tempo di un problema computazionale

La complessità in tempo relative a tutte le possibili soluzioni

- $O(f(n))$: Complessità del miglior algoritmo che risolve il problema
- $\Omega(f(n))$: Dimostrare che nessun algoritmo può risolvere il problema in tempo inferiore a $\Omega(f(n))$
- $\Theta(f(n))$: Algoritmo ottimo

Tipologia dell'input

In questa sezione ci poniamo l'obiettivo di **valutare gli algoritmi in base all'input**.
In alcuni casi, gli algoritmi si comportano diversamente a seconda delle caratteristiche dell'input, e conoscere in maniera anticipata tali caratteristiche permette di scegliere il migliore algoritmo per quella situazione.
Il problema dell'**ordinamento** rispecchia bene queste caratteristiche.

L'analisi nel caso **pessimo** è la più importante, e abbiamo che il tempo di esecuzione nel caso peggiore è un **limite superiore** per qualsiasi input. Ricordiamoci poi che non si può valutare un algoritmo sulla fortuna, quindi è sempre meglio considerare il caso peggiore.
L'analisi nel caso **medio** vale per la maggior parte dei casi, nella quale l'input è distribuito in maniera uniforme (casuale).
L'analisi nel caso **ottimo** può avere senso se si hanno informazioni particolari sull'input.

Problema dell'ordinamento

Problema dell'ordinamento

- **Input**: Una sequenza $A = a_1, a_2, \ldots, a_n$ di n valori
- **Output**: Una sequenza $B = b_1, b_2, \ldots, b_n$ che sia una permutazione di A e tale per cui $b_1 \leq b_2 \leq \ldots \leq b_n$

Un approccio “demente” è quello di generare tutte le possibili permutazioni fino a quando non ne trovo una ordinata (ovviamente **non** efficiente).

Un approccio “naif” è quello di cercare il minimo e metterlo nella prima posizione, così da ridurre il problema ad $n-1$ elementi, questo è il **selection sort**:

```
SelectionSort(ITEM[] A, int n)
for i = 1 to n − 1 do
    int min = min(A, i, n)
    A[i] ↔ A[min]
```

```
int min(ITEM[] A, int i, int n)
% Posizione del minimo parziale
int min = i
for j = i + 1 to n do
    if A[j] < A[min] then
        % Nuovo minimo parziale
        min = j
return min
```

	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$i=1$	7	4	2	1	8	3	5
$i=2$	1	4	2	7	8	3	5
$i=3$	1	2	4	7	8	3	5
$i=4$	1	2	3	7	8	4	5
$i=5$	1	2	3	4	8	7	5
$i=6$	1	2	3	4	5	7	8
$i=7$	1	2	3	4	5	7	8

La complessità di questo algoritmo è costante per tutti e tre i casi (pessimo, medio e ottimo), questo perché esegue sempre tutte le operazioni indifferentemente dalla struttura dell’input:

$$\sum_{i=1}^{n-1}(n-i)=\sum_{i=1}^{n-1}i=\frac{n(n-1)}{2}=n^2-n/2=O(n^2)$$

Vediamo ora un altro algoritmo, l’**insertion sort** (efficiente per ordinare piccoli insiemi di elementi), che consiste nel creare un vettore ordinato, ed aggiungere ogni elemento nella giusta posizione rispetto all’ordinamento (come l’ordinamento di una mano di carte da gioco):

```
insertionSort(ITEM[] A, int n)
for i = 2 to n do
    ITEM temp = A[i]
    int j = i
    while j > 1 and A[j − 1] > temp do
        A[j] = A[j − 1]
        j = j − 1
    A[j] = temp
```

3 Domanda: Dimostrazione della correttezza di Insertion Sort

Quando il ciclo è un ciclo **for**, il punto in cui verifichiamo l'invariante di ciclo è appena dopo la modifica della variable (per inizializzazione o incremento), e appena prima della verifica. Questa è un regola generale, ma non è "scritta nella pietra".
L'invariante di ciclo è la seguente: *all'inizio del ciclo i-esimo, il sottovettore $A[1 \dots i-1]$ è ordinato.*

- **Inizializzazione**: Inizialmente, $j = 2$. Il sottoarray è dato da $A[1 \dots 1]$, che è naturalmente ordinato.
- **Conservazione**: una dimostrazione informale; una dimostrazione formale richiederebbe la definizione di un invariante di ciclo per il while.
  - Sappiamo che il subarray $A[1 \dots j-1]$ è ordinato.
  - Il ciclo while sposta tutti gli elementi maggiori di $A[j]$ fino a una posizione k (inclusa) di una posizione verso destra, preservando l'ordine ma duplicando il valore in posizione k.
  - Tutti i valori precedenti k sono minori di $A[j]$ e ordinati.
  - Tutti i valori dopo k sono maggiori di $A[j]$ e ordinati.
  - Inserendo $A[j]$ in posizione k dimostriamo l'invariante.
- **Terminazione**: Alla fine del ciclo, quando $j = n+1$, l'invariante dice che l'intero array $A[1 \dots j-1] = A[1 \dots n]$ è ordinato.

Questo algoritmo è efficiente per ordinare piccoli insiemi di elementi.
In questo caso il costo di esecuzione non dipende solo dalla dimensione dell'input ma anche dalla distribuzione dei dati in ingresso:

La complessità è data dal numero di confronti che devono essere effettuati per ogni elemento i, con $i \in [2, n]$. Chiamiamo questo numero di confronti t_i. Il numero di confronti varia a seconda della disposizione iniziale degli elementi. Il numero totale di confronti è pari a

$$\sum_{i=2}^{n} t_i$$

- **Caso ottimo**: L'array è ordinato, quindi $t_i = 1$ per ogni i. Questo perché $A[j-1] = A[i] \leq A[j], \forall j$, quindi il ciclo **while** esce subito. La complessità è quindi $O(n)$.
- **Caso pessimo**: L'array è ordinato nell'ordine inverso, tutti gli elementi in $A[1 \dots i-i]$ sono superiori a $A[i]$, quindi vengono tutti spostati. Quindi il numero di confronti è pari a i. Otteniamo:

$$\sum_{i=2}^{n} i - 1 = \left(\sum_{i=1}^{n-1} i \right) - 1 = \frac{n(n+1)}{2}$$

Quindi, senza perderci nei dettagli dei conti, possiamo scrivere che il risultato finale sarà:

$$T(n) = an^2 + bn + c$$

- **Caso medio**: In media, metà degli elementi in $A[1 \dots i-1]$ sono più grandi di $A[i]$, metà sono più piccoli. Quindi, quello che si ottiene per i singoli elementi è:

$$\sum_{i=2}^{n} i/2 = \frac{n(n+1)}{4} - 1$$

Il costo finale sarà quindi simile al precedente.

$$T(n) = an^2 + bn + c$$

Per l'ultimo algoritmo riprendiamo il concetto di **divide-et-impera**, utilizzando per l'appunto il **merge sort**.
Vediamo prima la funzione di merge (usata per unire due sequenze già ordinate):

```
Merge(ITEM A[ ], int first, int last, int mid)
int i, j, k, h
i = first
j = mid + 1
k = first
while i ≤ mid and j ≤ last do
    if A[i] ≤ A[j] then
        B[k] = A[i]
        i = i + 1
    else
        B[k] = A[j]
        j = j + 1
    k = k + 1
j = last
for h = mid downto i do
    A[j] = A[h]
    j = j − 1
for j = first to k − 1 do
    A[j] = B[j]
```

La quale ha complessità $O(n)$:

> **5 Domanda: Complessità di merge()**
>
> Il costo di merge(A, p, q, r) è dato da n r p $O(n)$, perché tutti gli elementi di entrambi gli array devono essere visitati almeno una volta.

Ora possiamo vedere la funzione completa, la quale chiama per l'appunto la funzione *merge*() per unire le sequenze ordinate ricorsivamente:

```
MergeSort(ITEM A[ ], int first, int last)
if first < last then
    int mid = ⌊(first + last)/2⌋
    MergeSort(A, first, mid)
    MergeSort(A, mid + 1, last)
    Merge(A, first, last, mid)
```

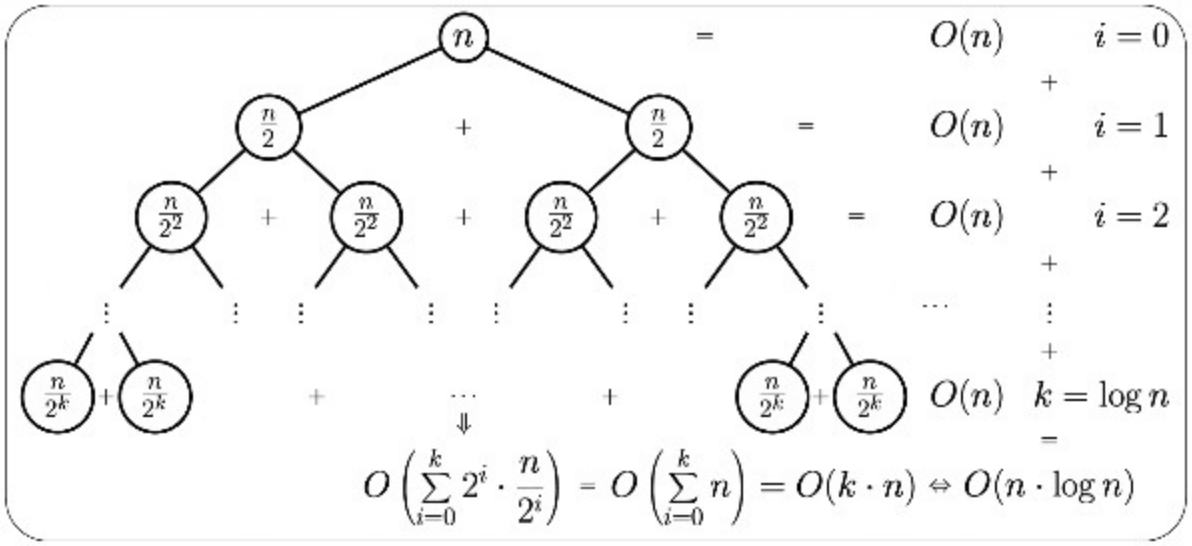
Per calcolare la complessità di *MergeSort*() assumiamo che $n = 2^k$, ovvero che l'altezza dell'albero di suddivisioni è esattamente $k = \log\ n$ (tutti i sottovettori hanno dimensioni che sono potenze esatte di 2).

Costo computazionale

$$T(n) = \begin{cases} c & n = 1 \\ 2T(n/2) + dn & n > 1 \end{cases}$$

Domanda

Qual è il costo computazionale di MergeSort()?

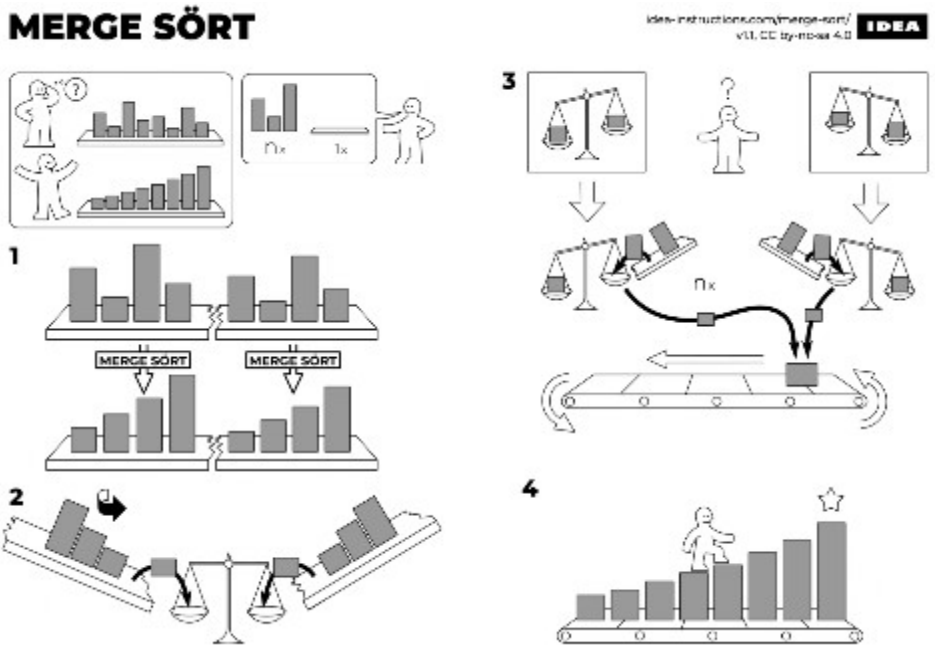


6 Domanda: Complessità di mergesort()

Svolgendo la ricorsione, otteniamo:

$$\begin{aligned} T(n) &= cn + 2T(n/2) \\ &= cn + 2cn/2 + 4T(n/4) \\ &= cn + 2cn/2 + 4cn/4 - 8T(n/8) \\ &= \dots \\ &= cn + 2cn/2 + 4cn/4 - \dots + 2^{k-1} n/2^{k-1} + 2^k n/2^k T(1) \\ &= cn + 2cn/2 + 4cn/4 - \dots + 2^{k-1} cn/2^{k-1} + nc_0 \end{aligned}$$

ovvero, ogni livello costa cn. Quanti livelli ci sono? Se $n = 2^k$, allora ci sono $k = \log n$ livelli. Quindi il costo finale sarà $cn \log n + c_0 n$.



Analisi delle funzioni di costo

Proprietà della notazione asintotica

Abbiamo una regola generale, ovvero se il **grado massimo** di una funzione polinomiale è k, allora la funzione è Theta di n^k, ovvero $f(n)=\Theta(n^k)$
Ecco la dimostrazione:

Espressioni polinomiali

$$f(n) = a_k n^k + a_{k-1} n^{k-1} + \dots a_1 n + a_0, a_k > 0 \Rightarrow f(n) = \Theta(n^k)$$

Limite superiore: $\exists c > 0, \exists m \geq 0 : f(n) \leq cn^k, \forall n \geq m$

$$\begin{aligned} f(n) &= a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \\ &\leq a_k n^k + |a_{k-1}| n^{k-1} + \dots + |a_1| n + |a_0| \\ &\leq a_k n^k + |a_{k-1}| n^k + \dots + |a_1| n^k + |a_0| n^k \qquad \forall n \geq 1 \\ &= (a_k + |a_{k-1}| + \dots + |a_1| + |a_0|) n^k \\ &\overset{?}{\leq} cn^k \end{aligned}$$

che è vera per $c \geq (a_k + |a_{k-1}| + \dots + |a_1| + |a_0|) > 0$ e per $m = 1$.

Limite inferiore: $\exists d > 0, \exists m \geq 0 : f(n) \geq dn^k, \forall n \geq m$

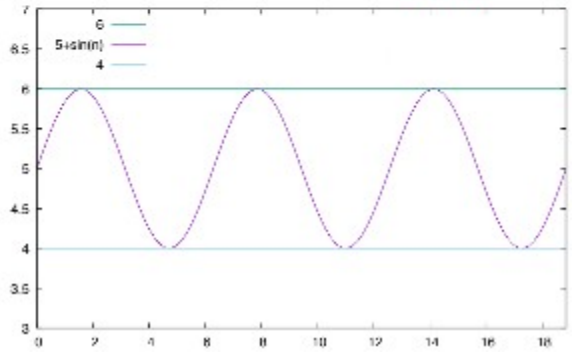
$$\begin{aligned} f(n) &= a_k n^k + a_{k-1} n^{k-1} + \ldots + a_1 n + a_0 \\ &\geq a_k n^k - |a_{k-1}| n^{k-1} - \ldots - |a_1| n - |a_0| \\ &\geq a_k n^k - |a_{k-1}| n^{k-1} - \ldots - |a_1| n^{k-1} - |a_0| n^{k-1} \qquad \forall n \geq 1 \\ &\stackrel{?}{\geq} dn^k \end{aligned}$$

L'ultima equazione è vera se:

$$d \leq a_k - \frac{|a_{k-1}|}{n} - \frac{|a_{k-2}|}{n} - \ldots - \frac{|a_1|}{n} - \frac{|a_0|}{n} > 0 \Leftrightarrow n > \frac{|a_{k-1}| + \ldots + |a_0|}{a_k}$$

Alcuni casi particolari

- Qual è la complessità di $f(n) = 5$?
  - $f(n) = 5 \geq c_1 n^0 \Rightarrow c_1 \leq 5$
  - $f(n) = 5 \leq c_2 n^0 \Rightarrow c_2 \geq 5$
  - $f(n) = \Theta(n^0) = \Theta(1)$
- Qual è la complessità di $f(n) = 5 + \sin(n)$? $\Theta(1)$



3.7 Qual è la complessità di $f(n) = 5 + \sin(n)$?

Il seno è compreso fra -1 e 1; quindi comunque il valore può essere limitato superiormente e inferiormente da una una costante, e la complessità è $\Theta(1)$.

Per stimare gli ordini di grandezza si possono usare le seguenti regole:

- Riflessività
  Per ogni costante c ed ogni funzione f, $c \cdot f(n)$ è $O(f(n))$ (lo stesso vale per Ω e Θ)
- Transitività
  Se $g(n)$ è $O(f(n))$ ed $f(n)$ è $O(h(n))$, allora $g(n)$ è $O(h(n))$ (lo stesso vale per Ω e Θ)
- Simmetria trasposta (o Dualità)
  $g(n)$ è $O(f(n))$ se e solo se $f(n)$ è $\Omega(g(n))$

- Simmetria
  $g(n)$ è $\Theta(f(n))$ se e solo se $f(n)$ è $\Theta(g(n))$
- Somma
  Se $g(n)$ è $O(f(n))$ e $h(n)$ è $O(q(n))$, allora $g(n)+h(n)$ è $O(max(f(n),q(n)))$(lo stesso vale per Ω e Θ)
- Prodotto
  Se $g(n)$ è $O(f(n))$ e $h(n)$ è $O(q(n))$, allora $g(n)\cdot h(n)$ è $O(f(n)\cdot q(n))$ (lo stesso vale per Ω e Θ)
- Eliminazione delle costanti
  Se $g(n)=O(f(n))$ allora $c\cdot g(n)=O(f(n))$ per ogni $c>0$ (lo stesso vale per Ω e Θ)

Utilizzando gli ordini di grandezza, ogni operazione elementare ha costo $O(1)$. Le uniche operazioni a non dare costo unitario sono quelle **condizionali**, di **iterazione** e le chiamate a **funzioni**.

Dimostriamo che le funzioni logaritmiche sono più efficienti delle funzioni polinomiali, dimostrando che $\log\ n$ è O grande di n:

Proprietà dei logaritmi

Vogliamo provare che $\log n = O(n)$. Dimostriamo per induzione che

$$\exists c > 0, \exists m \geq 0 : \log n \leq cn, \forall n \geq m$$

- **Caso base** $(n = 1)$:
$$\log 1 = 0 \leq cn = c \cdot 1 \Leftrightarrow c \geq 0$$
- **Ipotesi induttiva**: sia $\log k \leq ck, \forall k \leq n$
- **Passo induttivo**: Dimostriamo la proprietà per $n+1$

$$\begin{aligned} \log(n+1) &\leq \log(n+n) = \log 2n && \forall n \geq 1 \\ &= \log 2 + \log n && \log ab = \log a + \log b \\ &= 1 + \log n && \log_2 2 = 1 \\ &\leq 1 + cn && \text{Per induzione} \\ &\overset{?}{\leq} c(n+1) && \text{Obiettivo} \end{aligned}$$

$$1 + cn \leq c(n+1) \Leftrightarrow c \geq 1$$

Possiamo osservare che per le proprietà matematiche (logaritmiche ed esponenziali) valgono le seguenti osservazioni sulle complessità:

Giocando con le espressioni

- È vero che $\log_a n = \Theta(\log n)$?
  - Sì: $\log_a n = (\log_a 2) \cdot (\log_2 n) = \Theta(\log n)$
- È vero che $\log n^a = \Theta(\log n)$, per $a > 0$?
  - Sì: $\log n^a = a \log n = \Theta(\log n)$
- È vero che $2^{n+1} = \Theta(2^n)$?
  - Sì: $2^{n+1} = 2 \cdot 2^n = \Theta(2^n)$
- È vero che $2^n = \Theta(3^n)$?
  - Ovviamente $2^n = O(3^n)$
  - Ma: $3^n = \left(\frac{3}{2} \cdot 2\right)^n = \left(\frac{3}{2}\right)^n \cdot 2^n$:
    Quindi non esiste $c > 0$ tale per cui $\left(\frac{3}{2}\right)^n \cdot 2^n \leq c2^n$, e quindi $2^n \neq \Omega(3^n)$

Vediamo ora altri due concetti per la complessità:

Definizione – Notazioni o, ω

Sia $g(n)$ una funzione di costo; indichiamo con $o(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\forall c, \exists m : f(n) < cg(n), \forall n \geq m.$$

Sia $g(n)$ una funzione di costo; indichiamo con $\omega(g(n))$ l'insieme delle funzioni $f(n)$ tali per cui:

$$\forall c, \exists m : f(n) > cg(n), \forall n \geq m.$$

- Come si leggono: $f(n)$ è "o piccolo", "omega piccolo" di $g(n)$
- Come si scrivono: $f(n) = o(g(n))$ oppure $f(n) = \omega(g(n))$

Utilizzando il concetto di limite, date due funzioni $f(n)$ e $g(n)$ si possono fare le seguenti affermazioni:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f(n) = o(g(n))$$
$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c \neq 0 \Rightarrow f(n) = \Theta(g(n))$$
$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = +\infty \Rightarrow f(n) = \omega(g(n))$$

Si noti che:

$$f(n) = o(g(n)) \quad \Rightarrow \quad f(n) = O(g(n))$$
$$f(n) = \omega(g(n)) \quad \Rightarrow \quad f(n) = \Omega(g(n))$$

Possiamo fare una classificazione delle complessità delle principali espressioni, estendendo le relazioni che abbiamo dimostrato fino ad ora:

Per ogni $r < s, h < k, a < b$:

$$O(1) \subset O(\log^r n) \subset O(\log^s n) \subset O(n^h) \subset O(n^h \log^r n) \subset$$
$$O(n^h \log^s n) \subset O(n^k) \subset O(a^n) \subset O(b^n)$$

Classi di complessità

$f(n)$	$n = 10^1$	$n = 10^2$	$n = 10^3$	$n = 10^4$	**Tipo**
$\log n$	3	6	9	13	logaritmico
$\sqrt{n}$	3	10	31	100	sublineare
n	10	100	1000	10000	lineare
$n \log n$	30	664	9965	132877	loglineare
n^2	10^2	10^4	10^6	10^8	quadratico
n^3	10^3	10^6	10^9	10^{12}	cubico
2^n	1024	10^{30}	10^{300}	10^{3000}	esponenziale

Ricorrenze

Quando si calcola la complessità di un **algoritmo ricorsivo**, essa viene espressa tramite un'**equazione di ricorrenza**, ovvero una formula matematica definita in maniera ricorsiva appunto. Ad esempio come abbiamo visto la complessità del merge sort è:

MergeSort

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) & n > 1 \\ \Theta(1) & n \leq 1 \end{cases}$$

Il nostro obiettivo è quello di ottenere una **formula chiusa** (cioè senza ricorsione) che rappresenti la classe di complessità della funzione, come la seguente:

MergeSort

$$T(n) = \Theta(n \log n)$$

Le equazioni di ricorrenza possono essere usate anche per risolvere dei veri e propri problemi, ad esempio:

Problema

Un bambino scende una scala composta da n scalini. Ad ogni passo, può decidere di fare 1,2,3,4 scalini alla volta. Determinare in quanti modi diversi può scendere le scale. Ad esempio, se $n = 7$, alcuni dei modi possibili sono i seguenti:

- 1,1,1,1,1,1,1
- 1,2,4
- 4,2,1
- 2,2,2,1
- 1,2,2,1,1

Soluzione

Sia $M(n)$ il numero di modi in cui è possibile scegliere n scalini; allora $M(n)$ può essere espresso nel modo seguente:

$$M(n) = \begin{cases} 0 & n < 0 \\ 1 & n = 0 \\ \sum_{k=1}^{4} M(n-k) & n > 0 \end{cases}$$

Questa ricorrenza può essere trasformata in un algoritmo tramite semplice ricorsione o tramite programmazione dinamica.

Per poter risolvere queste equazioni di ricorrenza e ottenere formule chiuse esistono tre metodi:

- **Analisi per livelli** (chiamato anche **metodo dell'albero di ricorsione**)
  Converte la ricorrenza in un albero i cui nodi rappresentano i costi ai vari livelli della ricorsione
- **Analisi per tentativi** (o **per sostituzione**)
  Ipotizziamo una soluzione e utilizziamo l'induzione matematica per dimostrare che la nostra ipotesi è corretta
- **Metodo dell'esperto** (o **delle ricorrenze comuni**)
  Fornisce una soluzione per ricorrenze della forma:
  $T(n) = a \cdot T(\frac{N}{b}) + f(n) \; con \; a > 0, \; b > 1$

Analisi per livelli

Nel metodo di **analisi per livelli**, oppure **metodo dell'albero di ricorsione**, "srotoliamo" la ricorrenza in un albero i cui i nodi rappresentano i costi ai vari livelli della ricorsione, quindi andremo a sommare il costo di ogni livello (contando che la dimensione dell'input diminuirà ad ogni chiamata).
Il risultato sarà una sommatoria, la quale trattata bene con trucchetti matematici, ci darà in forma polinomiale il costo dell'algoritmo, vediamo qualche esempio:

Primo esempio

$$T(n) = \begin{cases} T(n/2) + b & n > 1 \\ c & n \leq 1 \end{cases}$$

È possibile risolvere questa ricorrenza nel modo seguente:

$$\begin{aligned} T(n) &= b + T(n/2) \\ &= b + b + T(n/4) \\ &= b + b + b + T(n/8) \\ &= \dots \\ &= \underbrace{b + b + \dots + b}_{\log n} + T(1) \end{aligned}$$

Assumiamo per semplicità: $n = 2^k$, ovvero $k = \log n$

$T(n) = b \log n + c = \Theta(\log n)$

Secondo esempio

$$T(n) = \begin{cases} 4T(n/2) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

È possibile risolvere questa ricorrenza nel modo seguente:

$$\begin{aligned} T(n) &= n + 4T(n/2) \\ &= n + 4n/2 + 16T(n/2^2) \\ &= n + 2n + 16n/4 + 64T(n/8) \\ &= \dots \\ &= n + 2n + 4n + 8n + \dots + 2^{\log n - 1} n + 4^{\log n} T(1) \\ &= n \sum_{j=0}^{\log n - 1} 2^j + 4^{\log n} \end{aligned}$$

$$T(n) = n \cancel{\sum_{j=0}^{\log n - 1} 2^j} \quad n \cdot \frac{2^{\log n} - 1}{2 - 1} \qquad + 4^{\log n}$$

Serie geometrica finita:

$$\forall x \neq 1 : \sum_{j=0}^{k} x^j = \frac{x^{k+1} - 1}{x - 1}$$

$$T(n) = n \cancel{\sum_{j=0}^{\log n - 1} 2^j} \quad \cancel{n \cdot \frac{2^{\log n} - 1}{2 - 1}} \quad n(n-1) + 4^{\log n}$$

Passaggi algebrici

$$T(n) = n \cancel{\sum_{j=0}^{\log n - 1} 2^j} \quad \cancel{n \cdot \frac{2^{\log n} - 1}{2 - 1}} \quad n(n-1) + \cancel{4^{\log n}} \quad n^{\log 4}$$

Cambiamento di base:

$$\log_b n = (\log_b a) \cdot (\log_a n) \Rightarrow$$
$$a^{\log_b n} = n^{\log_b a}$$

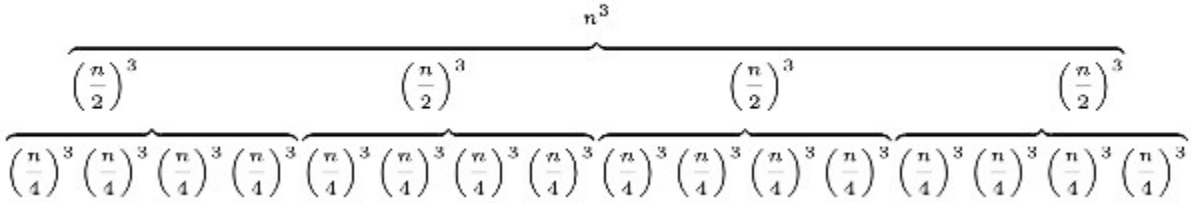
$$T(n) = n \cancel{\sum_{j=0}^{\log n - 1} 2^j} \quad \cancel{n \cdot \frac{2^{\log n} - 1}{2 - 1}} \quad n(n-1) + \cancel{4^{\log n}} \quad \cancel{n^{\log 4}} \quad n^2$$

$$= 2n^2 - n = \Theta(n^2)$$

Terzo esempio

$$T(n) = \begin{cases} 4T(n/2) + n^3 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Proviamo a visualizzare l'albero delle chiamate, per i primi tre livelli:



Livello	**Dim.**	**Costo chiam.**	**N. chiamate**	**Costo livello**
0	n	n^3	1	n^3
1	$n/2$	$(n/2)^3$	4	$4(n/2)^3$
2	$n/4$	$(n/4)^3$	16	$16(n/4)^3$
$\cdots$	$\cdots$	$\cdots$	$\cdots$	$\cdots$
i	$n/2^i$	$(n/2^i)^3$	4^i	$4^i(n/2^i)^3$
$\cdots$	$\cdots$	$\cdots$	$\cdots$	$\cdots$
$\ell - 1$	$n/2^{\ell-1}$	$(n/2^{\ell-1})^3$	$4^{\ell-1}$	$4^{\ell-1}(n/2^{\ell-1})^3$
$\ell = \log n$	1	$T(1)$	$4^{\log n}$	$4^{\log n}$

La sommatoria dà origine a:

$$T(n) = \sum_{i=0}^{\log n - 1} 4^i \cdot n^3/2^{3i} + 4^{\log n}$$

$$= n^3 \sum_{i=0}^{\log n - 1} \frac{2^{2i}}{2^{3i}} + 4^{\log n}$$

Passaggi algebrici

$$= n^3 \sum_{i=0}^{\log n - 1} \left(\frac{1}{2}\right)^i + 4^{\log n}$$

Passaggi algebrici

La sommatoria dà origine a:

$$T(n) = n^3 \sum_{i=0}^{\log n - 1} \left(\frac{1}{2}\right)^i + 4^{\log n}$$

$$= n^3 \sum_{i=0}^{\log n - 1} \left(\frac{1}{2}\right)^i + n^2$$

Cambiamento di base

$$\leq n^3 \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i + n^2$$

Estensione della sommatoria

La sommatoria dà origine a:

$$T(n) \leq n^3 \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i + n^2$$

$$= n^3 \cdot \frac{1}{1 - \frac{1}{2}} + n^2$$

$$= 2n^3 + n^2$$

Serie geometrica infinita decrescente:

$$\forall x, |x| < 1 : \sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$$

Abbiamo dimostrato che:

$$T(n) \leq 2n^3 + n^2$$

- Possiamo affermare che $T(n) = O(n^3)$
- La dimostrazione precedente non afferma che $T(n) = \Theta(n^3)$, perché ad un certo punto siamo passati a $\leq$
- Però è possibile notare che $T(n) \geq n^3$, quindi è possibile affermare che $T(n) = \Omega(n^3)$ e quindi $T(n) = \Theta(n^3)$

Quarto esempio

$$T(n) = \begin{cases} 4T(n/2) + n^2 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Livello	Dimensione	Costo chiamata	N. chiamate	Costo livello
0	n	n^2	1	n^2
1	$n/2$	$(n/2)^2$	4	$4(n/2)^2$
2	$n/4$	$(n/4)^2$	16	$16(n/4)^2$
$\cdots$	$\cdots$	$\cdots$	$\cdots$	$\cdots$
i	$n/2^i$	$(n/2^i)^2$	4^i	$4^i(n/2^i)^2$
$\cdots$	$\cdots$	$\cdots$	$\cdots$	$\cdots$
$\ell - 1$	$n/2^{\ell-1}$	$(n/2^{\ell-1})^2$	$4^{\ell-1}$	$4^{\ell-1}(n/2^{\ell-1})^2$
$\ell = \log n$	1	$T(1)$	$4^{\log n}$	$4^{\log n}$

$$T(n) = \begin{cases} 4T(n/2) + n^2 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

$$\begin{aligned} T(n) &= \sum_{i=0}^{\log n - 1} n^2/2^{2i} \cdot 4^i + 4^{\log n} \\ &= n^2 \sum_{i=0}^{\log n - 1} \frac{2^{2i}}{2^{2i}} + n^2 \\ &= n^2 \sum_{i=0}^{\log n - 1} 1 + n^2 \\ &= n^2 \log n + n^2 = \Theta(n^2 \log n) \end{aligned}$$

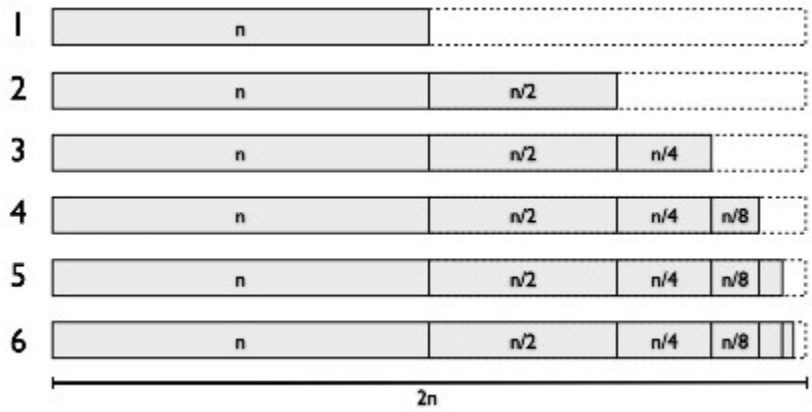
Analisi per tentativi

L’analisi per livelli richiede una approfondita conoscenza delle serie matematiche e certe ricorrenze risultano complicate perché non seguono lo schema che abbiamo visto. Introduciamo quindi l’**analisi per tentativi**, o **metodo di sostituzione**, questo metodo consiste nell’**indovinare** una soluzione (in base all’esperienza si riesce a stimare più o meno bene la complessità di un algoritmo) e successivamente **dimostrare** che la complessità è corretta tramite **induzione**.

Vediamo qualche esempio:

Cerchiamo di indovinare

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



$$T(n) = n \cdot \sum_{i=0}^{\log n} (1/2)^i \leq n \cdot \sum_{i=0}^{\infty} (1/2)^i = n \cdot \frac{1}{1 - \frac{1}{2}} = 2n$$

Serie geometrica decrescente infinita

$$\forall x, |x| < 1 : \sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$$

Grazie all'analisi per livelli sappiamo che questo algoritmo ha complessità $O(n)$, facciamo finta di averlo "indovinato" e proviamo a dimostrarlo:

Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = O(n)$

$\exists c > 0, \exists m \geq 0 : T(n) \leq cn, \forall n \geq m$

- **Caso base**: Dimostriamo la disequazione per $T(1)$

  $T(1) = 1 \overset{?}{\leq} 1 \cdot c \Leftrightarrow \forall c \geq 1$

- **Ipotesi induttiva**: $\forall k < n : T(k) \leq ck$.
- **Passo di induzione**: Dimostriamo la disequazione per $T(n)$

$$
\begin{aligned}
T(n) &= T(\lfloor n/2 \rfloor) + n \\
&\leq c\lfloor n/2 \rfloor + n && \text{Sostituzione} \\
&\leq cn/2 + n && \text{Intero inferiore} \\
&= (c/2 + 1)n && \text{Passo algebrico} \\
&\overset{?}{\leq} cn && \text{Obiettivo} \\
&\Leftrightarrow c/2 + 1 \leq c \Leftrightarrow c \geq 2 && \text{Risultato finale}
\end{aligned}
$$

- Abbiamo provato che $T(n) \leq cn$
  - Nel caso base: $c \geq 1$
  - Nel passo induttivo: $c \geq 2$
  - Un valore c che rispetta entrambe le disequazioni è $c = 2$
- Questo vale per $n = 1$, e per tutti i valori di n seguenti
  - Quindi $m = 1$

Abbiamo quindi provato che $T(n) = O(n)$

Limite inferiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \Omega(n)$

$\exists d > 0, \exists m \geq 0 : T(n) \geq dn, \forall n \geq m$

- **Caso base**: Dimostriamo la disequazione per $T(1)$

$T(1) = 1 \overset{?}{\geq} 1 \cdot d \Leftrightarrow \forall d \leq 1$

- **Ipotesi induttiva**: $\forall k < n : T(k) \geq dk$.
- **Passo di induzione**: Dimostriamo la disequazione per $T(n)$

$$
\begin{aligned}
T(n) &= T(\lfloor n/2 \rfloor) + n \\
&\geq d\lfloor n/2 \rfloor + n && \text{Sostituzione} \\
&\geq dn/2 - 1 + n && \text{Intero inferiore} \\
&= \left(\frac{d}{2} - \frac{1}{n} + 1\right) n \overset{?}{\geq} dn && \text{Passo algebrico} \\
&\Leftrightarrow \frac{d}{2} - \frac{1}{n} + 1 \geq d \Leftrightarrow d \leq 2 - 2/n && \text{Risultato finale}
\end{aligned}
$$

- Abbiamo provato che $T(n) \geq dn$
  - Nel caso base: $d \leq 1$
  - Nel passo induttivo: $d \leq 2 - \frac{2}{n}$
  - Un valore d che rispetta entrambe le disequazioni, per ogni valore di $n \geq 2$, è $d = 1$
- Questo vale per $n = 2$, e per tutti i valori di n seguenti
  - Quindi $m = 2$

Abbiamo quindi provato che $T(n) = \Omega(n)$

$T(n) = O(n) \wedge T(n) = \Omega(n) \Leftrightarrow T(n) = \Theta(n)$

- É possibile dimostrare che $T(n) = \Omega(n)$ in maniera molto più semplice, senza fare nemmeno ricorso all'ipotesi induttiva.

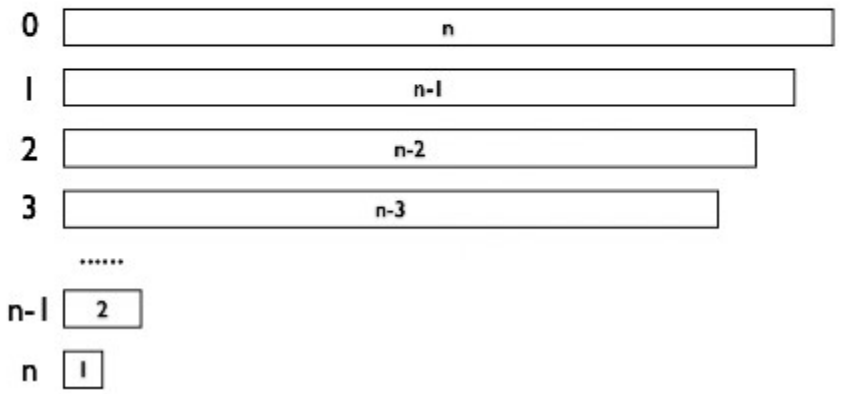
$$T(n) = T(\lfloor n/2 \rfloor) + n \geq n \overset{?}{\geq} dn$$

- L'ultima equazione è vera per $d \leq 1$, condizione identica a quella del caso base

Errore nell'intuizione

Vediamo ora cosa succede quando si sbaglia l'intuizione, ovvero indoviniamo "male" e abbiamo problemi con la dimostrazione:

$$T(n) = \begin{cases} T(n-1) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



$$T(n) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$$

$$T(n) = \begin{cases} T(n-1) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo sbagliato: $T(n) = O(n)$

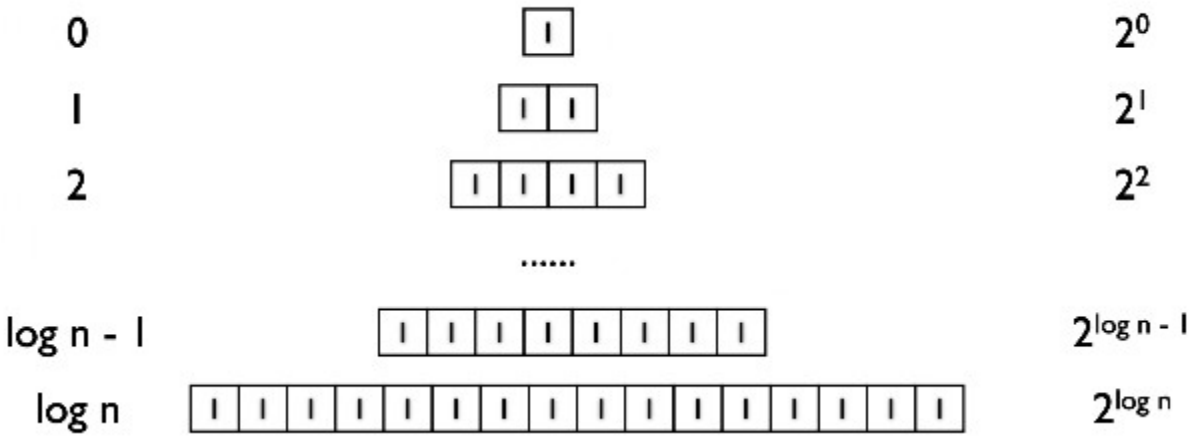
$\exists c > 0, \exists m \geq 0 : T(n) \leq cn, \forall n \geq m$

- **Ipotesi induttiva**: $\forall k < n : T(k) \leq ck$.
- **Passo di induzione**: Dimostriamo la disequazione per $T(n)$

$$\begin{aligned} T(n) &= T(n-1) + n \\ &\leq c(n-1) + n && \text{Sostituzione} \\ &\leq (c+1)n - c && \text{Passo algebrico} \\ &\leq (c+1)n && \text{Rimozione elemento negativo} \\ &\overset{?}{\leq} cn && \text{Obiettivo} \\ &\Rightarrow c + 1 \leq c && \text{Impossibile} \end{aligned}$$

Difficoltà matematica – Limite superiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$



$$T(n) = \sum_{i=0}^{\log n} 2^i = 1 + 2 + \ldots + n/4 + n/2 + n = O(n)$$

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = O(n)$
$\exists c > 0, \exists m \geq 0 :$
$T(n) \leq cn, \forall n \geq m$

- **Ipotesi induttiva**: $\forall k < n : T(k) \leq ck$.
- **Passo di induzione**: Dimostriamo la disequazione per $T(n)$

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \\ &\leq c\lfloor n/2 \rfloor + c\lceil n/2 \rceil + 1 && \text{Sostituzione} \\ &= cn + 1 && \text{Passo algebrico} \\ &\overset{?}{\leq} cn && \text{Obiettivo} \\ &\Rightarrow 1 \leq 0 && \text{Impossibile} \end{aligned}$$

Cosa succede?

- Il tentativo è corretto...
- ma non riusciamo a dimostrarlo per un termine di ordine inferiore

$$cn + 1 \leq cn$$

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = O(n)$
$\exists c > 0, \exists m \geq 0 :$
$T(n) \leq cn, \forall n \geq m$

- **Ipotesi induttiva più stretta**: $\exists b > 0, \forall k < n : T(k) \leq ck - b$.
- **Passo di induzione**: Dimostriamo la disequazione per $T(n)$:

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \\ &\leq c\lfloor n/2 \rfloor - b + c\lceil n/2 \rceil - b + 1 && \text{Sostituzione} \\ &= cn - 2b + 1 && \text{Passo algebrico} \\ &\overset{?}{\leq} cn - b && \text{Obiettivo} \\ &\Rightarrow -2b + 1 \leq -b && \text{Eliminazione } cn \\ &\Rightarrow b \geq 1 && \text{Passo algebrico} \end{aligned}$$

- **Caso base**: Dimostriamo la disequazione per $T(1)$

$$T(1) = 1 \overset{?}{\leq} 1 \cdot c - b \Leftrightarrow \forall c \geq b + 1$$

- Abbiamo provato che $T(n) \leq cn - b \leq cn$
  - Nel passo induttivo: $\forall b \geq 1, \forall c$
  - Nel caso base: $\forall c \geq b + 1$
  - Una coppia di valori b, c che rispettano queste disequazioni sono $b = 1$, $c = 2$
- Questo vale per $n = 1$, e per tutti i valori di n seguenti
  - Quindi $m = 1$

Abbiamo quindi provato che $T(n) = O(n)$

Difficoltà matematica – Limite inferiore

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = \Omega(n)$

$\exists d > 0, \exists m \geq 0 :$

$T(n) \geq dn, \forall n \geq m$

- **Passo di induzione**: Dimostriamo la disequazione per $T(n)$

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 \\ &\geq d\lfloor n/2 \rfloor + d\lceil n/2 \rceil + 1 && \text{Sostituzione} \\ &= dn + 1 \overset{?}{\geq} dn && \text{Vero per ogni } d \end{aligned}$$

- **Caso base**: Dimostriamo la disequazione per $T(1)$

$T(n) = 1 \geq d \cdot 1 \Leftrightarrow d \leq 1$

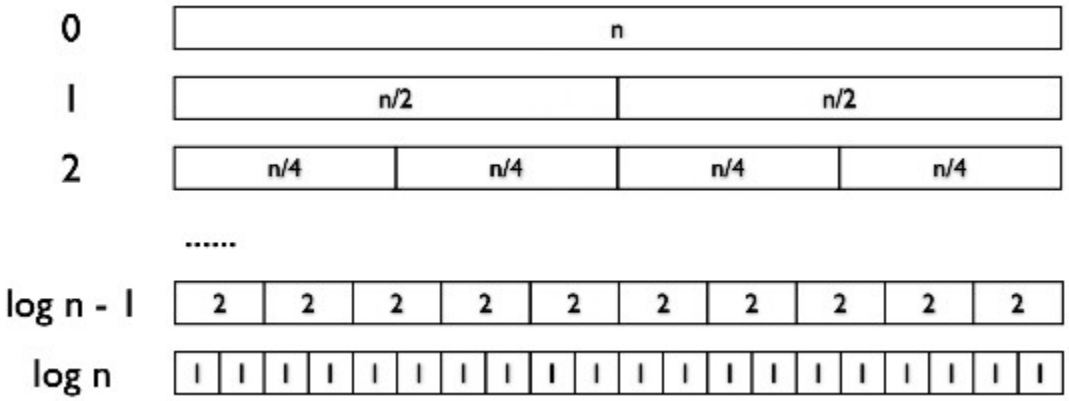
Abbiamo quindi provato che $T(n) = \Omega(n)$

Per capire se l'intuizione è sbagliata oppure ci sono solo difficoltà matematiche dobbiamo guardare di che **grado** è il termine che rende impossibile la nostra disequazione.
Se il termine è **un grado inferiore** rispetto ad n allora abbiamo difficoltà matematiche, e possiamo eliminarle aggiungendo una sottrazione di un termine di grado inferiore (ad esempio $-b$, oppure $-kb$, oppure $-k^2 b$ e cosi via), altrimenti è impossibile.

Difficoltà matematiche nel caso base

Problemi con i casi base

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$



$$T(n) = O(n \log n)$$

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Tentativo: $T(n) = O(n \log n)$
$\exists c > 0, \exists m \geq 0 :$
$T(n) \leq cn \log n, \forall n \geq m$

- **Ipotesi induttiva**: $\exists c > 0, \forall k < n : T(k) \leq ck \log k$.
- **Passo di induzione**: Dimostriamo la disequazione per $T(n)$:

$$\begin{aligned} T(n) &= 2T(\lfloor n/2 \rfloor) + n \\ &\leq 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n && \text{Sostituzione} \\ &\leq 2cn/2 \log n/2 + n && \text{Intero inferiore} \\ &= cn(\log n - 1) + n && \text{Passo algebrico} \\ &= cn \log n - cn + n && \text{Passo algebrico} \\ &\overset{?}{\leq} cn \log n && \text{Obiettivo} \end{aligned}$$

$$T(n) \leq cn\log n - cn + n \overset{?}{\leq} cn\log n \qquad \text{Obiettivo}$$
$$\Rightarrow -cn + n \leq 0 \qquad \text{Eliminazione } cn\log n$$
$$\Rightarrow c \geq 1 \qquad \text{Passo algebrico}$$

- **Caso base**: Dimostriamo la disequazione per $T(1)$

$$T(1) = 1 \overset{?}{\leq} 1 \cdot c\log 1 = 0 \Rightarrow 1 \not\leq 0$$

Cosa succede?

- È falso, ma non è un problema: non a caso si chiama notazione asintotica.
- Il valore iniziale di m lo possiamo scegliere noi

- **Caso base**: Dimostriamo la disequazione per $T(2)$, $T(3)$:

$$T(2) = 2T(\lfloor 2/2 \rfloor) + 2 = 4 \leq 1 \cdot c \cdot 2\log 2 \Leftrightarrow c \geq 2$$
$$T(3) = 2T(\lfloor 3/2 \rfloor) + 3 = 5 \leq 1 \cdot c \cdot 3\log 3 \Leftrightarrow c \geq \frac{5}{3\log 3}$$
$$T(4) = 2T(\lfloor 4/2 \rfloor) + 4 = 2T(\lfloor 2 \rfloor) + 4$$

- Non è necessario provare la terza disequazione, in quanto viene espressa in base a casi base diversi da $T(1)$ che sono già stati dimostrati e quindi possono costituire la base della nostra induzione.
- Abbiamo provato che $T(n) \leq cn\log n$
  - Nel passo induttivo: $\forall c \geq 1$
  - Nel caso base: $\forall c \geq 2, c \geq \frac{5}{3\log 3}$
  - Visto che sono tutti disequazioni con il segno $\geq$, è sufficiente utilizzare un valore $c \geq \max\{1, 2, \frac{5}{3\log 3}\}$
- Questo vale per $n = 2, n = 3$, e per tutti i valori di n seguenti
  - Quindi $m = 2$

Se falliamo il caso base, vuol dire che non è verificata per quel valore, ma essendo una notazione **asintotica**, noi possiamo partire dal valore che vogliamo, quindi iniziamo ad alzare il valore fino a quando non ne troviamo uno verificato.

Facciamo attenzione però che alzando il caso base, rischiamo che sui successivi valori abbiamo dei casi non verificati e non compresi nell'induzione, che dovremo verificare noi **a mano**.

together now!

$$T(n) = \begin{cases} 9T(\lfloor n/3 \rfloor) + n & n > 1 \\ 1 & n \leq 1 \end{cases} \quad \begin{array}{l} \text{Tentativo: } T(n) = O(n^2) \\ \exists c > 0, \exists m \geq 0 : T(n) \leq cn^2, \forall n \geq m \end{array}$$

- **Ipotesi induttiva**: $\exists c > 0 : T(k) \leq ck^2, \forall k < n$
- **Passo induttivo**: Dimostriamo la disequazione per $T(n)$

$$\begin{aligned} T(n) &= 9T(\lfloor n/3 \rfloor) + n \\ &\leq 9c(\lfloor n/3 \rfloor)^2 + n && \text{Sostituzione} \\ &\leq 9c(n^2/9) + n && \text{Limite inferiore} \\ &= cn^2 + n && \text{Passo algebrico} \\ &\not\leq cn^2 && \text{Falso} \end{aligned}$$

- **Ipotesi induttiva**: $\exists c > 0 : T(k) \leq c(k^2 - k), \forall k < n$
- **Passo induttivo**: Dimostriamo la disequazione per $T(n)$

$$\begin{aligned} T(n) &= 9T(\lfloor n/3 \rfloor) + n \\ &\leq 9c\left(\lfloor n/3 \rfloor^2 - \lfloor n/3 \rfloor\right) + n && \text{Sostituzione} \\ &\leq cn^2 - 3cn + n && \text{Limite inferiore} \\ &\overset{?}{\leq} cn^2 - cn && \text{Obiettivo} \\ &\Leftrightarrow c \geq \frac{1}{2} \end{aligned}$$

- **Ipotesi induttiva**: $\exists c > 0 : T(k) \leq c(k^2 - k), \forall k < n$
- **Passo base**:
  - $T(1) = 1 \leq c(1^2 - 1) = 0$, falso
  - $T(2) = 9T(0) + 2 = 11 \leq c(4 - 2) \Leftrightarrow c \geq 11/2$
  - $T(3) = 9T(1) + 3 = 12 \leq c(9 - 3) \Leftrightarrow c \geq 12/6$
  - $T(4) = 9T(1) + 4 = 13 \leq c(16 - 4) \Leftrightarrow c \geq 13/12$
  - $T(5) = 9T(1) + 5 = 14 \leq c(25 - 5) \Leftrightarrow c \geq 14/20$
  - $T(6) = 9T(2) + 6$
- Non è necessario andare oltre, perchè $T(6)$ dipende da $T(2)$ che è già stato dimostrato
- Parametri:
  - $c \geq \max\left\{\frac{1}{2}, \frac{11}{2}, \frac{12}{6}, \frac{13}{12}, \frac{14}{20}\right\}$
  - $m = 2$
- Notare che l'esempio combina le due difficoltà insieme, ma è artificiale:
  - Se avessimo scelto come ipotesi più stretta $T(n) \leq cn^2 - bn$, il problema sui casi base non si sarebbe posto

Riassumendo:

- Si **indovina** una possibile soluzione e si forma un'ipotesi induttiva in base a quella
- Si **sostituisce** nella ricorrenza le espressioni del tipo $T(\ldots)$, utilizzando l'ipotesi induttiva, e si verifica che il passo induttivo sia verificato
- Si **dimostra** che l'ipotesi è valida anche nel caso base
- Bisogna fare attenzione
  - Ad ipotizzare soluzione troppo strette
  - Ad alcuni casi particolari che richiedono qualche giochetto matematico
  - Ai casi base (il logaritmo può complicare le cose)

Metodo dell'esperto

Nel **metodo dell'esperto**, chiamato anche **metodo delle ricorrenze comuni**, riconosciamo che la nostra ricorrenza appartiene ad una classe particolare di ricorrenze (in base alla forma) e attraverso dei **teoremi** possiamo calcolarne facilmente la complessità.

Ricorrenze lineari con partizione bilanciata

Teorema

Siano a e b costanti intere tali che $a \geq 1$ e $b \geq 2$, e c, β costanti reali tali che $c > 0$ e $\beta \geq 0$. Sia $T(n)$ data dalla relazione di ricorrenza:

$$T(n) = \begin{cases} aT(n/b) + cn^\beta & n > 1 \\ d & n \leq 1 \end{cases}$$

Posto $\alpha = \log a / \log b = \log_b a$, allora:

$$T(n) = \begin{cases} \Theta(n^\alpha) & \alpha > \beta \\ \Theta(n^\alpha \log n) & \alpha = \beta \\ \Theta(n^\beta) & \alpha < \beta \end{cases}$$

Per la dimostrazione assumiamo che n sia una potenza intera di b, ovvero $n = b^k$ e quindi $k = \log_b n$. Questo non influisce sul risultato in quanto anche se l'input avesse dimensione $b^k + 1$, dovremmo estenderlo fino alla dimensione b^{k+1} (con il padding). Quindi estendiamo l'input di un fattore costante b, il quale è ininfluente per il calcolo della complessità.

$$T(n) = aT(n/b) + cn^\beta \qquad T(1) = d$$

Liv.	Dim.	Costo chiam.	N. chiamate	Costo livello
0	b^k	$cb^{k\beta}$	1	$cb^{k\beta}$
1	b^{k-1}	$cb^{(k-1)\beta}$	a	$acb^{(k-1)\beta}$
2	b^{k-2}	$cb^{(k-2)\beta}$	a^2	$a^2cb^{(k-2)\beta}$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
i	b^{k-i}	$cb^{(k-i)\beta}$	a^i	$a^icb^{(k-i)\beta}$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$k-1$	b	cb^β	a^{k-1}	$a^{k-1}cb^\beta$
k	1	d	a^k	da^k

Sommando i costi totali di tutti i livelli, si ottiene:

$$T(n) = \quad da^k + cb^{k\beta} \sum_{i=0}^{k-1} \frac{a^i}{b^{i\beta}} = \quad da^k + cb^{k\beta} \sum_{i=0}^{k-1} \left(\frac{a}{b^\beta}\right)^i$$

Osservazioni

- $a^k = a^{\log_b n} = a^{\log n / \log b} = 2^{\log a \log n / \log b} = n^{\log a / \log b} = n^\alpha$
- $\alpha = \log a / \log b \Rightarrow \alpha \log b = \log a \Rightarrow \log b^\alpha = \log a \Rightarrow a = b^\alpha$
- Poniamo $q = \frac{a}{b^\beta} = \frac{b^\alpha}{b^\beta} = b^{\alpha-\beta}$

$$T(n) = da^k + cb^{k\beta} \sum_{i=0}^{k-1} \left(\frac{a}{b^\beta}\right)^i = dn^\alpha + cb^{k\beta} \sum_{i=0}^{k-1} q^i$$

Caso 1: $\alpha > \beta$

Ne segue che: $q = b^{\alpha-\beta} > 1$:

$$\begin{aligned}
T(n) &= dn^\alpha + cb^{k\beta} \textstyle\sum_{i=0}^{k-1} q^i \\
&= n^\alpha d + cb^{k\beta}[(q^k - 1)/(q - 1)] && \text{Serie geometrica finita} \\
&\leq n^\alpha d + cb^{k\beta} q^k/(q-1) && \text{Disequazione} \\
&= n^\alpha d + \frac{cb^{k\beta}a^k}{b^{k\beta}}/(q-1) && \text{Sostituzione } q \\
&= n^\alpha d + ca^k/(q-1) && \text{Passi algebrici} \\
&= n^\alpha[d + c/(q-1)] && a^k = n^\alpha, \text{raccolta termini}
\end{aligned}$$

- Quindi $T(n)$ è $O(n^\alpha)$.
- Per via della componente dn^α, $T(n)$ è anche $\Omega(n^\alpha)$, e quindi $T(n) = \Theta(n^\alpha)$.

Caso 2: $\alpha = \beta$

Ne segue che: $q = b^{\alpha-\beta} = 1$:

$$\begin{aligned}
T(n) &= dn^\alpha + cb^{k\beta}\textstyle\sum_{i=0}^{k-1} q^i \\
&= n^\alpha d + cn^\beta k && q^i = 1^i = 1 \\
&= n^\alpha d + cn^\alpha k && \alpha = \beta \\
&= n^\alpha (d + ck) && \text{Raccolta termini} \\
&= n^\alpha [d + c \log n / \log b] && k = \log_b n
\end{aligned}$$

e quindi $T(n)$ è $\Theta(n^\alpha \log n)$;

Caso 3: $\alpha < \beta$

Ne segue che: $q = b^{\alpha-\beta} < 1$:

$$\begin{aligned}
T(n) &= dn^\alpha + cb^{k\beta}\textstyle\sum_{i=0}^{k-1} q^i \\
&= n^\alpha d + cb^{k\beta}[(q^k - 1)/(q - 1)] && \text{Serie geometrica finita} \\
&= n^\alpha d + cb^{k\beta}[(1 - q^k)/(1 - q)] && \text{Inversione} \\
&\leq n^\alpha d + cb^{k\beta}[1/(1 - q)] && \text{Disequazione} \\
&= n^\alpha d + cn^\beta/(1 - q) && b^k = n
\end{aligned}$$

- Quindi $T(n)$ è $O(n^\beta)$.
- Poichè $T(n) = \Omega(n^\beta)$ per il termine non ricorsivo, si ha che $T(n) = \Theta(n^\beta)$.

Forma estesa per le ricorrenze lineari con partizione bilanciata

Teorema

Sia $a \geq 1$, $b > 1$, $f(n)$ asintoticamente positiva, e sia

$$T(n) = \begin{cases} aT(n/b) + f(n) & n > 1 \\ d & n \leq 1 \end{cases}$$

Sono dati tre casi:

(1)	$\exists \epsilon > 0 : f(n) = O(n^{\log_b a - \epsilon})$	$\Rightarrow$	$T(n) = \Theta(n^{\log_b a})$
(2)	$f(n) = \Theta(n^{\log_b a})$	$\Rightarrow$	$T(n) = \Theta(f(n) \log n)$
(3)	$\exists \epsilon > 0 : f(n) = \Omega(n^{\log_b a + \epsilon}) \wedge$ $\exists c : 0 < c < 1, \exists m \geq 0 :$ $af(n/b) \leq cf(n), \forall n \geq m$	$\Rightarrow$	$T(n) = \Theta(f(n))$

A parole, cosa ci dice questo teorema? Se si confronta $n^{\log_b a}$, si ottiene:

- se $n^{\log_b a}$ è più grande di $f(n)$ di un fattore polinomiale n^ϵ, allora $T(n)$ è dominato da $n^{\log_b a}$;
- se $n^{\log_b a}$ è $\Theta(n^{\log_b a})$, allora nessuno dei due domina, e viene aggiunto un fattore logaritmico;
- se $n^{\log_b a}$ è più piccolo di $f(n)$ di un fattore polinomiale n^ϵ, allora $T(n)$ è dominato da $f(n)$.

Alcuni esempi

Ricorrenza	a	b	$\log_b a$	Caso	Funzione
$T(n) = 9T(n/3) + n \log n$	9	3	2	(1)	$T(n) = \Theta(n^2)$

$f(n) = n \log n = O(n^{\log_b a - \epsilon}) = O(n^{2-\epsilon})$, con $\epsilon < 1$

Ricorrenza	a	b	$\log_b a$	Caso	Funzione
$T(n) = T(2n/3) + 1$	1	$\frac{3}{2}$	0	(2)	$T(n) = \Theta(\log n)$

$f(n) = n^0 = \Theta(n^{\log_b a}) = \Theta(n^0)$

Ricorrenza	a	b	$\log_b a$	Caso	Funzione
$T(n) = 3T(n/4) + n \log n$	3	4	≈ 0.79	(3)	$T(n) = \Theta(n \log n)$

$f(n) = n \log n = \Omega(n^{\log_4 3+\epsilon})$, con $\epsilon < 1 - \log_4 3 \approx 0.208$

Dobbiamo dimostrare che:
$\exists c \leq 1, \exists m \geq 0 : af(n/b) \leq cf(n), \forall n \geq m$

$$\begin{aligned} af(n/b) &= 3(n/4 \log n/4) \\ &= 3/4n(\log n - \log 4) \\ &\leq 3/4n \log n \\ &\overset{?}{\leq} cn \log n \end{aligned}$$

L'ultima disequazione è soddisfatta da $c = 3/4$ e qualsiasi m.

Ricorrenza	a	b	$\log_b a$	Caso	Funzione
$T(n) = 2T(n/2) + n \log n$	2	2	1	–	Non applicabile

$f(n) = n \log n \neq O(n^{1-\epsilon})$, con $\epsilon > 0$
$f(n) = n \log n \neq \Theta(n)$
$f(n) = n \log n \neq \Omega(n^{1+\epsilon})$, con $\epsilon > 0$

Nessuno dei tre casi è applicabile e bisogna utilizzare altri metodi.

I teoremi che abbiamo appena visto non considerano equazioni di ricorrenza con interi inferiori/superiori, questo dovrebbe essere scorretto in quanto le equazioni di ricorrenza dovrebbero sempre riferirsi a interi inferiori/superiori trattandosi delle dimensioni dell'input. Però i risultati dei teoremi valgono allo stesso modo anche quando le equazioni di ricorrenza sono espresse tramite interi superiori/inferiori

Teorema

Siano $a_1, a_2, \ldots, a_h$ costanti intere non negative, con h costante positiva, c e β costanti reali tali che $c > 0$ e $\beta \geq 0$, e sia $T(n)$ definita dalla relazione di ricorrenza:

$$T(n) = \begin{cases} \sum_{1 \leq i \leq h} a_i T(n-i) + cn^\beta & n > m \\ \Theta(1) & n \leq m \leq h \end{cases}$$

Posto $a = \sum_{1 \leq i \leq h} a_i$, allora:

1. $T(n)$ è $\Theta(n^{\beta+1})$, se $a = 1$,
2. $T(n)$ è $\Theta(a^n n^\beta)$, se $a \geq 2$.

Alcuni esempi

	Ricorrenza	a	β	Caso	Funzione
(A)	$T(n) = T(n-10) + n^2$	1	2	(1)	$T(n) = \Theta(n^3)$
(B)	$T(n) = T(n-2) + T(n-1) + 1$	2	0	(2)	$T(n) = 2^n$

(A) Poiché $a = 1$, il costo è polinomiale.

(B) Poiché $a = 2$, il costo è esponenziale.

Esercizio

Siano

- $T(n) = 7T(n/2) + n^2$ una funzione di costo di un algoritmo A, e
- $T'(n) = aT'(n/4) + n^2$ una funzione di costo di un algoritmo A'.

Qual è il massimo valore intero di a che rende A' asintoticamente più veloce di A?

Poichè $\log_2 7$ è ≈ 2.81, il Master Theorem dice che $T(n) = \Theta(n^{\log_2 7})$.

Utilizzando alcune trasformazioni algebriche, si ottiene che:

$$\log_2 7 = \frac{\log_4 7}{\log_4 2} = \frac{\log_4 7}{1/2}$$
$$= 2\log_4 7 = \log_4 49$$

- $a < 16 \Rightarrow \alpha = \log_4 a < 2 \Rightarrow T'(n) = \Theta(n^2) = O(T(n))$
- $a = 16 \Rightarrow \alpha = \log_4 a = 2 \Rightarrow T'(n) = \Theta(n^2 \log n) = O(T(n))$
- $16 < a \leq 49 \Rightarrow \alpha = \log_4 a > 2 \Rightarrow T'(n) = \Theta(n^\alpha) = O(T(n))$
- $a < 49 \Rightarrow \alpha = \log_4 a > 2 \Rightarrow T'(n) = \Theta(n^\alpha) = \Omega(T(n))$

Esempi di applicazione

9.3 Esempi di applicazione

	Ricorrenza	a	b	$\log_b a$	Caso	Funzione
(1)	$T(n) = 9T(n/3) - n$	9	3	2	(1)	$T(n) = \Theta(n^2)$
(2)	$T(n) = T(2n/3) + 1$	1	$\frac{3}{2}$	0	(2)	$T(n) = \Theta(\log n)$
(3)	$T(n) \quad 3T(n/4) + n\log n$	3	4	≈ 0.79	(3)	$T(n) \quad \Theta(n \log n)$
(4)	$T(n) = 2T(n/2) + n\log n$	2	2	1	-.	Non applicabile

	Ricorrenza	a	β	Caso	Funzione
(5)	$T(n) = T(n-10) + n^2$	1	2	(1)	$T(n) = \Theta(n^3)$
(6)	$T(n) = T(n-2) + T(n-1) + 1$	1	0	(2)	$T(n) = 2^n$

1. $f(n) = n = O(n^{\log_3 2 - \epsilon}) = O(n^{2 - \epsilon})$, con $\epsilon > 1$
2. $f(n) = n^0 = \Theta(n^{\log_4 a}) = \Theta(n^0)$
3. $f(n) = n\log n = \Omega(n^{\log_4 3 + \epsilon})$, con $\epsilon < 1 - \log_4 3 \approx 0.208$

   Dobbiamo anche dimostrare che: $\exists c \leq 1, m \geq: af(n/b) \leq cf(n), \forall n \geq m$

   $$af(n/b) = 3n/4 \log n/4$$
   $$= 3/4n\log n - 3/4n\log 2$$
   $$\leq 3/4n\log n$$
   $$\overset{?}{\leq} cn\log n$$

   L'ultima è diseguazione è soddisfatta da $c = 3/4$ e $m = 1$.
4. $f(n) = n\log n \neq O(n^{1-\epsilon})$, con $\epsilon > 0$

   $f(n) = n\log n \neq \Theta(n)$

   $f(n) = n\log n \neq \Omega(n^{1+\epsilon})$, con $\epsilon > 0$

   Nessuno dei tre casi è applicabile e bisogna utilizzare altri metodi.
5. Poiché $a = 1$, il costo è polinomiale.
6. Poiché $a = 2$, il costo è esponenziale.

Tipo di dato e strutture di dati

Strutture dati astratte

In un linguaggio di programmazione, un **dato** è un valore che una variabile può assumere.
Un **tipo di dato astratto** è un modello matematico, dato da una collezione di valori e un insieme di operazioni ammesse su questi valori.
I **tipi di dato primitivo** sono dei tipi di dato astratto, solo che sono forniti direttamente dal linguaggio (ad esempio $\int$ ¿ con le operazioni $+, -, *, /, \%$)
La **specifica** di un tipo di dato astratto è il **manuale d'uso**, ovvero descrive come si usa ma nasconde i dettagli implementativi. La specifica serve all'utilizzatore del dato.
La **implementazione** invece descrive come avviene effettivamente l'implementazione vera e propria (appoggiandosi su livelli più bassi)

Esempi

Specifica	Implementazione
Numeri reali	IEEE-754
Pile	Pile basate su vettori Pile basate su puntatori
Code	Code basate su vettori circolari Code basate su puntatori

Le **strutture di dati** sono collezioni di dati dove viene data più importanza all'organizzazione della collezione piuttosto che al tipo dei dati contenuti. Una struttura dati è caratterizzata da **un insieme di operatori** che permettono di manipolare la struttura e **un modo sistematico** di organizzare l'insieme di dati.
Una struttura dati può essere:

- **Lineare / non lineare**
  Nel caso in cui sia presente (o non sia presente) un concetto di sequenza
- **Statica / dinamica**
  Nel caso in cui non sia possibile (o sia possibile) una variazione della dimensione o del contenuto
- **Omogenee / disomogenee**
  Nel caso in cui i dati contenuti devono (o non devono) essere tutti dello stesso tipo

Ecco qualche esempio di strutture dati in vari linguaggi:

Tipo	Java	C++	Python
Sequenze	List, Queue, Deque LinkedList, ArrayList, Stack, ArrayDeque	list, forward_list vector stack queue, deque	list tuple
Insiemi	Set TreeSet, HashSet, LinkedHashSet	set unordered_set	set, frozenset
Dizionari	Map HashTree, HashMap, LinkedHashMap	map unordered_map	dict
Alberi	-	-	-
Grafi	-	-	-

Sequenza

Una struttura dati **dinamica, lineare** che rappresenta una sequenza **ordinata** di valori, dove un valore può comparire più di una volta (l'ordine all'interno della sequenza è importante).
Data la **posizione** è possibile aggiungere/togliere elementi.

- $s = s_1, s_2, \ldots, s_n$
- l'elemento s_i è in posizione pos_i
- esistono le posizioni (fittizie) pos_0, pos_{n+1}

È possibile accedere direttamente alla testa (e alla coda in alcune implementazioni) e a tutti gli altri elementi è possibile accedere sequenzialmente (ovvero spostandosi partendo dalla testa/coda).
Ecco la specifica:

SEQUENCE

% Restituisce **true** se la sequenza è vuota
boolean isEmpty()

% Restituisce **true** se p è uguale a pos_0 oppure a pos_{n+1}
boolean finished(POS p)

% Restituisce la posizione del primo elemento
POS head()

% Restituisce la posizione dell'ultimo elemento
POS tail()

% Restituisce la posizione dell'elemento che segue p
POS next(POS p)

% Restituisce la posizione dell'elemento che precede p
POS prev(POS p)

% Inserisce l'elemento v di tipo ITEM nella posizione p.
% Restituisce la posizione del nuovo elemento, che diviene il predecessore di p
POS insert(POS p, ITEM v)

% Rimuove l'elemento contenuto nella posizione p.
% Restituisce la posizione del successore di p,
% che diviene successore del predecessore di p
POS remove(POS p)

% Legge l'elemento di tipo ITEM contenuto nella posizione p
ITEM read(POS p)

% Scrive l'elemento v di tipo ITEM nella posizione p
write(POS p, ITEM v)

Insieme

Una struttura dati **dinamica, non lineare** che memorizza una **collezione non ordinata** di elementi senza valori ripetuti (l'ordinamento fra elementi è dato dall'eventuale relazione d'ordine definita sul tipo degli elementi stessi).
Si possono eseguire le operazioni base di inserimento, cancellazione e di verifica di contenimento. Esistono delle operazioni di ordinamento (ottenere il massimo e ottenere il minimo) e ovviamente le operazioni insiemistiche (unione, intersezione, differenza). Per scorrere tutto l'insieme si usa un **iteratore**.
Ecco la specifica:

SET

% Restituisce la cardinalità dell'insieme
int $= ize()$

% Restituisce **true** se x è contenuto nell'insieme
boolean contains(ITEM x)

% Inserisce x nell'insieme, se non già presente
insert(ITEM x)

% Rimuove x dall'insieme, se presente
remove(ITEM x)

% Restituisce un nuovo insieme che è l'unione di A e B
SET union(SET A, SET B)

% Restituisce un nuovo insieme che è l'intersezione di A e B
SET intersection(SET A, SET B)

% Restituisce un nuovo insieme che è la differenza di A e B
SET difference(SET A, SET B)

Dizionario

Una struttura dati che rappresenta il concetto matematico di **relazione univoca** $R : D \to C$, o associazione chiave-valore:

- L'insieme D è il **dominio** (gli elementi vengono detti chiavi)
- L'insieme C è il **codominio** (gli elementi vengono detti valori)

Si possono eseguire le operazioni di ricerca di un valore associato ad una determinata chiave (con risultato nil se la chiave non è presente), di inserimento di una nuova associazione chiave-valore (cancellando eventuali associazioni precedenti con la stessa chiave) e di rimozione di un'associazione chiave-valore.
Ecco la specifica:

DICTIONARY

% Restituisce il valore associato alla chiave k se presente, **nil** altrimenti
ITEM lookup(ITEM k)

% Associa il valore v alla chiave k
insert(ITEM k, ITEM v)

% Rimuove l'associazione della chiave k
remove(ITEM k)

Dizionario

È un insieme dinamico che implementa le seguenti funzionalità:

- ITEM lookup(ITEM k)
- insert(ITEM k, ITEM v)
- remove(ITEM k)

Possibili implementazioni

Struttura dati	lookup	insert	remove
Vettore ordinato	$O(\log n)$	$O(n)$	$O(n)$
Vettore non ordinato	$O(n)$	$O(1)^*$	$O(1)^*$
Lista non ordinata	$O(n)$	$O(1)^*$	$O(1)^*$

* Assumendo che l'elemento sia già stato trovato, altrimenti $O(n)$

Albero

Un **albero ordinato** è dato da un insieme finito di elementi detti **nodi**. Uno di questi nodi è designato come **radice**. I rimanenti nodi (se esistono) sono partizionati in insiemi **ordinati** e **disgiunti**, ovvero sono altri alberi ordinati (ad esempio albero genealogico).

Grafo

La struttura dati **grafo** è composta da due insiemi:

- Un insieme di elementi detti **nodi** o **vertici**
- Un insieme di coppie di nodi (ordinate oppure no) dette **archi**

Tutte le operazioni su alberi e grafi ruotano attorno alla possibilità di effettuare visite su di essi.

Commenti

- I concetti di sequenza, insieme e dizionario sono collegati
  - Insieme delle chiavi / insieme dei valori
  - Scorrere la sequenza di tutte le chiavi
- Alcune realizzazioni sono "naturali"
  - Implementare una sequenza con una lista
  - Implementare un albero astratto con un albero basato su puntatori
- Esistono tuttavia realizzazioni alternative
  - Implementare un insieme tramite un vettore booleano
  - Implementare un albero tramite un vettore dei padri
- La scelta della struttura dati ha riflessi sull'efficienza e sulle operazioni ammesse
  - Dizionario come hash table: lookup $O(1)$, ricerca minimo $O(n)$

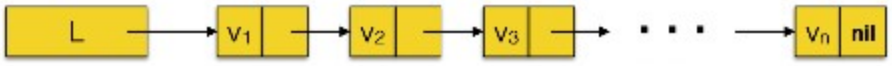
- Dizionario come albero: lookup $O(\log\ n)$, ricerca minimo $O(1)$

Implementazione strutture dati elementari

Lista

Una sequenza di nodi, contenenti dati arbitrari e 1 (o 2) puntatori all'elemento successivo (e/o precedente).
Importante notare che la contiguità di elementi nella lista non significa che c'è contiguità anche in memoria (a differenza dei vettori). Tutte le operazioni hanno costo $O(1)$
Le liste si possono implementare:

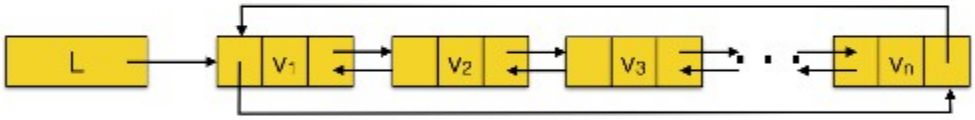
- Bidirezionali o monodirezionali
- Con sentinella o senza sentinella
- Circolare o non circolare



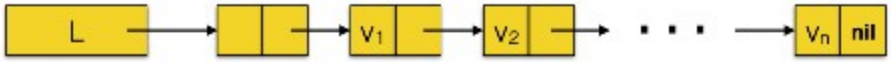
Monodirezionale



Bidirezionale



Bidirezionale circolare



Monodirezionale con sentinella

Ecco l'implementazione di una lista bidirezionale con sentinella:

```
LIST
LIST pred                          % Predecessore
LIST succ                          % Successore
ITEM value                         % Elemento

LIST List()
    LIST t = new LIST
    t.pred = t
    t.succ = t
    return t

boolean isEmpty()
    return pred = succ = this

POS head()
    return succ

POS tail()
    return pred

POS next(POS p)
    return p.succ

POS prev(POS p)
    return p.pred

boolean finished(POS p)
    return (p = this)

ITEM read(POS p)
    return p.value

write(POS p, ITEM v)
    p.value = v

POS insert(POS p, ITEM v)
    LIST t = List()
    t.value = v
    t.pred = p.pred
    p.pred.succ = t
    t.succ = p
    p.pred = t
    return t;

POS p)
    p.pred.succ = p.succ
    p.succ.pred = p.pred
    LIST t = p.succ
    delete p
    return t;
```

List e Pos sono tipi equivalenti

Pila

Una struttura dati dinamica, lineare in cui l'elemento rimosso dall'operazione di cancellazione è predeterminato: "quello che per meno tempo è rimasto nell'insieme" (**LIFO** - Last-in, First-out).
Ecco la specifica:

```
STACK
% Restituisce true se la pila è vuota
boolean isEmpty()

% Inserisce v in cima alla pila
push(ITEM v)

% Estrae l'elemento in cima alla pila e lo restituisce al chiamante
ITEM pop()

% Legge l'elemento in cima alla pila
ITEM top()
```

Dei possibili utilizzi della pila sono:

- Gestione dei record di attivazione (nei linguaggi con procedure/funzioni annidate)
- Nei linguaggi stack-oriented le operazioni prendono gli operandi dallo stack e inseriscono il risultato nello stack.

Possono venir implementate tramite **liste bidirezionali** (con puntatore all'elemento top) o tramite **vettori** (con una dimensione limitata e overhead più basso).
Ecco un'implementazione di una pila basata su un vettore:

```
STACK
ITEM[] A                          % Elementi
int n                             % Cursore
int m                             % Dim. massima

STACK Stack(int dim)
    STACK t = new STACK
    t.A = new int[1...dim]
    t.m = dim
    t.n = 0
    return t

ITEM top()
    precondition: n > 0
    return A[n]

boolean isEmpty()
    return n = 0

ITEM pop()
    precondition: n > 0
    ITEM t = A[n]
    n = n - 1
    return t

push(ITEM v)
    precondition: n < m
    n = n + 1
    A[n] = v
```

Coda

Una struttura dati dinamica, lineare in cui l'elemento rimosso dall'operazione di cancellazione è predeterminato: "quello che per più tempo è rimasto nell'insieme" (**FIFO** - First-in, First-out). Ecco la specifica:

```
QUEUE
% Restituisce true se la coda è vuota
boolean isEmpty()

% Inserisce v in fondo alla coda
enqueue(ITEM v)

% Estrae l'elemento in testa alla coda e lo restituisce al chiamante
ITEM dequeue()

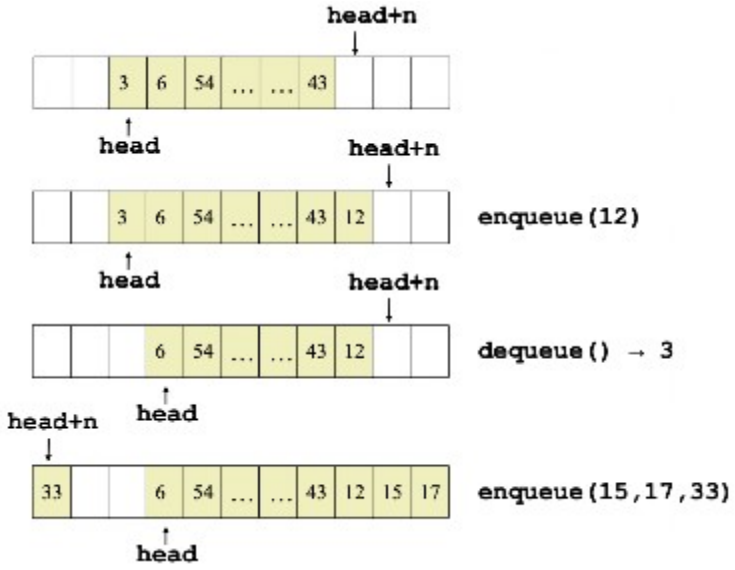
% Legge l'elemento in testa alla coda
ITEM top()
```

Dei possibili utilizzi della coda sono:

- Nei sistemi operativi, i processi in attesa di utilizzare una risorsa vengono gestiti tramite una coda
- La politica FIFO è **fair**

Possono venir implementate tramite **liste monodirezionali** (con puntatore a head per estrazione e puntatore a tail per inserimento) o tramite **vettori circolari** (con dimensione limitata e overhead più basso).

L’implementazione tramite vettori circolari utilizza l’operazione **modulo** per rappresentare la circolarità (bisogna fare però attenzione all’overflow).



Analisi ammortizzata

L’**analisi ammortizzata** è una tecnica di analisi della complessità che valuta il tempo richiesto per eseguire, **nel caso pessimo**, una sequenza di operazioni su una **struttura dati**.

In una struttura dati esistono operazioni più o meno costose, se le operazioni più costose sono poco frequenti, allora il loro costo può essere **ammortizzato** dalle operazioni meno costose.

Bisogna capire subito un’importante differenza:

- L’**analisi del caso medio** è probabilistica e si basa su una **singola operazione**
- L’**analisi ammortizzata** è deterministica, si basa su **operazioni multiple** e considera il caso pessimo

Ci sono tre metodi per effettuare l’analisi ammortizzata:

- **Metodo dell’aggregazione**
  Si calcola la complessità $T(n)$ per eseguire n operazioni in sequenza nel **caso pessimo** (tecnica derivata dalla matematica)
- **Metodo degli accantonamenti**
  Alle operazioni vengono assegnati dei **costi ammortizzati** che possono essere maggiori o minori rispetto al loro costo effettivo (tecnica derivata dall’economia)
- **Metodo del potenziale**
  Lo stato del sistema viene descritto con una **funzione di potenziale** (tecnica derivata dalla fisica)

Esempio: Contatore Binario

Prendiamo un contatore binario di k bit, rappresentato da un vettore A di booleani $0/1$. Il bit meno significativo è in $A[0]$ e il bit più significativo è in $A[k-1]$.

Di conseguenza il valore del contatore è $x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$.

Consideriamo l'operazione *increment*$()$, che incrementa il contatore di 1.

```
increment(int[] A, int k)
int i = 0
while i < k and A[i] == 1 do
    A[i] = 0
    i = i + 1
if i < k then
    A[i] = 1
```

Vediamo un esempio che parte da $x = 0$ ed esegue 16 volte *increment*$()$:

x	$A[7]$	$A[6]$	$A[5]$	$A[4]$	$A[3]$	$A[2]$	$A[1]$	$A[0]$
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1
2	0	0	0	0	0	0	1	0
3	0	0	0	0	0	0	1	1
4	0	0	0	0	0	1	0	0
5	0	0	0	0	0	1	0	1
6	0	0	0	0	0	1	1	0
7	0	0	0	0	0	1	1	1
8	0	0	0	0	1	0	0	0
9	0	0	0	0	1	0	0	1
10	0	0	0	0	1	0	1	0
11	0	0	0	0	1	0	1	1
12	0	0	0	0	1	1	0	0
13	0	0	0	0	1	1	0	1
14	0	0	0	0	1	1	1	0
15	0	0	0	0	1	1	1	1
16	0	0	0	1	0	0	0	0

Metodo dell'aggregazione

Si calcola la complessità $T(n)$ per eseguire n **operazioni in sequenza nel caso pessimo**

Si calcola poi il costo ammortizzato $\frac{T(n)}{n}$ come **media** su n operazioni.

I concetti base sono:

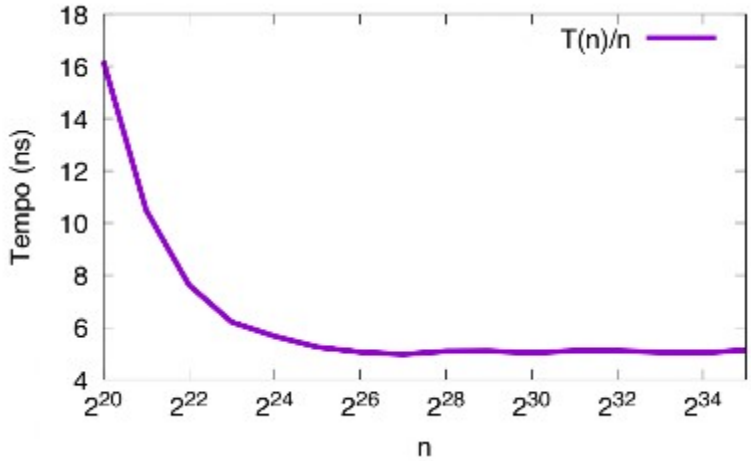
- Una **sequenza** di operazioni
  Si considera l'evoluzione della struttura dati data una sequenza di operazioni
- Nel **caso pessimo**
  Si considera la peggior sequenza di operazioni
- L'**aggregazione**
  Sommatoria delle complessità individuali

Vediamo una prima analisi "grossolana" (ovvero troviamo un limite superiore "largo", nel senso che possiamo essere più precisi)
Notiamo che per rappresentare n (dopo n operazioni avremo $x=n$) ci servono

$$k = \lceil \log(n+1) \rceil$$

bit.
Di conseguenza una chiamata $increment()$, nel caso pessimo, richiede tempo $O(k)$ (ovvero va a toccare tutti i bit).
Di conseguenza il costo di n operazioni sarà $T(n)=O(nk)$ e quindi il costo di un'operazione è $\frac{T(n)}{n}=O(k)=O(\log\ n)$. Questa stima è corretta (la complessità non sarà mai superiore a $O(k)$) però **non è precisa**.

Guardando però il grafico osserviamo che l'andamento non è quello che ci aspettiamo con il costo che abbiamo appena calcolato:



- $n = 2^k$ operazioni
- $k \in [20, 35]$
- Eseguito in Java

Osserviamo che per un valore crescente dell'input, l'algoritmo diventa **in media** più efficiente, per poi ad un certo punto diventare costante.
La parte decrescente non è algoritmica (è grazie alle ottimizzazioni di Java). La parte "rilevante" è la parte **costante**, visto che prima avevamo detto che l'algoritmo era $O(k)$.
Questo succede perchè non tutte le n operazioni generano il riporto fino in fondo.
Possiamo osservare che il tempo necessario ad eseguire l'intera sequenza è proporzionale al numero di bit che vengono modificati, quindi dobbiamo capire qual è il numero di bit che vengono modificati

x	$A[7]$	$A[6]$	$A[5]$	$A[4]$	$A[3]$	$A[2]$	$A[1]$	$A[0]$	#bit
0	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	2
3	0	0	0	0	0	0	1	1	1
4	0	0	0	0	0	1	0	0	3
5	0	0	0	0	0	1	0	1	1
6	0	0	0	0	0	1	1	0	2
7	0	0	0	0	0	1	1	1	1
8	0	0	0	0	1	0	0	0	4
9	0	0	0	0	1	0	0	1	1
10	0	0	0	0	1	0	1	0	2
11	0	0	0	0	1	0	1	1	1
12	0	0	0	0	1	1	0	0	3
13	0	0	0	0	1	1	0	1	1
14	0	0	0	0	1	1	1	0	2
15	0	0	0	0	1	1	1	1	1
16	0	0	0	1	0	0	0	0	5
#bit	0	0	0	1	2	4	8	16	

Nella tabella viene contato il numero di bit cambiati per ogni riga, ma proviamo a guardare le colonne.
Da questa simulazione si vede che:

- $A[0]$ viene modificato ogni incremento
- $A[1]$ viene modificato ogni 2 incrementi
- $A[2]$ viene modificato ogni 4 incrementi
- …
- $A[i]$ viene modificato ogni 2^i incrementi

Possiamo quindi scrivere la seguente equazione per la complessità:

Analisi ammortizzata

- Costo totale: $T(n) = \sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor \leq n \sum_{i=0}^{k-1} \frac{1}{2^i} \leq n \sum_{i=0}^{+\infty} \left(\frac{1}{2}\right)^i = 2n$
- Costo ammortizzato: $T(n)/n \leq 2n/n = 2 = O(1)$

E quindi otteniamo un costo costante, che è coerente con quello che abbiamo ottenuto nella pratica.

Metodo degli accantonamenti

Nel metodo dell'aggregazione abbiamo dovuto avere l'intuizione di guardare le colonne invece delle righe, col metodo degli accantonamenti invece esiste un procedimento standard.
Si assegna un costo ammortizzato **potenzialmente** distinto ad ognuna delle operazioni possibili.
Il costo ammortizzato può essere diverso dal costo effettivo:

- Le operazioni meno costose vengono caricate di un costo aggiuntivo detto **credito**
  $costo\ \ ammortizzato = costo\ \ effettivo + credito\ \ prodotto$
- I crediti accumulati sono usati per pagare le operazioni più costose
  $costo\ \ ammortizzato = costo\ \ effettivo - credito\ \ consumato$

L'obiettivo è dimostrare che la somma dei costi ammortizzati a_i è un limite superiore ai costi effettivi c_i, ovvero

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} a_i$$

Così da poter dire che la funzione dei costi ammortizzati è un O grande della funzione dei costi effettivi.
Bisogna ricordare alcune cose importanti:

- La dimostrazione deve valere per tutte le sequenze (quindi caso pessimo, ma qui abbiamo una sequenza sola)
- Il credito dopo l'operazione $t\text{-}esima$ è espresso dalla seguente formula ed è sempre non negativo $\sum_{i=1}^{t} a_i - \sum_{i=1}^{t} c_i$

Quindi, il costo effettivo dell'operazione $increment()$ è d (dove d è il numero di bit che cambiano valore).
Il costo ammortizzato dell'operazione di $increment()$ è 2:

- 1 per il cambio di bit da 0 a 1 (costo effettivo)
- 1 per il futuro cambio dello stesso bit da 1 a 0

Di conseguenza, dopo alcune osservazioni, otteniamo:

- In ogni istante il credito è pari al numero di bit a 1 presenti (visto che aumento il credito di 1 per ogni cambio di bit da 0 a 1)
- Il costo totale è $O(n)$
- Il costo ammortizzato è $O(1)$

Questo perchè possiamo dire che $\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} a_i$ in quanto nel costo ammortizzato ho già pagato per cose che non sono ancora state fatte, e quindi sarà sempre maggiore. E quindi vale $\sum_{i=1}^{n} c_i \leq 2n$ che ci dice che il costo totale è $O(n)$ e di conseguenza il costo ammortizzato è $O(1)$

Metodo del potenziale

Una **funzione di potenziale** Φ associa ad uno stato S della struttura dati la "**quantità di lavoro**" $\Phi(S)$ che è stato contabilizzato nell'analisi ammortizzata ma non ancora eseguito.
In altre parole, $\Phi(S)$ rappresenta la quantità di energia potenziale "immagazzinata" in quello stato

Il costo ammortizzato è:

$$costo\ \ ammortizzato = costo\ \ effettivo + variazione\ \ di\ \ potenziale$$

Ovvero:

$$a_i = c_i + \Phi(S_i) - \Phi(S_{i-1})$$

dove S_i è lo stato associato alla $i-esima$ operazione.

Per una sequenza di n operazioni abbiamo:

$$\begin{aligned} A &= \sum_{i=1}^{n} a_i \\ &= \sum_{i=1}^{n} (c_i + \Phi(S_i) - \Phi(S_{i-1})) \\ &= \sum_{i=1}^{n} c_i + \sum_{i=1}^{n} (\Phi(S_i) - \Phi(S_{i-1})) \\ &= C + \Phi(S_1) - \Phi(S_0) + \Phi(S_2) - \Phi(S_1) + \ldots + \Phi(S_n) - \Phi(S_{n-1}) \\ &= C + \Phi(S_n) - \Phi(S_0) \end{aligned}$$

Se la variazione di potenziale $\Phi(S_n) - \Phi(S_0)$ è non negativa, allora il costo ammortizzato A è un limite superiore al costo reale.

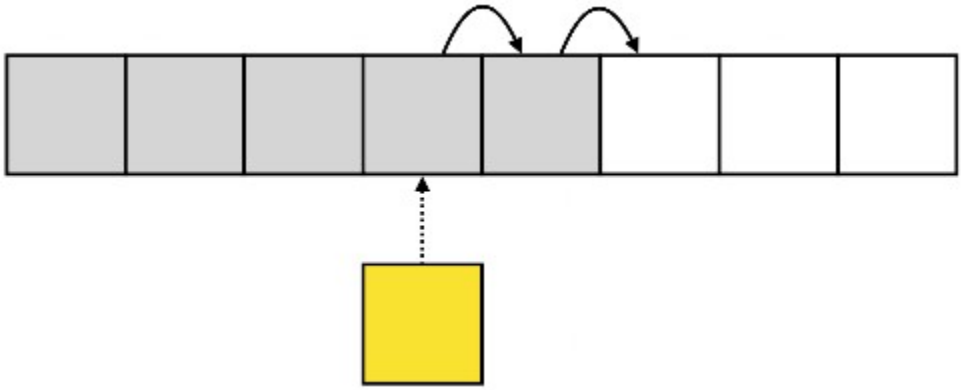
Nel nostro esempio possiamo scegliere come funzione potenziale Φ il numero di bit 1 presenti nel contatore.
Per l'operazione di $increment()$ prendiamo in considerazione t, ovvero il numero di bit 1 incontrati a partire dal meno significativo, prima di incontrare uno 0.

- Costo effettivo: $1+t$
- Variazione di potenziale: $1-t$ ($-t$ perchè devo sottrarre tutti i bit 1 che trasformo in 0, e dopo $+1$ che è il bit 0 che trasformo in 1)
- Costo ammortizzato: $1+t+1-t=2$

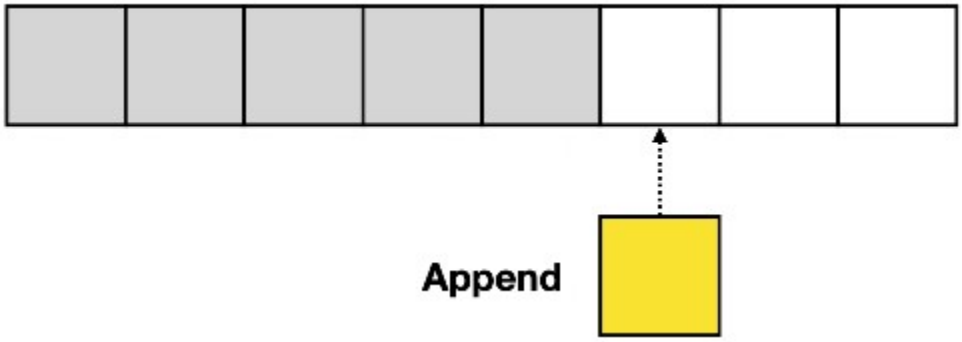
All'inizio ci sono 0 bit accesi, quindi $\Phi(S_0)=0$, alla fine $\Phi(S_n)\geq 0$, quindi la differenza di potenziale è non negativa.

Esempio: Vettore dinamico

Vediamo ora un esempio più “reale”.
Prendiamo come esempio le sequenze implementate tramite **vettori dinamici**, che vengono allocati di una certa dimensione detta **capacità**.
Notiamo che l’inserimento “in mezzo” non è efficiente in questa struttura dati, in quanto dobbiamo spostare gli elementi a destra di una posizione, con costo $O(n)$



L’inserimento in fondo invece è efficiente, perchè basta inserirlo, quindi ha costo $O(1)$



Il problema è che non è noto a priori quanti elementi entreranno nella sequenza. Quindi **la capacità selezionata può rivelarsi insufficiente**.
La soluzione è allocare un nuovo vettore di capacità maggiore, ricopiando il contenuto del vecchio vettore nel nuovo vettore e deallocando quello vecchio.

Esistono due tecniche per “decidere” la nuova capacità del vettore:

```
private Object[] buffer = new Object[INITSIZE];

// Raddoppiamento - Utilizzato in ArrayList (1.2)
private void doubleStorage() {
  Object[] newb = new Object[2*buffer.length];
  System.arraycopy(buffer,0, newb,0, buffer.length);
  buffer = newb;
}

// Incremento fisso - Utilizzato in Vector (1.0)
private void incrementStorage() {
  Object[] newb = new Object[buffer.length+INCREMENT];
  System.arraycopy(buffer,0, newb,0, buffer.length);
  buffer = newb;
}
```

La domanda è: qual'è il migliore dei due? Il migliore è il **raddoppiamento** (o comunque usando un fattore moltiplicativo, andrebbe bene anche 1.5), e ora vediamo il perché usando il **metodo dell'aggregazione**.

Somma: raddoppiamento

Analisi ammortizzata, raddoppiamento del vettore

Costo effettivo di un'operazione add():

$$c_i = \begin{cases} i & \exists k \in \mathbb{Z}_0^+ : i = 2^k + 1 \\ 1 & \text{altrimenti} \end{cases}$$

Assunzioni:

- Dimensione iniziale: 1
- Costo di scrittura di un elemento: 1

n	costo
1	1
2	$1 + 2^0 = 2$
3	$1 + 2^1 = 3$
4	1
5	$1 + 2^2 = 5$
6	1
7	1
8	1
9	$1 + 2^3 = 9$
10	1
11	1
12	1
13	1
14	1
15	1
16	1
17	$1 + 2^4 = 17$

Notiamo che ogni volta che il vettore è pieno (e succede quando inseriamo un elemento e ce ne sono già 2^k presenti) dobbiamo creare un nuovo vettore, copiando i 2^k elementi che erano già presenti e aggiungendo quello nuovo.
I momenti in cui raddoppio diventano più rari, perché raddoppio appunto.

Analisi ammortizzata, raddoppiamento del vettore

Costo effettivo di n operazioni add():

$$\begin{aligned} T(n) &= \sum_{i=1}^{n} c_i \\ &= n + \sum_{j=0}^{\lfloor \log n \rfloor} 2^j \\ &= n + 2^{\lfloor \log n \rfloor + 1} - 1 \\ &\leq n + 2^{\log n + 1} - 1 \\ &= n + 2n - 1 = O(n) \end{aligned}$$

Costo ammortizzato di un'operazione add():

$$T(n)/n = \frac{O(n)}{n} = O(1)$$

Il costo c_i è dato da 1 (che è l'inserimento del nuovo elemento e viene fatto sempre come vediamo dalla colonna dei costi) più l'eventuale ricopiamento del vecchio vettore (che viene fatto al massimo $\log\ n$ volte)

Somma: incremento

Analisi ammortizzata, incremento del vettore

Costo effettivo di un'operazione add():

$$c_i = \begin{cases} i & (i \bmod d) = 1 \\ 1 & \text{altrimenti} \end{cases}$$

Assunzioni:

- Incremento: d
- Dimensione iniziale: d
- Costo di scrittura di un elemento: 1

Nell'esempio:

- $d = 4$

n	costo
1	1
2	1
3	1
4	1
5	$1 + d = 5$
6	1
7	1
8	1
9	$1 + 2d = 9$
10	1
11	1
12	1
13	$1 + 3d = 13$
14	1
15	1
16	1
17	$1 + 4d = 17$

Le osservazioni sono le stesse di prima, solo che ora pago l'incremento a $d+1$ poi a $2d+1$ poi a $3d+1$ etc.

Analisi ammortizzata, incremento del vettore

Costo effettivo di n operazioni add():

$$
\begin{aligned}
T(n) &= \sum_{i=1}^{n} c_i \\
&= n + \sum_{j=1}^{\lfloor n/d \rfloor} d \cdot j \\
&= n + d \sum_{j=1}^{\lfloor n/d \rfloor} j \\
&= n + d\frac{(\lfloor n/d \rfloor + 1)\lfloor n/d \rfloor}{2} \\
&\leq n + \frac{(n/d+1)n}{2} = O(n^2)
\end{aligned}
$$

Costo ammortizzato di un'operazione add():

$$T(n)/n = \frac{O(n^2)}{n} = O(n)$$

Il costo c_i è dato da 1 (che è l'inserimento del nuovo elemento e viene fatto sempre come vediamo dalla colonna dei costi) più l'eventuale ricopiamento del vecchio vettore (che viene fatto al massimo $\frac{n}{d}$ volte)

Quindi il raddoppiamento ha costo costante invece l'incremento ha costo lineare. Infatti le implementazioni reali utilizzano il raddoppiamento, ecco alcuni esempi col proprio fattore di espansione:

Linguaggio	Struttura dati	Fattore espansione
GNU C++	`std::vector`	2.0
Microsoft VC++ 2003	`vector`	1.5
C#	`List`	2.0
Python	`list`	1.125
Oracle Java	`ArrayList`	2.0
OpenSDK Java	`ArrayList`	1.5

Abbassando il fattore di espansione aumenta la ricorrenza dell'incremento, ma diminuisce lo "spreco" di memoria.

Ora confrontiamo il vettore dinamico anche con le liste (parlando di memoria) per convincerci ancora di più che è la tecnica migliore.

Domande

- Quanto memoria occupa un intero 32 bit se memorizzato in un vettore dinamico, nel caso pessimo/ottimo?
- Quanto memoria occupa un intero 32 bit se memorizzato in un elemento di una lista, nel caso pessimo/ottimo?

Vettori

- Nel caso ottimo (vettore pieno): $4n$ byte
- Nel caso pessimo (vettore appena raddoppiato): $2 \cdot 4n$ byte

I vettori dinamici abbiamo visto occupano n nel caso ottimo e $2n$ nel caso pessimo.
Invece le liste occupano sempre $2n$ (32 *bit* per l'elemento e 32 *bit* per il puntatore)

Liste

Alcuni dati interessanti (e inquietanti) relativi a Java:

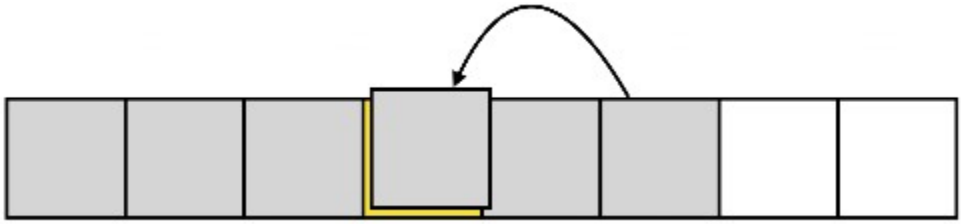
- un `Object` vuoto richiede 16 byte in un'architettura a 64 bit
- l'allocazione di memoria avviene per multipli di 8 byte
- un oggetto contenente un intero e due reference (lista bidirezionale) richiede 32-40 byte (Java compressed references)

In Java scopriamo che un object occupa molto di più (si arriva a $8/10n$)

Questo discorso si riscontra anche nelle *String* in java, le quali sono immutabili e ogni volta che la "modifichiamo" dobbiamo ricopiarla. Questo si risolve usando *StringBuffer* che utilizza un vettore dinamico.

Cancellazione

Ritiriamo fuori il metodo del potenziale.
Vediamo ora la cancellazione.
Se cancelliamo da un vettore ordinato, dobbiamo spostare tutti gli elementi a destra di una posizione a sinistra, con costo $O(n)$.
Invece se cancelliamo da un vettore non ordinato, basta spostare l'ultimo elemento nella posizione cancellata, con costo $O(1)$.



Il problema è che continuando a cancellare, potrei avere un vettore relativamente grande con dentro pochi elementi, quindi sto sprecando memoria.
Per ridurre lo spreco di memoria è opportuno **contrarre** il vettore quando il **fattore di carico** $\alpha = \frac{dimensione}{capacità}$ è troppo piccolo (la dimensione è il numero di elementi attualmente presenti).
La contrazione consiste (come per l'espansione) in una allocazione, una copia e una deallocazione.
Scegliamo di dimezzare quando facciamo una contrazione.
Dobbiamo scegliere a quale valore del fattore di carico eseguire la contrazione.
Se scegliamo $\alpha = \frac{1}{2}$ (che sembra il più ovvio) rischiamo di sprecare tanto costo in raddoppiamenti e contrazioni effettuati uno dopo l'altro come in questo esempio:

Considerate la seguente sequenza di Inserimenti / Rimozioni in un vettore di capacità 8:

```
Ops: I I I I I R I R I R I R I
Dim: 1 2 3 4 5 4 5 4 5 4 5 4 5
Cap: 8 8 8 8 8 4 8 4 8 4 8 4 8
```

Questo perché dimezzando quando $\alpha = \frac{1}{2}$ mi fa diventare il nuovo vettore pieno.

Di conseguenza si sceglie di utilizzare $\alpha = \frac{1}{4}$ e dopo una contrazione il fattore di carico diventa $\alpha = \frac{1}{2}$ (esattamente come dopo un'espansione)

Per eseguire l'analisi ammortizzata usiamo una funzione di potenziale particolare che:

- Vale 0 all'inizio e subito dopo una contrazione/espansione
- Cresce fino a raggiungere il numero di elementi presenti nella tavola quando α aumenta fino ad 1 o diminuisce fino ad $\frac{1}{4}$

Quindi devo caricare il mio potenziale del costo necessario per raddoppiare o dimezzare.
Così quando ci avviciniamo ad $\alpha = 1$ la mia funzione di potenziale varrà n e così avremo abbastanza costo per pagare il raddoppiamento (e lo stesso vale per $\frac{1}{4}$)

Funzione di potenziale

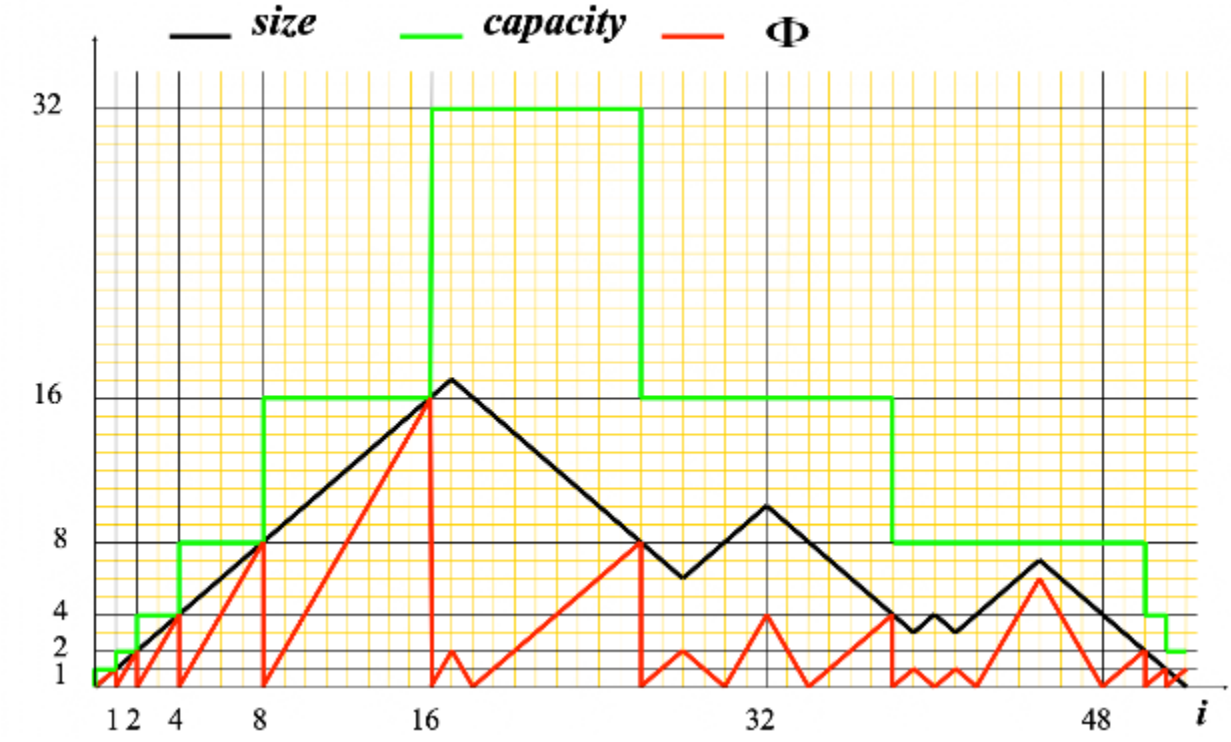
$$\Phi = \begin{cases} 2 \cdot dim - capacità & \alpha \geq \frac{1}{2} \\ capacità/2 - dim & \alpha \leq \frac{1}{2} \end{cases}$$

Alcuni casi esplicativi:

- $\alpha = \frac{1}{2}$ (dopo espansione/contrazione) $\Rightarrow \Phi = 0$
- $\alpha = 1$ (prima di espansione) $\Rightarrow dim = capacità \Rightarrow \Phi = dim$
- $\alpha = \frac{1}{4}$ (prima di contrazione) $\Rightarrow capacità = 4 \cdot dim \Rightarrow \Phi = dim$

In altre parole: subito prima di espansioni e contrazioni il potenziale è sufficiente per "pagare" il costo della copia dei *dim* valori

Quindi appena dopo un'espansione/contrazione io ho pagato tutto quello che dovevo pagare per fare quella operazione (infatti $\Phi = 0$)



Vediamo il costo ammortizzato nel caso in cui $\alpha \geq \frac{1}{2}$

Se $\alpha \geq \frac{1}{2}$ il costo ammortizzato di un inserimento senza espansione è:

$$\begin{aligned} a_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (2 \cdot dim_i - capacità_i) - (2 \cdot dim_{i-1} - capacità_{i-1}) \\ &= 1 + 2 \cdot (dim_{i-1} + 1) - capacità_{i-1} - 2 \cdot dim_{i-1} + capacità_{i-1} \\ &= 3 \end{aligned}$$

Notiamo che $capacità_i$ è uguale a $capacità_{i-1}$ (visto che non ho fatto l'espansione) e che dim_i è uguale a $dim_{i-1}+1$ visto che ho aggiunto un elemento.

Se $\alpha = 1$ il costo ammortizzato di un inserimento con espansione è:

$$\begin{aligned} a_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + dim_{i-1} + (2 \cdot dim_i - capacità_i) - (2 \cdot dim_{i-1} - capacità_{i-1}) \\ &= 1 + dim_{i-1} + 2 \cdot (dim_{i-1} + 1) - 2 \cdot dim_{i-1} - 2 \cdot dim_{i-1} + dim_{i-1} \\ &= 3 \end{aligned}$$

Bisognerebbe fare anche gli altri 6 casi, ma il costo viene sempre 3.
Di conseguenza la complessità delle operazioni su un array dinamico è costante.

Ristrutturazione in tabelle Hash

- Non è conveniente che α cresca troppo
  - In particolare con la scansione interna
  - Ma vale anche per le liste di trabocco
- Sopra una soglia t_α prefissata (tipicamente 0.5-0.75)
  - Si alloca una nuova tabella di dimensione $2m$
  - Si reinseriscono tutte le chiavi presenti nella nuova tabella
- Risultato
  - Fattore di carico dimezzato (tipicamente 0.25)
  - Nessun elemento **deleted**
- Costi
  - Costo $O(m)$ per la ristrutturazione nel caso pessimo
  - Costo ammortizzato costante (vedi dimostrazione per vettori)

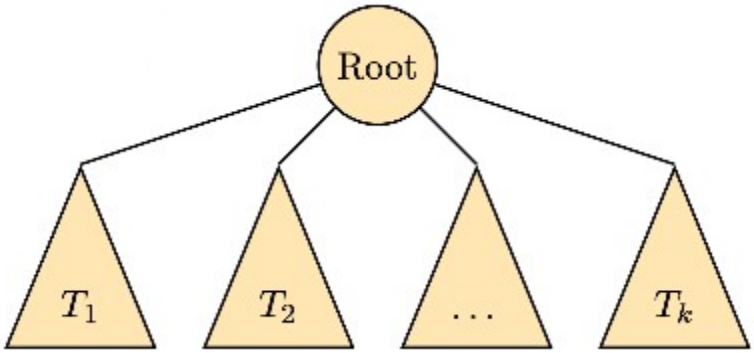
Alberi

Un **albero radicato** (rooted tree) consiste in un insieme di **nodi** e un insieme di **archi** orientati che connettono coppie di nodi, con le seguenti proprietà:

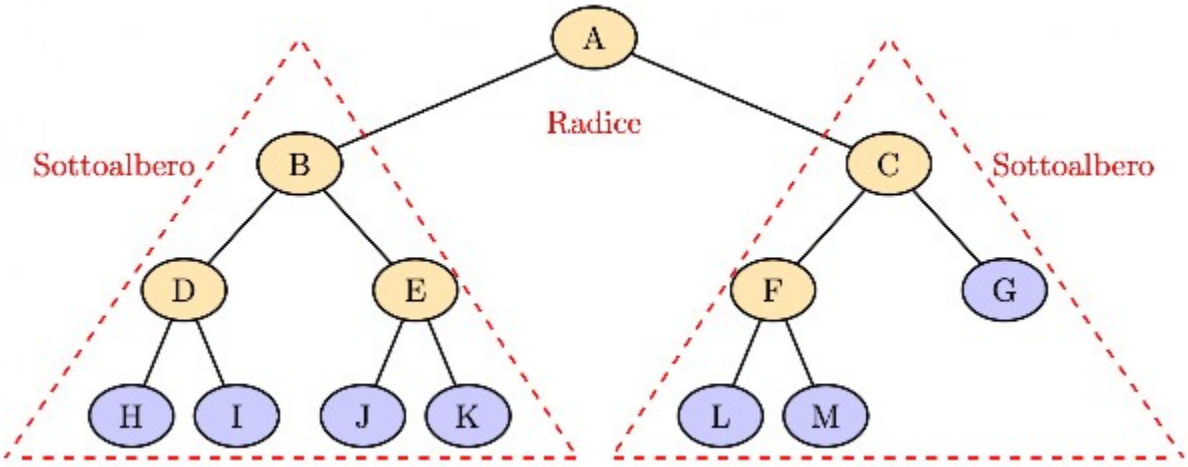
- Un nodo dell'albero è designato come **radice** (per questo si chiama albero radicato, ha un nodo "in cima")
- Ogni nodo n ha esattamente un arco entrante (ad esclusione della radice che ne ha zero)
- Esiste un **cammino unico** dalla radice ad ogni nodo
- L'albero è connesso

Un albero si può definire anche in maniera ricorsiva:
Un albero è dato da:

- Un insieme vuoto, oppure
- un nodo **radice** e zero o più **sottoalberi**, ognuno dei quali è un albero; la radice è connessa alla radice di ogni sottoalbero con un arco orientato



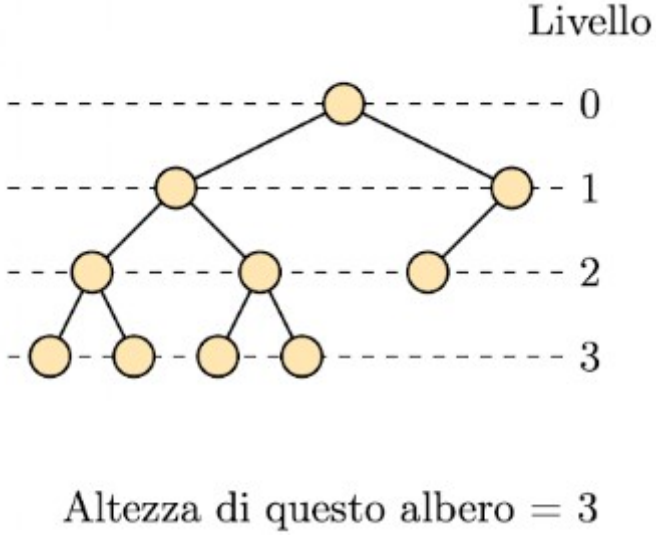
Vediamo ora un esempio per descrivere la **terminologia** degli alberi:



- A è la **radice**
- B e C sono **radici** dei due **sottoalberi** di A
- A è detto **padre** di B e C
- B e C sono detti **figli** di A
- Le coppie B, C e D, E etc, sono dette **fratelli** (hanno lo stesso padre)

- I nodi viola sono le **foglie**
- Gli altri nodi sono detti **nodi interni**

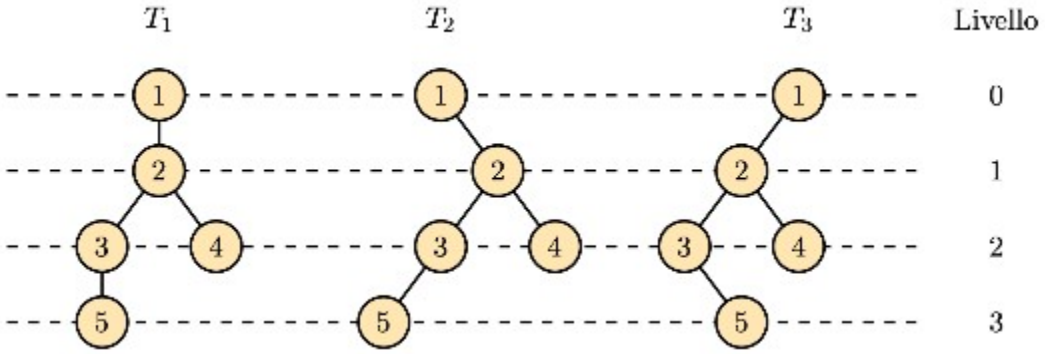
La **profondità** (depth) di un nodo è la lunghezza del cammino che porta dalla radice al nodo (misurata in numero di archi).
Un **livello** di un albero è l'insieme di nodi presenti alla stessa profondità.
L'**altezza di un albero** equivale alla profondità massime delle sue foglie.



Altezza di questo albero = 3

Albero binario

Un **albero binario** è un albero radicato in cui ogni nodo ha al massimo **due figli**, identificati come figlio **sinistro** e figlio **destro**. (Due alberi sono diversi anche se hanno gli stessi nodi e le stesse parentele nel caso in cui un figlio destro e un figlio sinistro siano scambiati)



T1 non lo consideriamo binario in quanto non si riesce a capire se 2 sia figlio sinistro o destro.
Gli altri due sono binari ma diversi perché la posizione dei figli conta.

TREE

% Costruisce un nuovo nodo, contenente v, senza figli o genitori
Tree(ITEM v)

% Legge il valore memorizzato nel nodo
ITEM read()

% Modifica il valore memorizzato nel nodo
write(ITEM v)

% Restituisce il padre, oppure **nil** se questo nodo è radice
TREE parent()

TREE

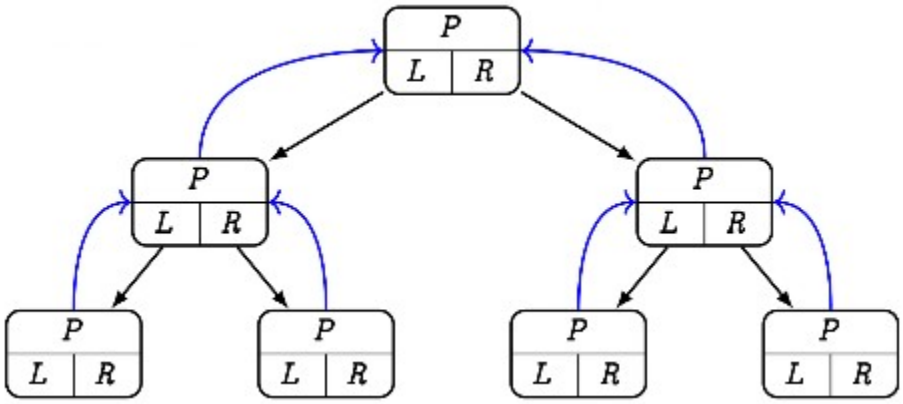
% Restituisce il figlio sinistro (destro) di questo nodo; restituisce **nil** se assente
TREE left()
TREE right()

% Inserisce il sottoalbero radicato in t come figlio sinistro (destro) di questo nodo
insertLeft(TREE t)
insertRight(TREE t)

% Distrugge (ricorsivamente) il figlio sinistro (destro) di questo nodo
deleteLeft()
deleteRight()

Per memorizzare un albero binario dobbiamo per ogni nodo sapere:

- Il valore
- Il padre (una reference/puntatore)
- Il figlio sinistro (una reference/puntatore)
- Il figlio destro (una reference/puntatore)



Ecco l'implementazione di alcune funzioni viste prima:

```
TREE
Tree(ITEM v)
    TREE t = new TREE
    t.parent = nil
    t.left = t.right = nil
    t.value = v
    return t

insertLeft(TREE T)
    if left == nil then
        T.parent = this
        left = T

insertRight(TREE T)
    if right == nil then
        T.parent = this
        right = T
```

```
TREE
deleteLeft()
    if left ≠ nil then
        left.deleteLeft()
        left.deleteRight()
        delete left
        left = nil

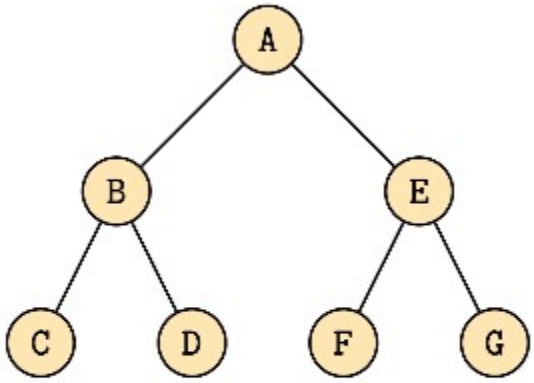
deleteRight()
    if right ≠ nil then
        right.deleteLeft()
        right.deleteRight()
        delete right
        right = nil
```

Per visitare un albero (ovvero analizzare tutti i nodi di un albero) binario esistono 4 metodi (tutti di costo $\Theta(n)$ in quanto ogni nodo viene visitato esattamente una volta):

- Visita in **profondità** (**DFS** Depth-First Search)
  Per visitare un albero si visitano ricorsivamente tutti i suoi sottoalberi (richiede uno stack)
  - **Pre** order
  - **In** order
  - **Post** order
- Visita in **ampiezza** (**BFS** Breadth-First Search)
  Si visita l'albero **livello** per livello partendo dalla radice (richiede una queue)

Depth-First Search

```
dfs(TREE t)
if t ≠ nil then
    % pre-order visit of t
    print t
    dfs(t.left())
    % in-order visit of t
    print t
    dfs(t.right())
    % post-order visit of t
    print t
```



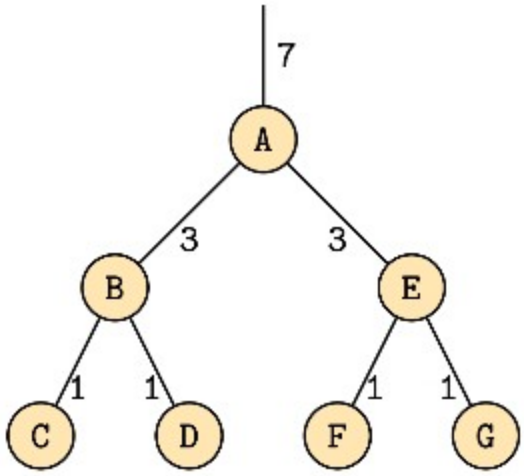
In questo esempio abbiamo i seguenti ordini di visita:

- Pre order: A B C D E F G
- In order: C B D A F E G
- Post order: C D B F G E A

I vari metodi della visita in profondità possono essere utili in diversi contesti, ad esempio la post order può servire per contare il numero dei nodi, la in order serve per visualizzare un'espressione etc.

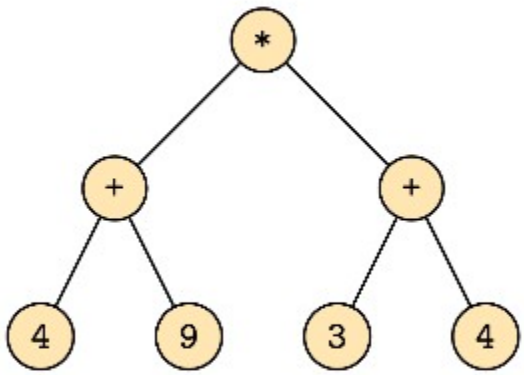
Contare nodi – Post-visita

```
int count(TREE T)
if T == nil then
    return 0
else
    C_ℓ = count(T.left())
    C_r = count(T.right())
    return C_ℓ + C_r + 1
```



Stampare espressioni – In-visita

```
int printExp(TREE T)
if T.left() == nil and
 T.right == nil then
    print T.read()
else
    print "("
    printExp(T.left())
    print T.read()
    printExp(T.right())
    print ")"
```



((4+9) * (3+4))

Albero generico

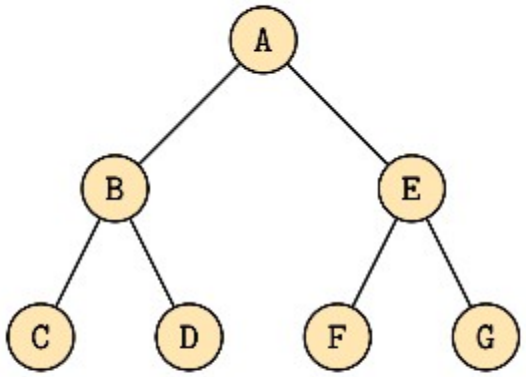
```
TREE
% Costruisce un nuovo nodo, contenente v, senza figli o genitori
Tree(ITEM v)
% Legge il valore memorizzato nel nodo
ITEM read()
% Modifica il valore memorizzato nel nodo
write(ITEM v)
% Restituisce il padre, oppure nil se questo nodo è radice
TREE parent()
```

```
TREE
% Restituisce il primo figlio, oppure nil se questo nodo è una foglia
TREE leftmostChild()
% Restituisce il prossimo fratello, oppure nil se assente
TREE rightSibling()
% Inserisce il sottoalbero t come primo nodo di questo nodo
insertChild(TREE t)
% Inserisce il sottoalbero t come prossimo fratello di questo nodo
insertSibling(TREE t)
% Distruggi l'albero radicato identificato dal primo figlio
deleteChild()
% Distruggi l'albero radicato identificato dal prossimo fratello
deleteSibling()
```

Vediamo come vengono implementate DFS e BFS in un albero generico:

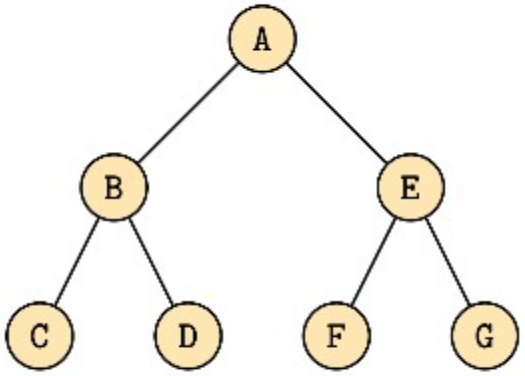
Depth-First Search

```
dfs(TREE t)
if t ≠ nil then
    % pre-order visit of node t
    print t
    TREE u = t.leftmostChild()
    while u ≠ nil do
        dfs(u)
        u = u.rightSibling()
    % post-order visit of node t
    print t
```



Breadth-First Search

```
bfs(TREE t)
QUEUE Q = Queue()
Q.enqueue(t)
while not Q.isEmpty() do
    TREE u = Q.dequeue()
    % visita per livelli nodo u
    print u
    u = u.leftmostChild()
    while u ≠ nil do
        Q.enqueue(u)
        u = u.rightSibling()
```

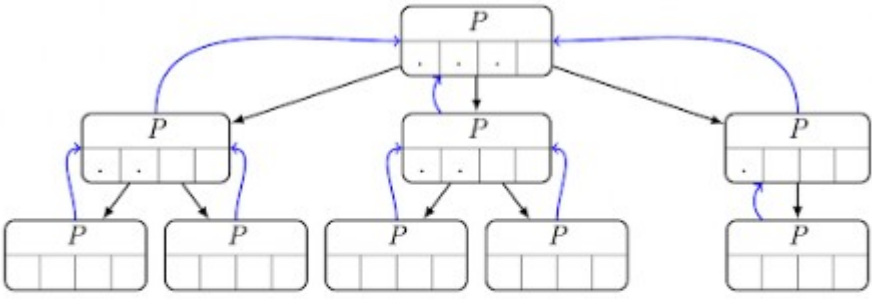


Sequence: A B E C D F G
Queue:

Memorizzazione

Esistono diversi metodi per memorizzare un albero, alcuni metodi sono più o meno indicati a seconda del numero massimo e medio dei figli presenti.

- Realizzazione con vettore dei figli

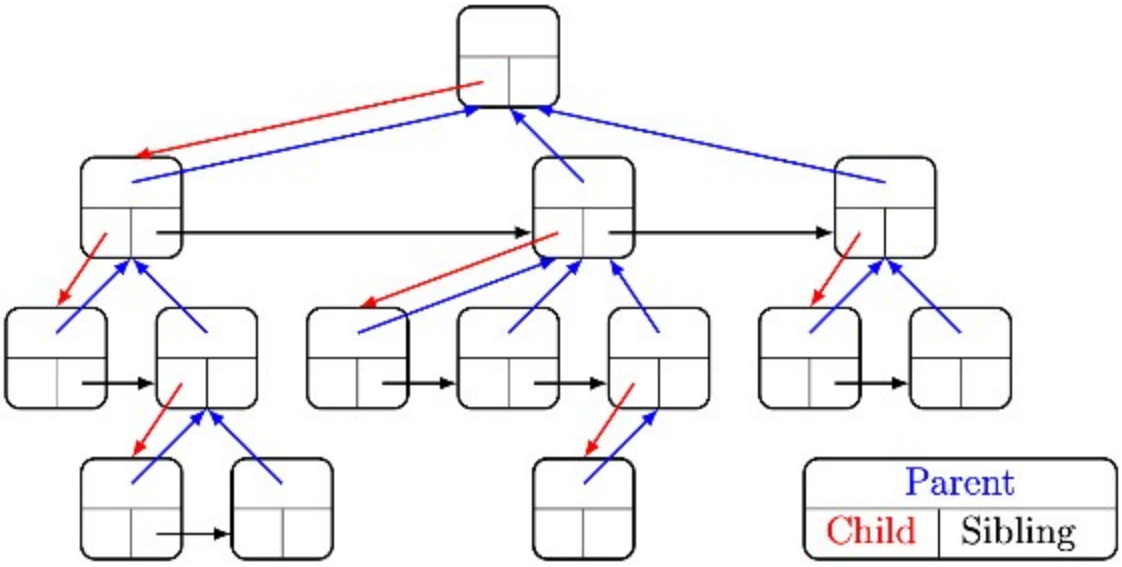


Campi memorizzati nei nodi

- *parent*: reference al nodo padre
- Vettore dei figli: a seconda del numero di figli, può comportare una discreta quantità di spazio sprecato

- Realizzazione primo figlio, prossimo fratello

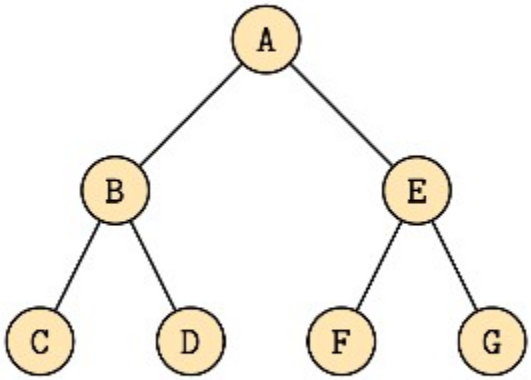
Implementato come una lista di fratelli



- Realizzazione con vettore dei padri

L'albero è rappresentato da un vettore i cui elementi contengono il valore associato al nodo e l'indice della posizione del padre nel vettore.

1	A	0
2	B	1
3	E	1
4	C	2
5	D	2
6	F	3
7	G	3



Alberi binari di ricerca

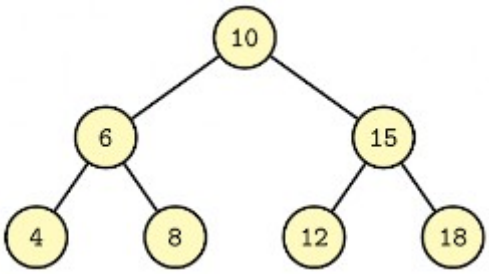
Lo scopo è quello di portare **l'idea di ricerca** (ad esempio di un dizionario) all'interno degli alberi binari.
Quindi ogni **associazione chiave-valore** viene memorizzata in un albero binario, ovvero ogni nodo u dell'albero conterrà la coppia $(u.key,\ u.value)$ (ovviamente le **chiavi** devono appartenere ad un **insieme totalmente ordinato**).

Nodo albero

TREE
TREE *parent*
TREE *left*
TREE *right*
ITEM *key*
ITEM *value*

Essendo l'obiettivo la ricerca, dobbiamo dare delle proprietà a questo albero per facilitarla:

- Le chiavi contenute nei nodi del **sottoalbero sinistro** di u sono **minori** di $u.key$
- Le chiavi contenute nei nodi del **sottoalbero destro** di u sono **maggiori** di $u.key$



Grazie a queste proprietà abbiamo la possibilità di creare un **algoritmo di ricerca dicotomica** (ovvero tagliando a metà il problema ad ogni passo).

Getters

- ITEM key()
- ITEM value()
- TREE left()
- TREE right()
- TREE parent()

Dizionario

- ITEM lookup(ITEM k)
- insert(ITEM k, ITEM v)
- remove(ITEM k)

Ordinamento

- TREE successorNode(TREE t)
- TREE predecessorNode(TREE t)
- TREE min()
- TREE max()

Alberi binari di ricerca – Funzioni interne

- TREE lookupNode(TREE T, ITEM k)
- TREE insertNode(TREE T, ITEM k, ITEM v)
- TREE removeNode(TREE T, ITEM k)

DICTIONARY

TREE *tree*

Dictionary()
  tree = **nil**

Ricerca

Abbiamo la funzione *Tree lookupNode(Tree T, Item k)* che ci restituisce il nodo dell'albero T che contiene la chiave k se presente, altrimenti *nil*.
Quindi per implementare *Item lookup(Item k)* basta usare *Tree lookupNode(Tree T, Item k)* per trovare il nodo e poi ritornare il valore di quel nodo:

ITEM lookup(ITEM k)

```
TREE t = lookupNode(tree, k)
if t ≠ nil then
    return t.value()
else
    return nil
```

Vediamo ora l'implementazione di *Tree lookupNode(Tree T, Item k)*:

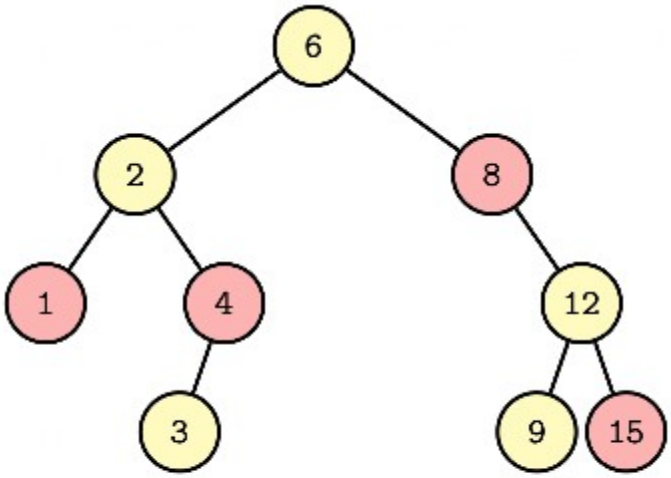
Iterativa

```
TREE lookupNode(TREE T, ITEM k)
TREE u = T
while u ≠ nil and u.key ≠ k do
    if k < u.key then
        u = u.left                    % Sotto-albero di sinistra
    else
        u = u.right                   % Sotto-albero di destra
return u
```

Ricorsiva

```
TREE lookupNode(TREE T, ITEM k)
if T == nil or T.key == k then
    return T
else
    return lookupNode(iif(k < T.key, T.left, T.right), k)
```

Minimo-massimo

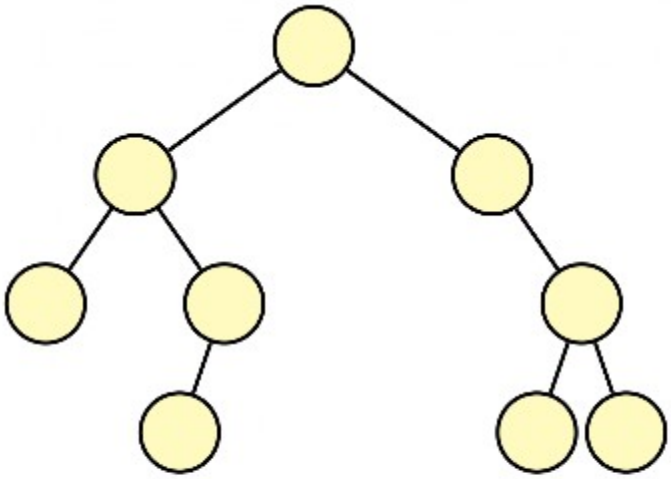


- **min** albero radice (6) ?
  - 1
- **max** albero radice (6) ?
  - 15
- **max** albero radice (2) ?
  - 4
- **min** albero radice (8) ?
  - 8

Come possiamo notare in questo esempio:

- Per trovare il minimo dalla radice prendiamo sempre il figlio sinistro fino a che non è nil
- Per trovare il massimo dalla radice prendiamo sempre il figlio destro fino a che non è nil

Infatti l'algoritmo è proprio questo:



```
TREE min(TREE T)
TREE u = T
while u.left ≠ nil do
    u = u.left
return u
```

```
TREE max(TREE T)
TREE u = T
while u.right ≠ nil do
    u = u.right
return u
```

Successore-predecessore

Il **successore** di un nodo u è il nodo che ha il più piccolo valore maggiore del valore di u.

Dobbiamo però dividere in casi:

- Se u **ha figlio destro**
  Allora il successore è il **minimo del sottoalbero destro** di u
- Se u **non ha figlio destro**
  Allora il successore, risalendo tra i padri, è il **primo avo** v tale per cui u **si trova** nel **sottoalbero sinistro** di v

Possiamo applicare lo stesso ragionamento per quanto riguarda il **predecessore**, scambiando destra con sinistra e min con max, ecco le implementazioni:

```
TREE predecessorNode(TREE t)
if t == nil then
    return t
if t.left ≠ nil then                % Caso 1
    return max(t.left)
else                                % Caso 2
    TREE p = t.parent
    while p ≠ nil and t == p.left do
        t = p
        p = p.parent
    return p
```

```
TREE successorNode(TREE t)
if t == nil then
    return t
if t.right ≠ nil then               % Caso 1
    return min(t.right)
else                                % Caso 2
    TREE p = t.parent
    while p ≠ nil and t == p.right do
        t = p
        p = p.parent
    return p
```

Inserimento

Abbiamo la funzione $Tree\ \ insertNode(Tree\ \ T,\ \ Item\ \ k,\ \ Item\ \ v)$ che inserisce l'associazione chiave-valore (k, v) nell'albero T. Questa funzione inserisce l'associazione nel caso in cui la chiave non sia già presente, altrimenti sostituisce il valore della vecchia associazione.
Se $T == nil$ (l'albero T è vuoto), viene creato un nuovo albero con radice il nuovo nodo, altrimenti viene restituito T (che rimane inalterato in quanto il puntatore alla radice è sempre quello).

Quindi per implementare $insert(Item\ \ k,\ \ Item\ \ v)$ basta usare
$Tree\ \ insertNode(Tree\ \ T,\ \ Item\ \ k,\ \ Item\ \ v)$:

Implementazione dizionario

```
insert(ITEM k, ITEM v)
tree = insertNode(tree, k, v)
```

Vediamo ora l'implementazione di $Tree\ \ insertNode(Tree\ \ T,\ \ Item\ \ k,\ \ Item\ \ v)$:

```
TREE insertNode(TREE T, ITEM k, ITEM v)
TREE p = nil                                          % Padre
TREE u = T
while u ≠ nil and u.key ≠ k do      % Cerca posizione inserimento
    p = u
    u = iif(k < u.key, u.left, u.right)
if u ≠ nil and u.key == k then
    u.value = v                                       % Chiave già presente
else
    TREE new = Tree(k, v)      % Crea un nodo coppia chiave-valore
    link(p, new, k)
    if p == nil then
        T = new                                       % Primo nodo ad essere inserito
return T          % Restituisce albero non modificato o nuovo nodo
```

```
link(TREE p, TREE u, ITEM k)
if u ≠ nil then
    u.parent = p                                      % Registrazione padre
if p ≠ nil then
    if k < p.key then p.left = u     % Attaccato come figlio sinistro
                 else p.right = u    % Attaccato come figlio destro
```

Cancellazione

Abbiamo la funzione *Tree removeNode(Tree T, Item k)* che rimuove il nodo contenente la chiave k dall'albero T, e successivamente restituisce la radice dell'albero (che potrebbe esser stata modificata se la vecchia radice era il nodo da rimuovere).

Quindi per implementare *remove(Item k)* basta usare *Tree removeNode(Tree T, Item k)*:

Implementazione dizionario

```
remove(ITEM k)
tree = removeNode(tree, k)
```

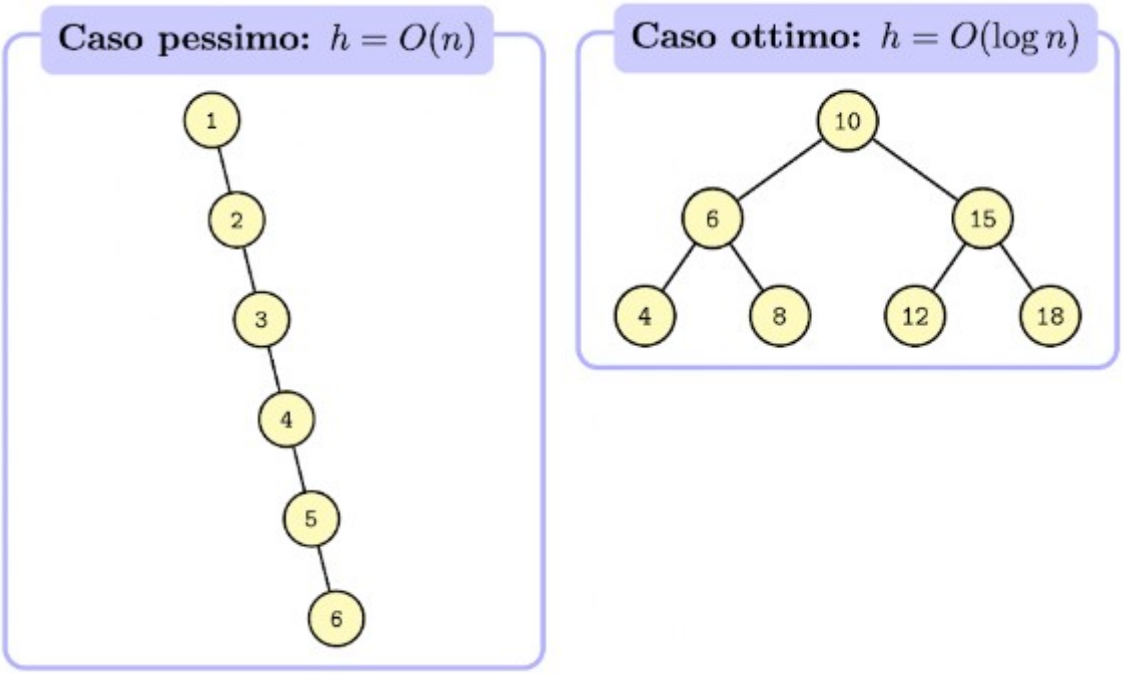
Vediamo ora l'implementazione di $Tree\ \ removeNode(Tree\ \ T,\ \ Item\ \ k)$, facendo prima qualche osservazione, dividendo il problema in casi:

1. Se il nodo da eliminare **non ha figli**
   Semplicemente si elimina (cambiando la reference del padre p rispetto a questo figlio in null).
   Non serve eseguire nessuna altra operazione perchè eliminare una foglia non influisce in alcun modo sull'ordine dei nodi rimanenti
2. Il nodo da eliminare **ha un solo figlio**
   Chiamiamo questo figlio f, in questo caso dobbiamo eliminare u e sistemare le reference in modo che f diventi figlio di p e p diventi padre di f
   Se il nostro nodo u è il figlio sinistro di p, allora tutti i valori del sottoalbero del figlio f di u sono minori di p. Di conseguenza possiamo mettere f come figlio sinistro di p. (La stessa cosa vale nel caso in cui u sia figlio destro di p)
3. Il nodo da eliminare **ha due figli**
   In questo caso troviamo il successore del nostro nodo, chiamiamolo s. Stacchiamo s (notiamo che è un successore di un nodo che ha per forza il figlio destro, quindi è un minimo e di conseguenza non ha figli sinistri, di conseguenza basta "linkare" il padre di s col figlio destro di s).
   Adesso copiamo s in u e eliminiamo il nodo s.
   Il successore s di u è sicuramente maggiore dei nodi nel sottoalbero sinistro di u ed è sicuramente minore dei nodi del sottoalbero destro di u, di conseguenza può essere sostituito ad u.

```
TREE removeNode(TREE T, ITEM k)
TREE t
TREE u = lookupNode(T, k)
if u ≠ nil then
    if u.left == nil and u.right == nil then                    % Caso 1
        link(u.parent, nil, k)
        delete u
    else if u.left ≠ nil and u.right ≠ nil then                 % Caso 3
        TREE s = successorNode()
        link(s.parent, s.right, s.key)
        u.key = s.key
        u.value = s.value
        delete s
```

```
    else if u.left ≠ nil and u.right == nil then                    % Caso 2
        link(u.parent, u.left, k)
        if u.parent = nil then
            T = u.left
    else
        link(u.parent, u.right, k)
        if u.parent = nil then
            T = u.right
return T
```

Tutte queste operazioni al massimo possono eseguire un cammino semplice da una radice ad una foglia, quindi se l'altezza dell'albero equivale ad h il tempo di ricerca è $O(h)$.
Il caso **pessimo** è quello in cui l'altezza dell'albero è n (ovvero il numero di nodi), quindi è una sequenza (ad esempio se ogni nodo ha un solo figlio), in questo caso ovviamente il tempo di ricerca è $O(n)$.
Il caso **ottimo** invece è quando l'altezza dell'albero è $\log\ n$, e questo avviene quando i nodi sono ben distribuiti, ovvero abbiamo un **albero bilanciato**, in questo caso il tempo di ricerca è $O(\log\ n)$.



Alberi binari di ricerca bilanciati

Come abbiamo detto precedentemente, l'altezza di un ABR (Albero Binario di Ricerca) è $O(n)$ nel caso pessimo.

Possiamo dimostrare invece che se facciamo degli inserimenti in **ordine casuale**, l'altezza media è $O(\log\ n)$, però la situazione inizia a farsi complicata quando ci sono delle cancellazioni.
Nella realtà ovviamente non ci affidiamo "al caso", quindi utilizziamo delle tecniche per mantenere in **maniera sicura e controllata** il nostro albero bilanciato. Mantenere un albero bilanciato è importante perché altrimenti perdiamo l'efficienza della ricerca binaria (che può diventare una ricerca lineare nel caso pessimo, appunto).

Il **fattore di bilanciamento** di un nodo v, denotato da $\beta(v)$, è la massima differenza di altezza fra i sottoalberi di v. Ci sono varie tipologie di alberi che permettono di mantenere $\beta(v)\leq 1$, e in certi casi anche $\beta(v)=0$, per qualsiasi nodo v.
In genere non ha senso tenere l'albero **completamente bilanciato** (ovvero con $\beta(v)=0$) perché l'analisi costi-benefici non sarebbe vantaggiosa.

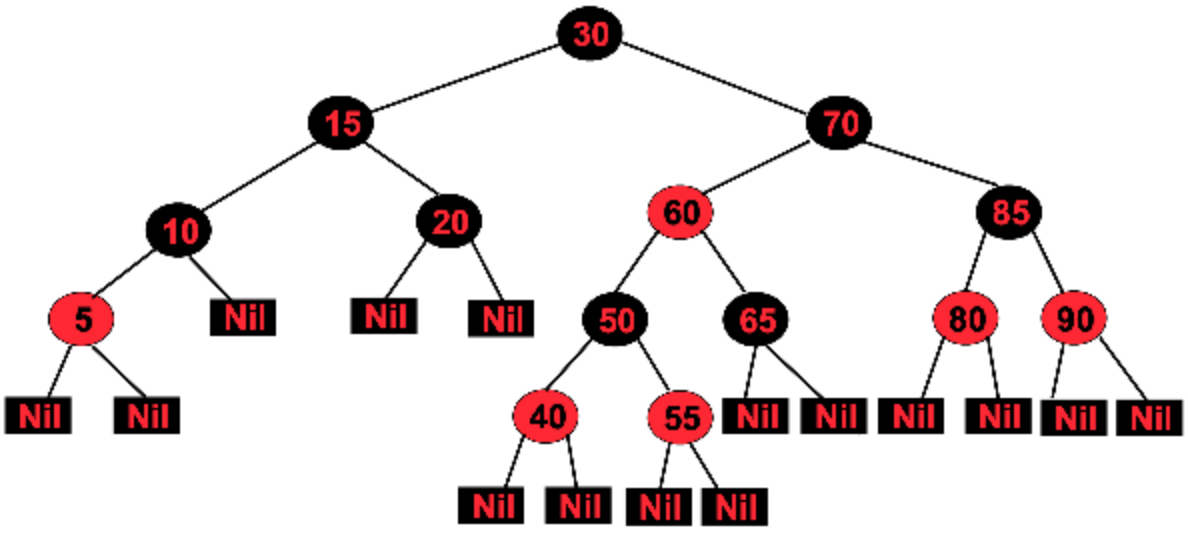
Noi vedremo gli alberi **Red-Black**, i quali sono degli **alberi binari di ricerca** in cui:

- Ad ogni nodo viene aggiunto un campo **colore**, che può essere **rosso** o **nero**
- Le **chiavi** vengono mantenute solo nei **nodi interni** dell'albero
- Le foglie (non avendo chiavi) sono dei nodi speciali **Nil**

Gli alberi Red-Black per mantenere il bilanciamento devono rispettare i seguenti vincoli:

1. La **radice** è **nera**
2. Tutte le **foglie** sono **nere** (e sono quei nodi speciali Nil)
3. Entrambi i **figli** di un **nodo rosso** sono **neri**
4. **Ogni cammino semplice** da un nodo u ad una delle foglie contenute nel suo sottoalbero **ha lo stesso numero di nodi neri**

Ecco un esempio di albero Red-Black (notiamo che rispetta tutti e 4 i vincoli):

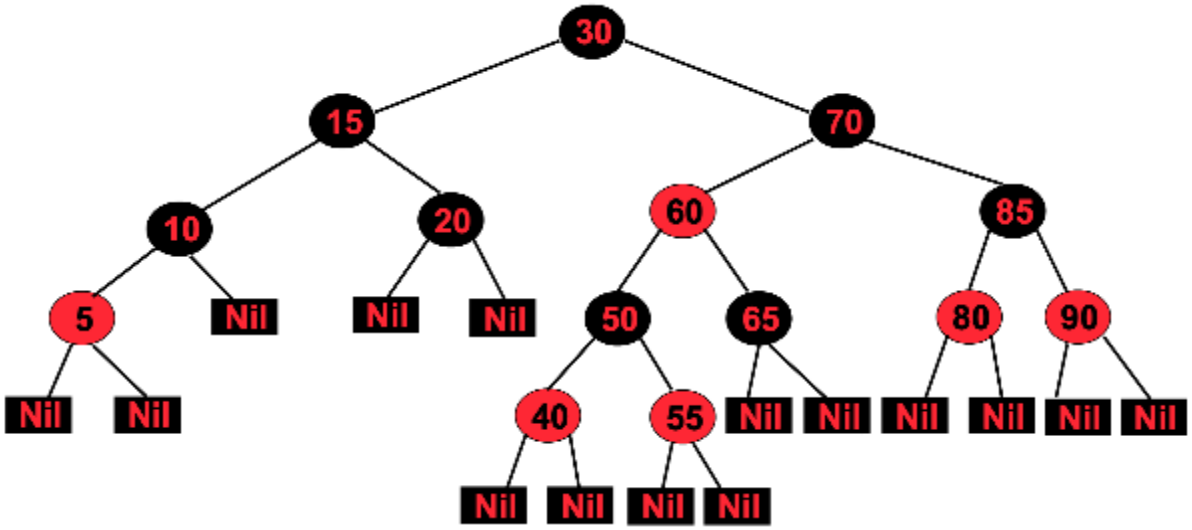


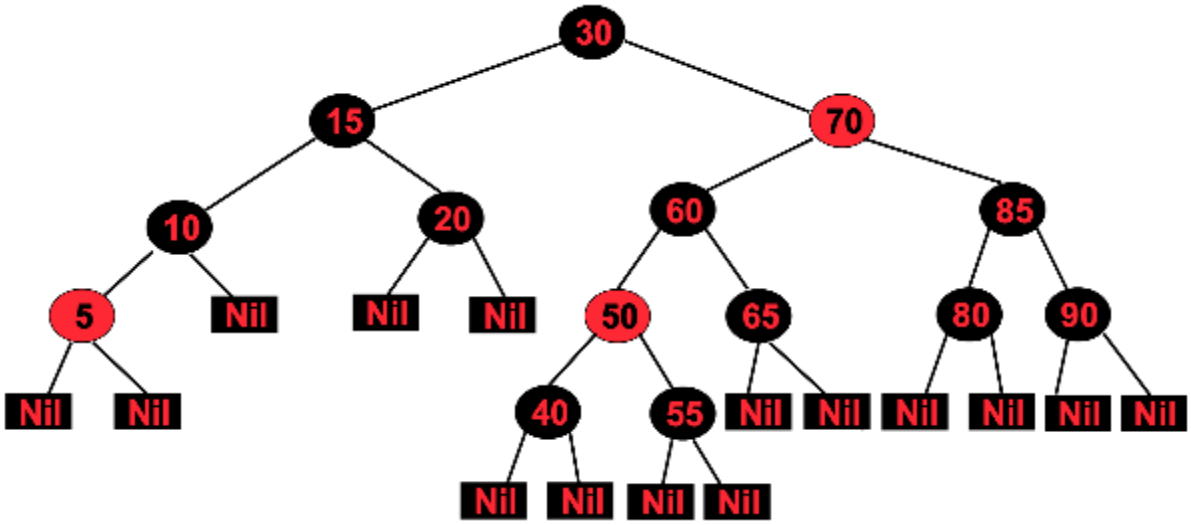
I **nodi Nil** vengono usati per comodità di spiegazione e scrittura di algoritmi e sono dei **nodi sentinella**, infatti così facendo possiamo sempre accedere al colore dei figli, anche quando i figli “non esistono”.
Nella pratica non vengono creati tanti nodi Nil quante sono le foglie, ma ne esiste solo uno che viene collegato al posto di tutte le foglie, per risparmiare memoria.
Ecco la struttura di un nodo di un Albero RB:

TREE
TREE *parent*
TREE *left*
TREE *right*
int *color*
ITEM *key*
ITEM *value*

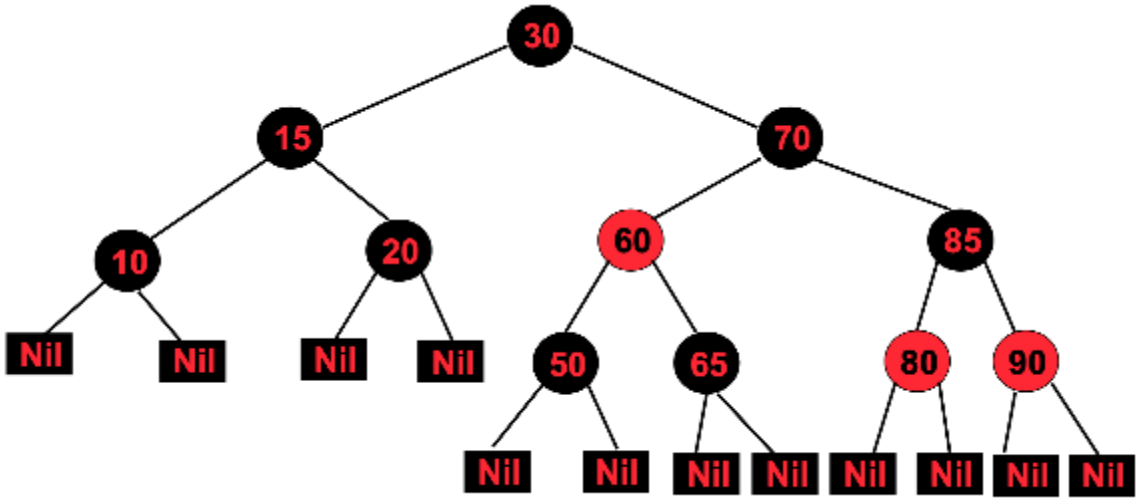
L’ **altezza nera** di un nodo v, denotata da $bh(v)$ (black height) è il **numero di nodi neri** lungo ogni cammino da v (escluso) ad ogni foglia (inclusa) del suo sottoalbero.
L’**altezza nera di un Albero RB** è pari alla altezza nera della radice.
Notiamo che entrambe queste altezze sono ben definite grazie al vincolo 4, che dice che tutti i cammini hanno lo stesso numero di nodi neri.

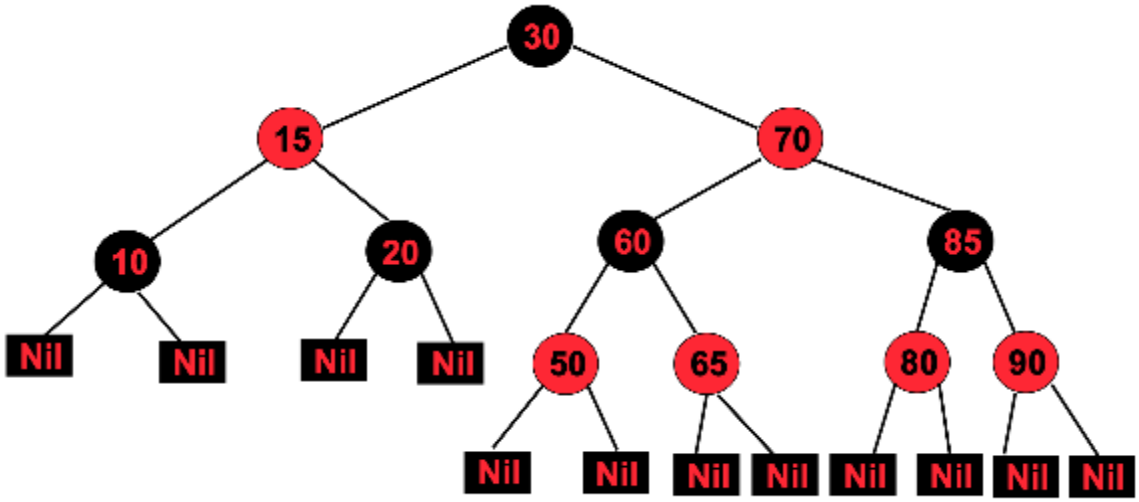
Lo stesso Albero RB può avere diverse configurazioni di colori, e diverse configurazioni possono anche cambiare l’altezza nera dell’albero.
Ecco un esempio:

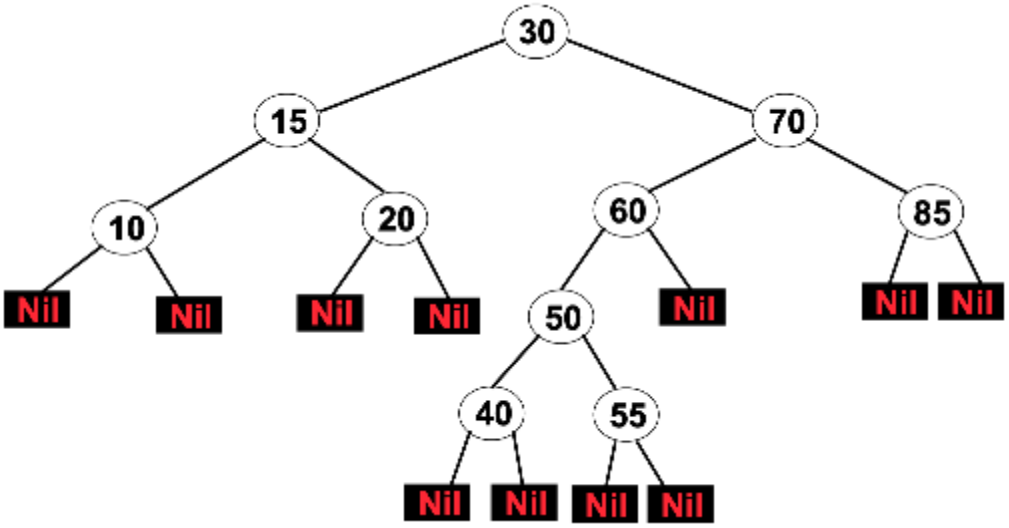




Cambiare la configurazione di colori può cambiare anche l'altezza nera di un Albero RB, ecco un esempio:







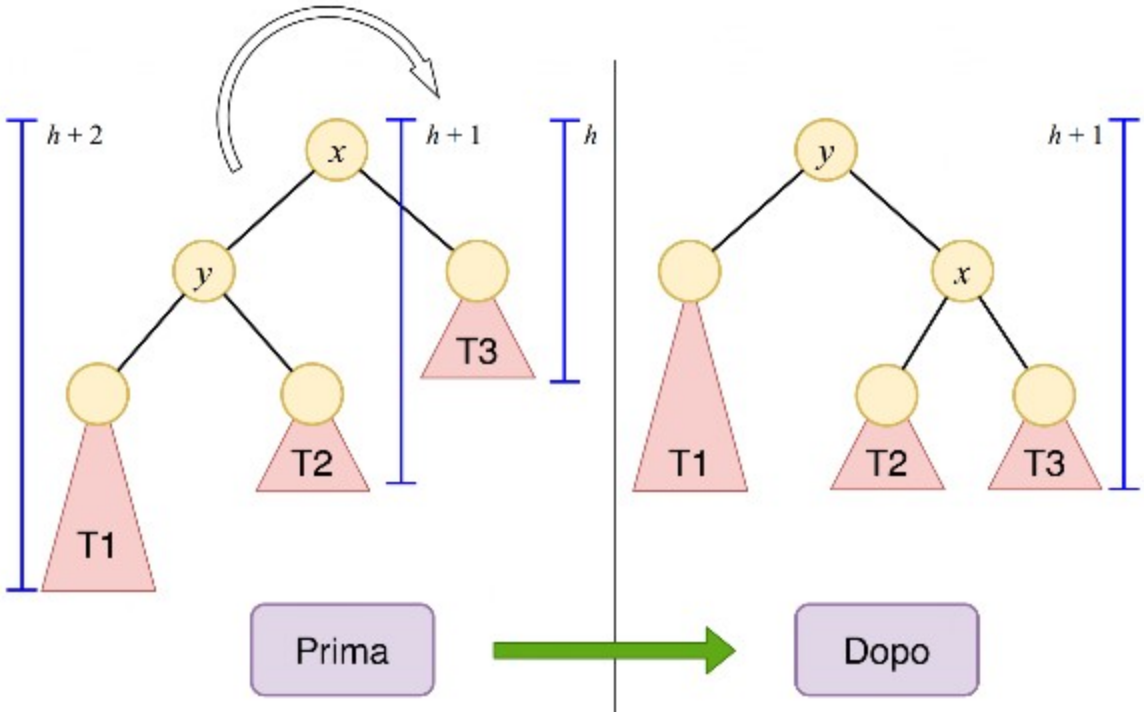
Questo albero non può essere un Albero RB (non esistono configurazioni di colori che rispettano i vincoli)

Rotazioni

Quando effettuiamo una modifica ad un Albero RB (ad esempio inserimento o cancellazione) è possibile che vengano **violati i vincoli**, in queste situazioni possiamo agire in due modi:

- Modificando la configurazione di colori
- Operando dei ribilanciamenti dell'albero tramite **rotazioni** (a destra o a sinistra)

Ecco una rappresentazione grafica di una rotazione verso destra su x

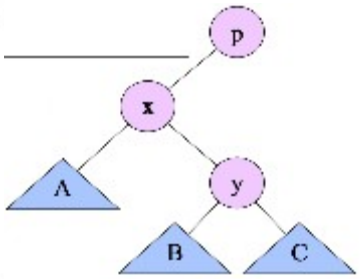


Per effettuare una rotazione (verso sinistra) dobbiamo:

1. far diventare B figlio destro di x

2. far diventare x il figlio sinistro di y
3. far diventare y figlio di p, il vecchio padre di x

```
TREE rotateLeft(TREE x)
    TREE y ← x.right
    TREE p ← x.parent
(1) x.right ← y.left                    % Il sottoalbero B diventa figlio destro di x
(1) if y.left ≠ nil then y.left.parent ← x
(2) y.left ← x                          % x diventa figlio sinistro di y
(2) x.parent ← y
(3) y.parent ← p                        % y diventa figlio di p
(3) if p ≠ nil then
        if p.left = x then p.left ← y else p.right ← y
    return y
```



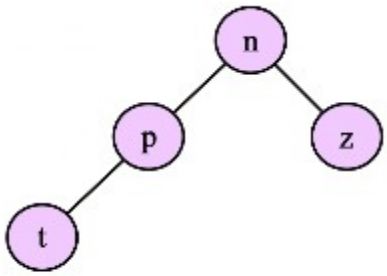
Inserimento

Per l'inserimento si cerca la posizione usando la stessa procedura di un normale albero binario di ricerca. Successivamente coloriamo il nuovo nodo di rosso. Per colorare il nodo di rosso e gestire tutti i casi di un inserimento chiamiamo la procedura *balanceInsert*()

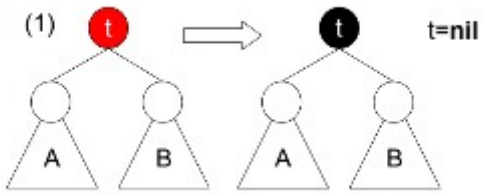
```
TREE insertNode(TREE T, ITEM k, ITEM v)
TREE p = nil                                          % Padre
TREE u = T
while u ≠ nil and u.key ≠ k do                        % Cerca posizione inserimento
    p = u
    u = iif(k < u.key, u.left, u.right)
if u ≠ nil and u.key == k then
    u.value = v                                       % Chiave già presente
else
    TREE new = Tree(k, v)                             % Crea un nodo coppia chiave-valore
    link(p, new, k)
    balanceInsert(new)
    if p == nil then
        T = n                                         % Primo nodo ad essere inserito
return T                                              % Restituisce albero non modificato o nuovo nodo
```

Vediamo ora nello specifico la procedura *balanceInsert*, che è molto complessa in quanto si possono verificare diverse situazioni nell'inserimento.
Il principio generale è quello di salire verso l'alto, mettendo a posto il vincolo 3 (ovvero che un nodo rosso ha due figli neri), spostando queste violazioni verso l'alto, e al termine coloriamo la radice di nero (Le operazioni di ripristino sono necessarie solo quando due nodi consecutivi sono rossi).
Vediamo i nodi che sono coinvolti in questa procedura:

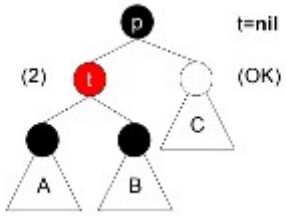
- **Nodi coinvolti**
  - Il nodo inserito t
  - Suo padre p
  - Suo nonno n
  - Suo zio z



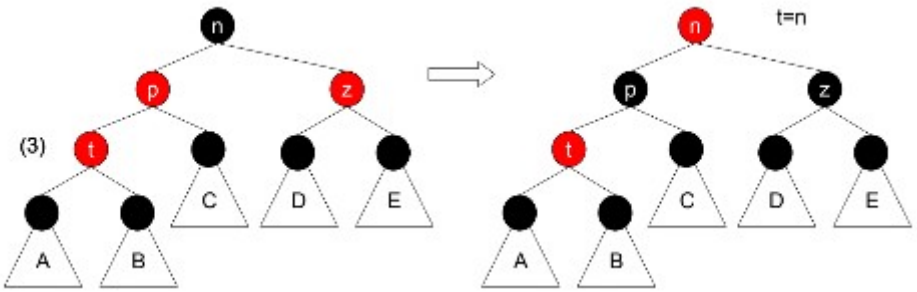
Caso 1:
Il nuovo nodo t **non ha padre** (ovvero $p = nil$).
Quindi t è il primo nodo inserito dell'albero, oppure siamo semplicemente risaliti fino alla radice nella risalita di controllo.
In questo caso basta **colorare** t **di nero**.



Caso 2:
Il **padre** p del nuovo nodo t è **nero**.
In questa situazione non abbiamo violato nessun vincolo.



Caso 3:
Il **padre** p del nuovo nodo t è **rosso** e lo **zio** z del nuovo nodo t è **rosso**.
Di conseguenza il nonno n è nero, per il vincolo 3.
In questa situazione possiamo colorare p e z di nero, colorando n di rosso (questo perchè tutti i cammini che passano per p o per z passano anche per n, quindi l'altezza nera dei cammini non cambia).
Ora però potremmo avere dei problemi sul nonno (se n è radice oppure se n ha un padre rosso), ma visto che abbiamo già affrontato questi problemi nel caso 1 e 2 ci basta porre $t = n$ e ricominciare il ciclo.



Caso 4a e 4b:

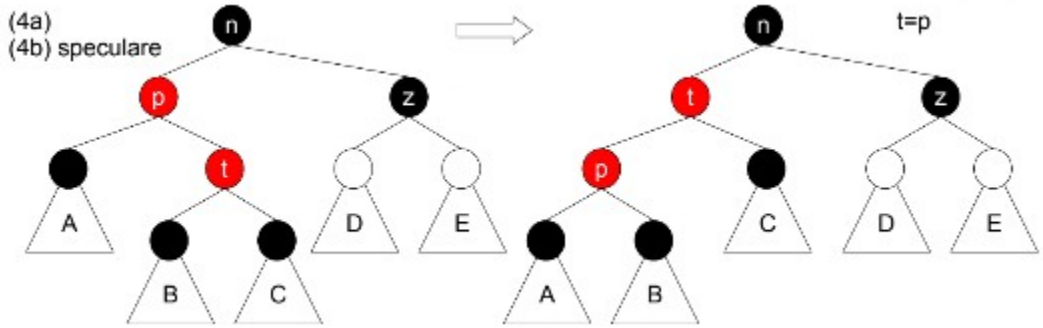
Il **padre** p del nuovo nodo t è **rosso** e lo **zio** z del nuovo nodo t è **nero**. In questo caso t è figlio **destro** di p e p è figlio **sinistro** di n.

Con una rotazione a **sinistra** su p ci riconduciamo al caso **5a**, dove i nodi rossi in conflitto sul vincolo 3 sono entrambi figli **sinistri** dei loro padri.

Poniamo $t = p$ così da ricominciare il ciclo e effettuare il caso **5a**

Essendo p e t entrambi rossi, le altezze nere dei cammini restano invariate.

(Scambiano destro con sinistro otteniamo il caso 4b, che ci riconduce al caso 5b)

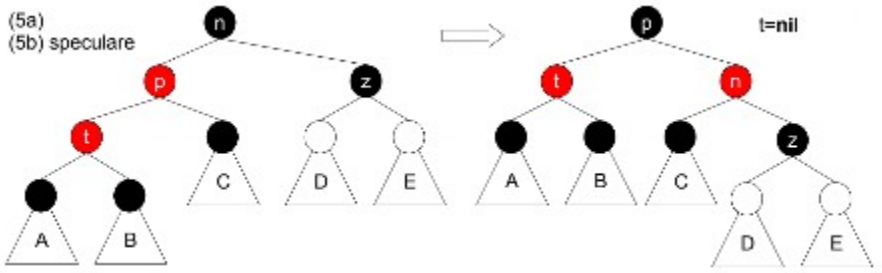


Caso 5a e 5b:

Il **padre** p del nuovo nodo t è **rosso** e lo **zio** z del nuovo nodo t è **nero**. In questo caso t è figlio **sinistro** di p e p è figlio **sinistro** di n. Notiamo che n è nero in quanto padre di un nodo rosso p

Con una rotazione a **destra** su n arriviamo in una situazione in cui t ed n sono figli di p, però abbiamo che t rosso è figlio di p rosso e abbiamo modificato le altezze nere.

Modificando p in nero e n in rosso ci troviamo in una situazione in cui tutti i vincoli Red-Black sono rispettati, e in particolare la lunghezza dei cammini neri che passano per la radice è uguale alla situazione precedente.



```
balanceInsert(TREE t)
  t.color ← RED
  while t ≠ nil do
    TREE p ← t.parent                                              % Padre
    TREE n ← iif(p ≠ nil, p.parent, nil)                           % Nonno
    TREE z ← iif(n = nil, nil, iif(n.left = p, n.right, n.left))   % Zio
    if p = nil then                                                % Caso (1)
      t.color ← BLACK
      t ← nil
    else if p.color = BLACK then                                   % Caso (2)
      t ← nil
    else if z.color = RED then                                     % Caso (3)
      p.color ← z.color ← BLACK
      n.color ← RED
      t ← n
    else
      if (t = p.right) and (p = n.left) then                       % Caso (4.a)
        n.left ← rotateLeft(p)
        t ← p
      else if (t = p.left) and (p = n.right) then                  % Caso (4.b)
        n.right ← rotateRight(p)
        t ← p
      else
        if (t = p.left) and (p = n.left) then                      % Caso (5.a)
          n.left ← rotateRight(n)
        else if (t = p.right) and (p = n.right) then               % Caso (5.b)
          n.right ← rotateLeft(n)
        p.color ← BLACK
        n.color ← RED
        t ← nil
```

Altezza

Teorema

In un Albero RB, un sottoalbero di radice *u* contiene $n \geq 2^{bh(u)} - 1$ nodi interni (ovvero non foglie Nil)

Dimostrazione. Dimostrazione per induzione sull'altezza (non sull'altezza nera).

- **Caso base**: $h = 0$. Allora u deve essere una foglia **nil** e il sottoalbero con radice in u contiene esattamente almeno $n \geq 2^{b(u)} - 1 = 2^0 - 1 = 0$ nodi interni.
- **Passo induttivo:** $h > 1$. Allora v è un nodo interno con due figli. Ogni figlio v ha un'altezza nera $b(v)$ pari a $b(u)$ o a $b(u) - 1$, a seconda del suo colore (rosso: $b(u)$, nero: $b(u) - 1$). Usando l'ipotesi induttiva, possiamo dire che ogni figlio ha almeno $2^{b(u)-1} - 1$ nodi interni. Quindi, il sottoalbero con radice in u ha almeno:

$$n \geq 2^{b(u)-1} - 1 + 2^{b(u)-1} - 1 + 1 = 2^{b(u)} - 1$$

Teorema

In un albero RB, almeno la metà dei nodi dalla radice ad una foglia deve essere nera.

Dimostrazione

Dimostrazione

- Per il vincolo 2, se un nodo è rosso, i suoi figli devono essere neri.
- La situazione in cui sono presenti il minor numero di nodi neri è il caso in cui rossi e neri sono alternati
- Quindi, almeno la metà dei nodi deve essere nera.

Teorema
In un albero RB, dati due cammini dalla radice a due foglie, non è possibile che uno sia più lungo del doppio dell'altro.
Dimostrazione

Dimostrazione

- Per il vincolo 4, ogni cammino da un nodo ad una qualsiasi foglia contiene lo stesso numero di nodi neri.
- Per il Lemma precedente, almeno metà dei nodi in ognuno di questi cammini sono neri.
- Quindi, al limite, uno dei due cammini è costituito da solo nodi neri, mentre l'altro è costituito da nodi neri e rossi alternati.

Teorema
L'altezza massima di un Albero RB con n nodi interni è al più $2 \cdot \log(n+1)$.
Dimostrazione
Prendiamo il caso in cui l'albero ha altezza massima, ovvero ci sono più nodi rossi possibili, ovvero $bh(r) = \frac{h}{2}$, perché l'altra metà di nodi sono rossi.

Dimostrazione

$$\begin{aligned} n \geq 2^{bh(r)} - 1 &\Leftrightarrow n \geq 2^{h/2} - 1 \\ &\Leftrightarrow n + 1 \geq 2^{h/2} \\ &\Leftrightarrow \log(n+1) \geq h/2 \\ &\Leftrightarrow h \leq 2\log(n+1) \end{aligned}$$

Visto che abbiamo ottenuto che l'altezza è limitata superiormente da un logaritmo possiamo dire che la complessità dell'inserimento è composta da:

- $O(\log\ n)$ per scendere fino al punto di inserimento
- $O(1)$ per effettuare l'inserimento
- $O(\log\ n)$ per risalire e aggiustare (questo succede solo nel caso 3)

Di conseguenza la complessità totale è $2 \cdot O(\log\ n) = O(\log\ n)$ (esistono degli inserimenti top-down, ovvero si scende fino al punto di inserimento aggiustando nel mentre l'albero, così si inserisce il nodo in una foglia senza altri problemi).

Cancellazione

L'algoritmo di cancellazione per alberi Red-Black è costruito sull'algoritmo di cancellazione per alberi binari di ricerca. Semplicemente dopo la cancellazione bisogna decidere se è necessario ribilanciare o meno.
Però le **operazioni di ripristino** del bilanciamento sono necessarie **solo quando** il **nodo cancellato** è **nero**.
Questo perché se il nodo cancellato è rosso:

- L'altezza nera rimane invariata
- Non sono stati creati nodi rossi consecutivi
- La radice resta nera

Invece se il nodo cancellato è nero:

- Vincolo 1: La radice potrebbe essere un nodo rosso (viene preso il minimo del sottoalbero destro che potrebbe essere rosso)
- Vincolo 3: Se il padre e uno dei figli del nodo cancellato sono rossi
- Vincolo 4: L'altezza nera cambia eliminando un nodo nero

Vediamo in che modo l'algoritmo *balanceDelete* viene implementato con rotazioni e cambiamenti di colore per ribilanciare l'albero. (Il codice non è importante, infatti è scritto a metà giusto per avere un'idea).

```
balanceDelete(TREE T, TREE t)
while t ≠ T and t.color = BLACK do
    TREE p = t.parent                                          % Padre
    if t = p.left then
        TREE f = p.right                                       % Fratello
        TREE ns = f.left                                       % Nipote sinistro
        TREE nd = f.right                                      % Nipote destro
        if f.color == RED then                                 % (1)
            p.color = RED
            f.color = BLACK
            rotateLeft(p)
            % t viene lasciato inalterato, quindi si ricade nei casi (2),(3),(4)
        else
            if ns.color == nd.color == BLACK then              % (2)
                f.color = RED
                t = p
            else if ns.color == RED and nd.color == BLACK then % (3)
                ns.color = BLACK
                f.color = RED
                rotateRight(f)
                % t viene lasciato inalterato, quindi si ricade nel caso (4)
            else if nd.color == RED then                       % (4)
                f.color = p.color
                p.color = BLACK
                nd.color = BLACK
                rotateLeft(p)
                t = T
    else
```

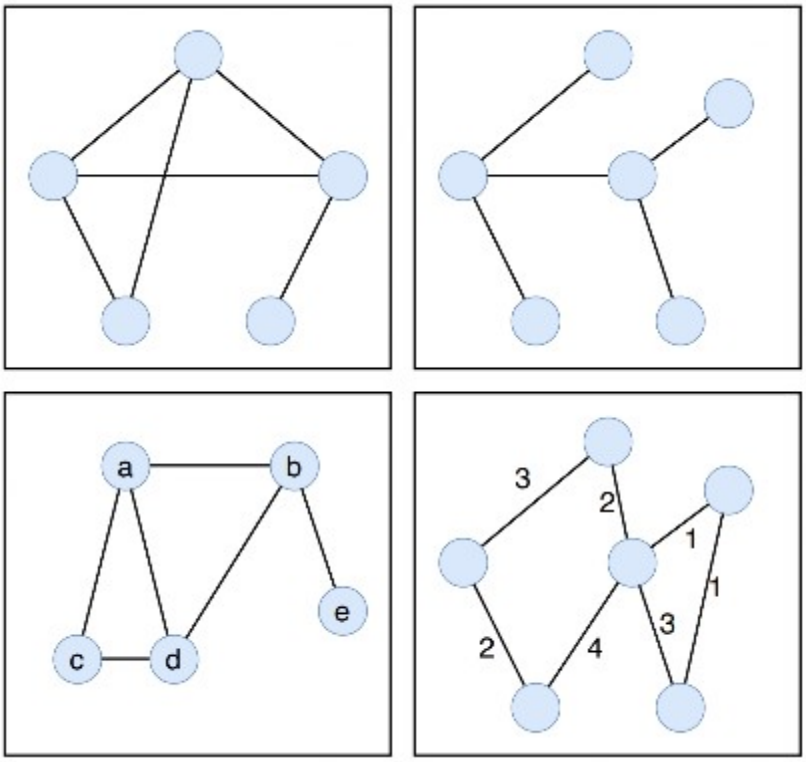
In sintesi:

- Dal caso 1 si passa ad uno dei casi 2,3,4
- Dal caso 2 si torna indietro ad uno degli altri casi, ma risalendo di un livello l'albero
- Dal caso 3 si passa al caso 4
- Nel caso 4 si termina

Anche in questo caso è possibile visitare al massimo un numero $O(\log\ n)$ di casi, ognuno dei quali è gestito in tempo $O(1)$.

Grafi

Ecco degli esempi di grafi:



I **problemi** principali nei grafi sono:

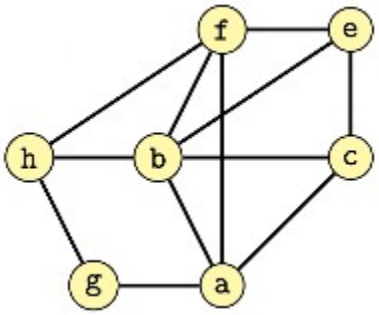
- Nei grafi **non pesati**
    - Ricerca del cammino più breve (misurato in numero di archi)
    - Trovare le componenti (fortemente) connesse
    - Verificare la ciclicità
    - L'ordinamento topologico
- Nei grafi **pesati**
    - Trovare il cammino di peso minimo
    - Costruire alberi di copertura di peso minimo
    - Flusso massimo

L'importanza dei grafi è enorme in moltissimi problemi di vario tipo, infatti:
Moltissimi problemi possono essere visti come problemi su grafi. Sebbene i problemi abbiano forma astratta, le loro applicazioni si trovano poi negli ambiti più disparati
Degli esempi sono:

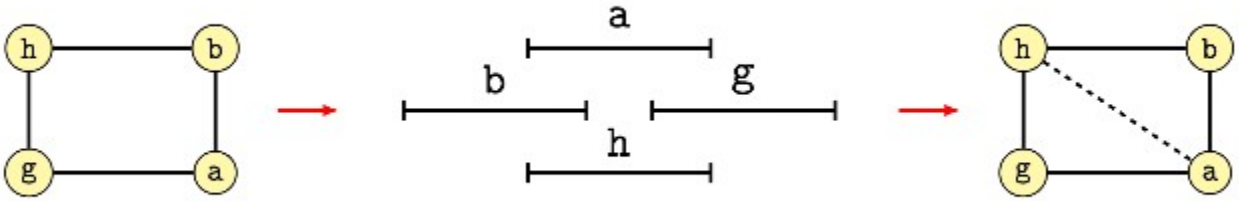
- Grado di conoscenza di LinkedIN (cammino più breve nella rete sociale)
- Ordinamento topologico per stabilire un ordine di azioni in un grafo di dipendenze
- Gli algoritmi di model checking utilizzati per la verifica formale del software sono basati sull'identificazione delle componenti fortemente connesse

Un esempio più pratico è quello della "morte del duca MacPollock", in cui vengono interrogate 7 donne, ognuna delle quali confessa quali altre donne ha incontrato durante la visita del castello (assumiamo che le donne siano state al castello una sola volta nella loro vita, apparte

l'assassino, e che dicano tutte la verità per quanto riguarda gli incontri), così costruiamo un grafo:



Notiamo però che se prendiamo 4 nodi, collegati sul "perimetro", allora ci deve essere per forza almeno un collegamento "interno", altrimenti costruendo la linea temporale abbiamo delle incongruenze:



E notiamo che il nodo che non rispetta più volte questa regola è il nodo a, quindi è lui ad aver effettuato almeno 2 visite al castello.

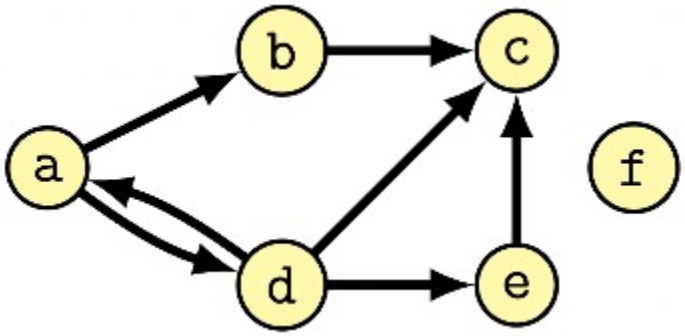
Definizioni

Un **grafo orientato** (o directed) è una coppia $G=(V,E)$ dove:

- V è un insieme di **nodi** o **vertici**
- E è un insieme di **coppie ordinate** di nodi (u,v) dette **archi** o **lati**

Ecco un esempio:

```
V = { a,b,c,d,e,f }
E = { (a,b), (a,d),
      (b,c), (d,a),
      (d,c), (d,e),
      (e,c) }
```

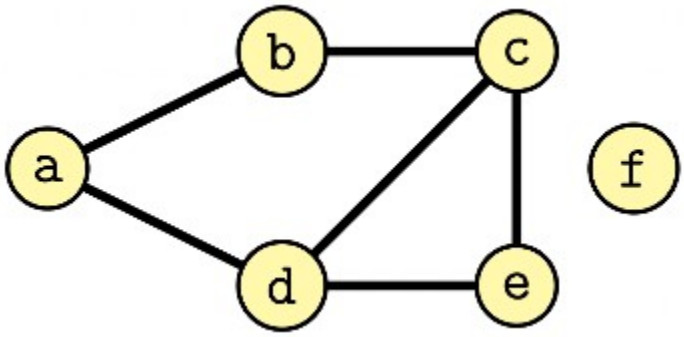


Un **grafo non orientato** (o undirected) è una coppia $G=(V,E)$ dove:

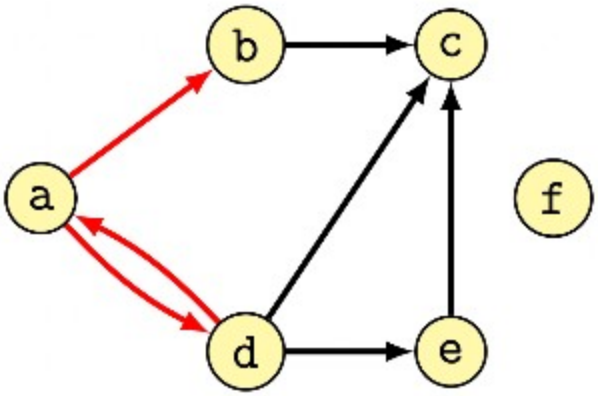
- V è un insieme di **nodi** o **vertici**
- E è un insieme di **coppie non ordinate** di nodi (u,v) dette **archi** o **lati**

Ecco un esempio:

```
V = { a,b,c,d,e,f }
E = { (a,b), (a,d),
      (b,c), (c,d),
      (d,e), (c,e)  }
```



Un vertice v è detto **adiacente** a u se esiste un arco (u,v)
Un arco (u,v) è detto **incidente** da u a v



- (a,b) è incidente da a a b
- (a,d) è incidente da a a d
- (d,a) è incidente da d a a
- b è adiacente a a
- d è adiacente a a
- a è adiacente a d

In un grafo non orientato le relazioni di adiacenza e incidenza sono simmetriche.

Il **numero di nodi** n di un grafo $G=(V,E)$ lo denoteremo anche con $|V|$ (la cardinalità dell'insieme dei nodi)

$$n=|V|$$

Il **numero di archi** m di un grafo $G=(V,E)$ lo denoteremo anche con $|E|$ (la cardinalità dell'insieme degli archi)

$$m=|E|$$

In un **grafo orientato** vale:

$$m \leq n(n-1)=O(n^2)$$

Questo perché ogni nodo può avere al massimo $n-1$ nodi adiacenti, di conseguenza gli archi saranno al massimo $n(n-1)$ quando il grafo è completamente connesso (o completo).

In un **grafo non orientato** invece, dobbiamo prendere la metà in quanto il numero di archi massimi è dimezzato:

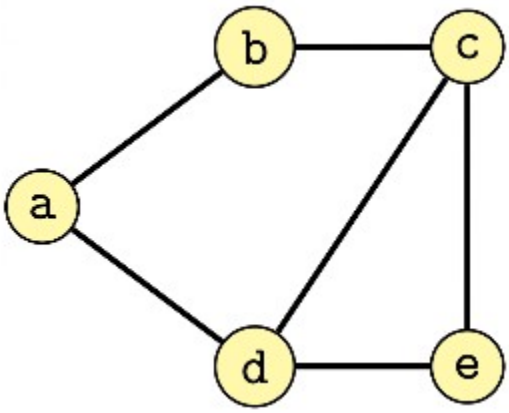
$$m \leq \frac{n(n-1)}{2}=O(n^2)$$

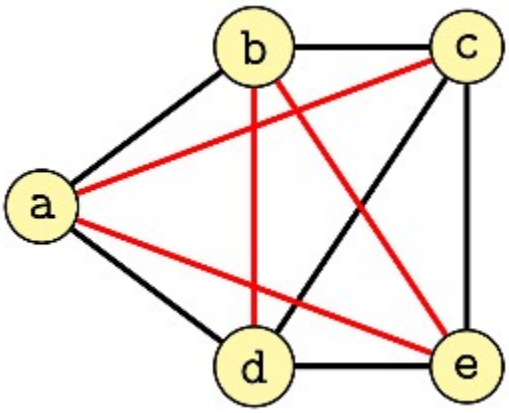
La complessità degli algoritmi sui grafi è espressa in termini sia di n che di m, in quanto entrambi i valori vanno ad influire sulle procedure (ad esempio $O(n+m)$).

Un grafo si dice **completo** se esiste un arco per ogni coppia di vertici.
Un grafo si dice:

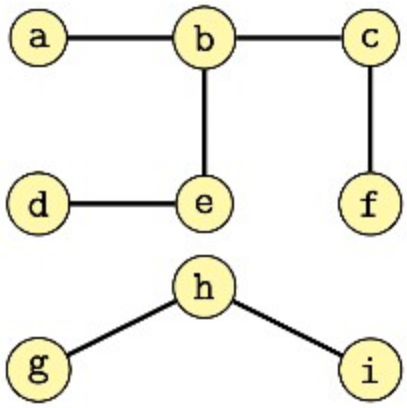
- **Sparso**
  Se ha “pochi archi”, ad esempio se $m = O(n)$ o $m = O(n \ \log \ n)$
- **Denso**
  Se ha “tanti archi”, ad esempio se $m = \Omega(n^2)$

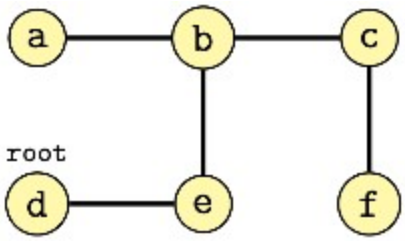
I concetti di pochi e tanti archi sono **informali**, ovvero non esiste una definizione vera e propria, ecco un esempio dove possiamo dire che il primo grafo è sparso e il secondo è denso:





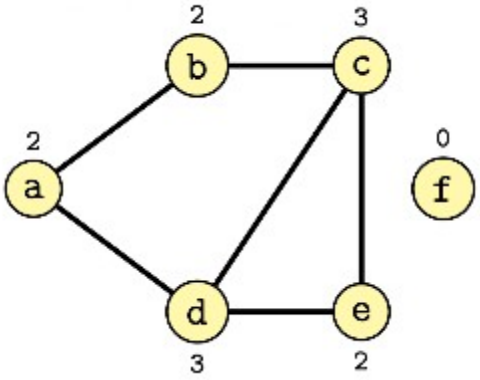
Un **albero libero** (free tree) è un grafo connesso con $m = n - 1$.
Un **albero radicato** (rooted tree) è un grafo connesso con $m = n - 1$ nel quale uno dei nodi è designato come **radice**
Un insieme di alberi è un grafo detto **foresta**

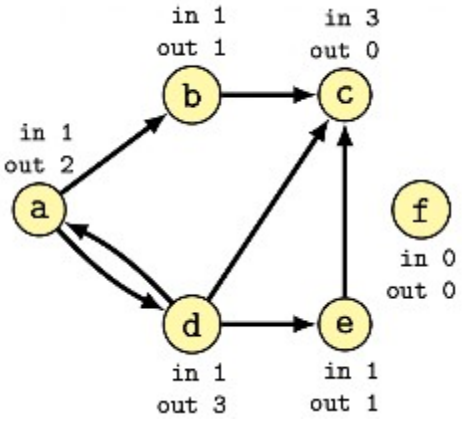




Il **grado** di un nodo è:

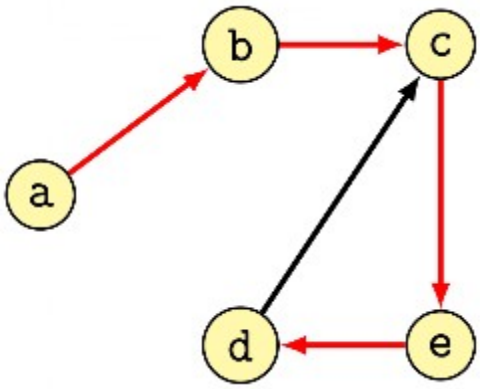
- In un grafo non orientato
  Il numero di archi incidenti su di esso (essendo una relazione simmetrica è uguale al numero di archi che lo contengono)
- In un grafo orientato
  - Grado **entrante**
    Il numero di archi incidenti **su** di esso
  - Grado **uscente**
    Il numero di archi incidenti **da** esso





In un grafo $G=(V,E)$, orientato o meno, un **cammino** C di lunghezza k è una sequenza di nodi $u_0,u_1,\ldots,u_k$ tale che $(u_i,u_{i+1}) \in E$ per $0 \le i \le k-1$

Ecco un esempio:



Esempio: a, b, c, e, d è un cammino nel grafo di lunghezza 4

Nota: un cammino è detto semplice se tutti i suoi nodi sono distinti

Un **cammino** è detto **semplice** se tutti i suoi nodi sono distinti.

Specifica

Vediamo ora la specifica dell'implementazione della struttura dati grafo, che nella versione più generale è una struttura dati dinamica che permette di aggiungere/rimuovere nodi e archi

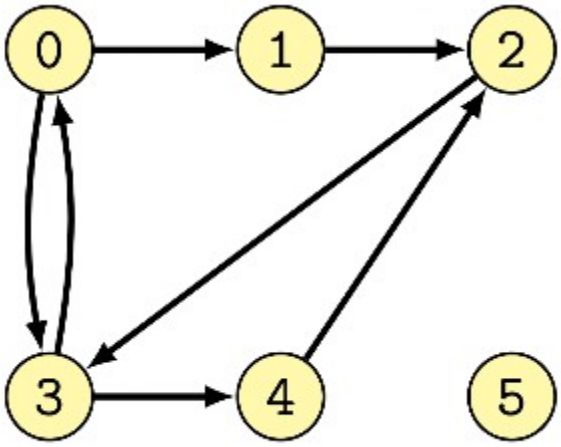
GRAPH	
Graph( )	% Crea un nuovo grafo
SET V()	% Restituisce l'insieme di tutti i nodi
int size()	% Restituisce il numero di nodi
SET adj(NODE u)	% Restituisce l'insieme dei nodi adiacenti a u
insertNode(NODE u)	% Aggiunge il nodo u al grafo
insertEdge(NODE u, NODE v)	% Aggiunge l'arco (u, v) al grafo
deleteNode(NODE u)	% Rimuove il nodo u dal grafo
deleteEdge(NODE u, NODE v)	% Rimuove l'arco (u, v) dal grafo

In alcuni casi non implementiamo le eliminazioni (ovvero *deleteNode*( ) e *deleteEdge*( )) perchè non vengono usate, in queste situazioni i grafi vengono caricati all'inizio e non vengono più modificati.

Memorizzazione di grafi

Per memorizzare i grafi ci sono due possibili approcci:

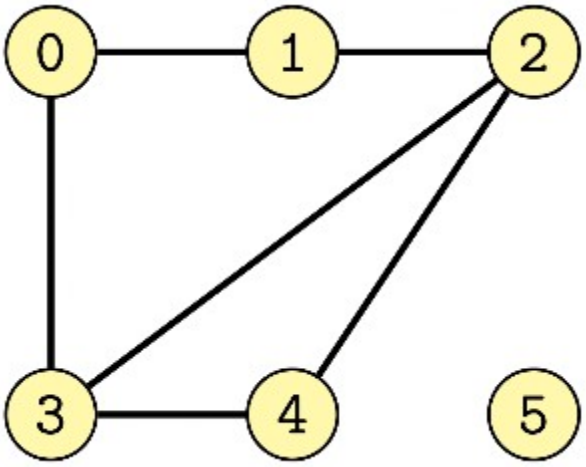
- Le matrici di adiacenza
  Abbiamo una matrice $n \times n$ e m_{uv} vale 1 se $(u, v) \in E$, altrimenti vale 0 se $(u, v) \notin E$.
  Occupiamo n^2 bit



	0	**1**	**2**	**3**	**4**	**5**
0	0	1	0	1	0	0
1	0	0	1	0	0	0
2	0	0	0	1	0	0
3	1	0	0	0	1	0
4	0	0	1	0	0	0
5	0	0	0	0	0	0

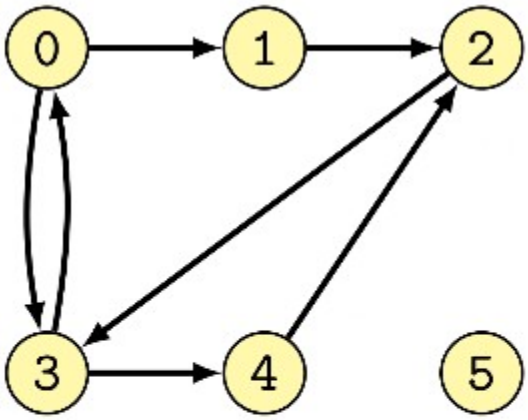
  Per un grafo non orientato notiamo che la matrice è simmetrica, quindi ci basta salvare

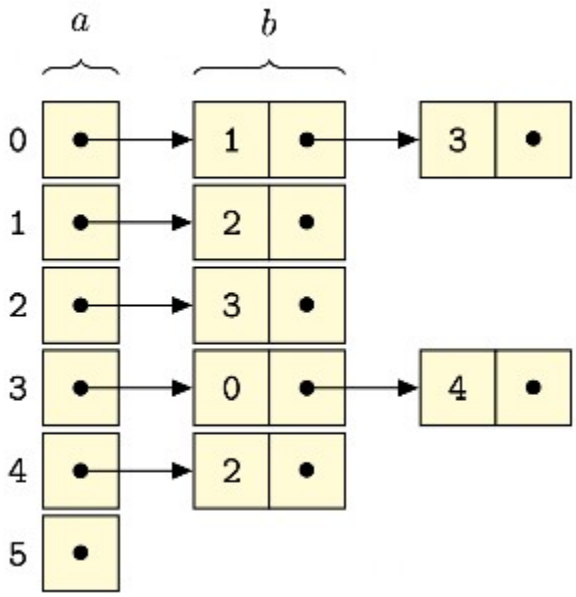
$\frac{n(n-1)}{2}$ bit (invece di n^2)



	0	1	2	3	4	5
0		1	0	1	0	0
1			1	0	0	0
2				1	1	0
3					1	0
4						0
5						

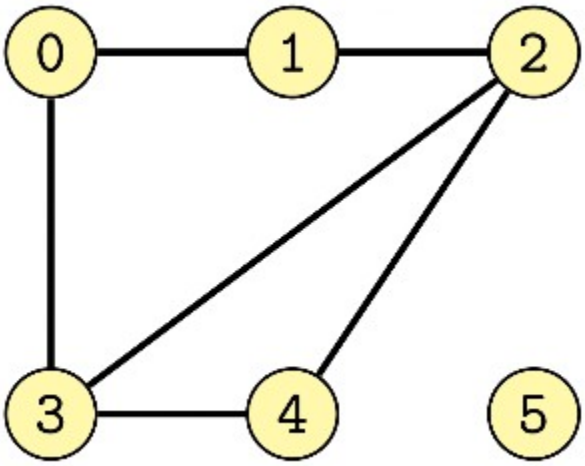
- Le liste di adiacenza
  In questa memorizzazione abbiamo per ogni vertice una lista di adiacenza, che include tutti i vertici adiacenti a quel vertice, quindi $G.adj(u) = \{v \mid (u,v) \in E\}$ e lo spazio di memorizzazione è di $an + bm$ bit

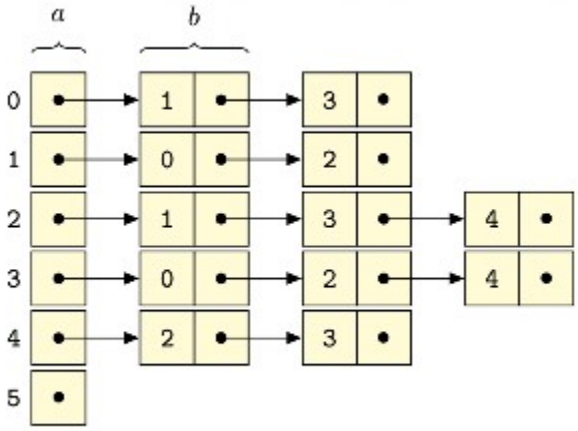




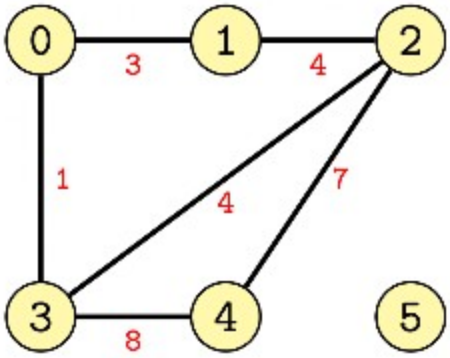
Nel caso di un grafo non orientato avremo delle ripetizioni in quanto per ogni arco

dobbiamo inserire due volte un elemento nelle liste, infatti lo spazio è $an + 2 \cdot bm$



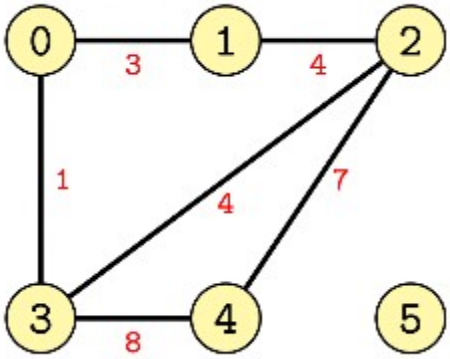


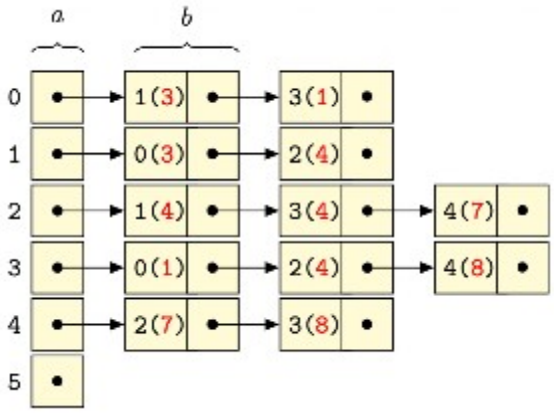
Gli archi possono avere un **peso** (costo, profitto, etc), questo peso è dato da una funzione di peso $\omega : V \times V \rightarrow R$. Se non esiste arco fra due vertici, il peso assume un valore che dipende dal problema (ad esempio 0 o $+\infty$).
Nella matrice di adiacenza in un grafo pesato inseriremo il peso dell'arco:



	0	**1**	**2**	**3**	**4**	**5**
0		3	0	1	0	0
1			4	0	0	0
2				4	7	0
3					8	0
4						0
5						

Nella lista di adiacenza dobbiamo aggiungere un attributo agli elementi delle liste che va a indicare il peso:

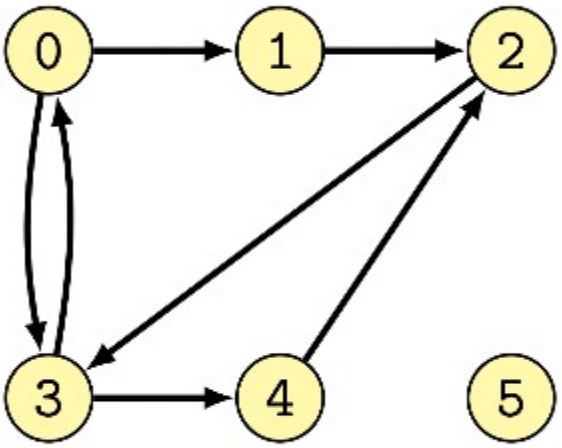


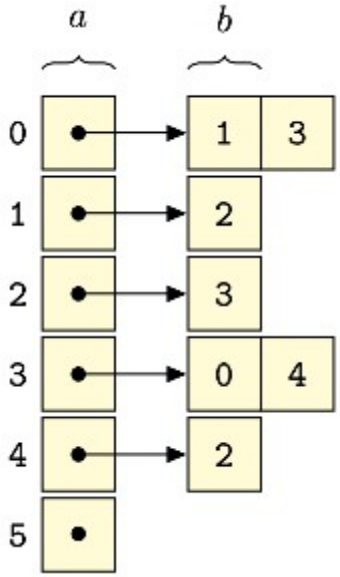


Sia il concetto di *lista di adjacenza* che il concetto di *lista dei nodi* possono essere declinati in molti modi:

Struttura	Java	C++	Python
Lista collegata	`LinkedList`	`list`	
Vettore statico	`[]`	`[]`	`list`
Vettore dinamico	`ArrayList`	`vector`	`list`
Insieme	`HashSet` `TreeSet`	`set`	`set`
Dizionario	`HashMap` `TreeMap`	`map`	`dict`

Nel caso in cui il grafo sia statico, possiamo usare i vettori al posto delle liste (**vettore di adiacenza**) occupando sempre $an+bm$ bit:





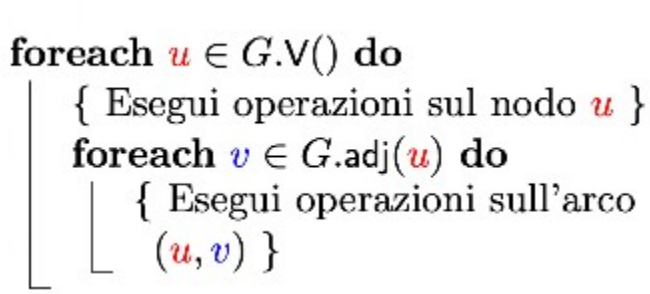
Nel seguito faremo le seguenti assunzioni per ogni problema e esempio (se non diversamente specificato):

- Assumeremo che l'implementazione sia basata su vettori di adiacenza, statici o dinamici
- Assumeremo che la classe Node sia equivalente a int (quindi l'accesso alle informazioni avrà costo $O(1)$)
- Assumeremo che le operazioni per aggiungere nodi e archi abbiano costo $O(1)$
- Assumeremo che dopo l'inizializzazione il grafo sia statico

Per eseguire un'iterazione su tutti i nodi di un grafo usiamo il seguente costrutto:

foreach $u \in G.\mathsf{V}()$ **do**
  { Esegui operazioni sul nodo u }

Se vogliamo iterare anche tutti gli archi del grafo usiamo:



Costo computazionale
- $O(m+n)$ con liste di adiacenza
- $O(n^2)$ con matrici di adiacenza

Ecco un riassunto delle due tecniche di memorizzazione:

Matrici di adiacenza
- Spazio richiesto $O(n^2)$
- Verificare se u è adiacente a v richiede tempo $O(1)$
- Iterare su tutti gli archi richiede tempo $O(n^2)$
- Ideale per grafi densi

Liste di adiacenza
- Spazio richiesto $O(n+m)$
- Verificare se u è adiacente a v richiede tempo $O(n)$
- Iterare su tutti gli archi richiede tempo $O(n+m)$
- Ideale per grafi sparsi

Visita di un grafo

La visita di un grafo consiste nel, dato un grafo $G=(V,E)$ e un vertice $r \in V$ (chiamata **radice** o **sorgente**) **visitare** una e una sola volta **tutti i nodi** del grafo che possono essere raggiunti da *r* (tramite un cammino si intende).

Esistono due tipi di visite:

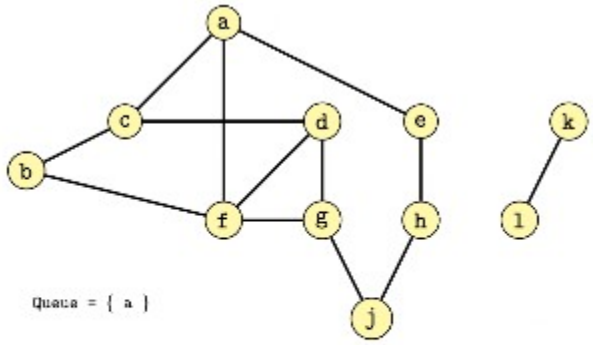
- **Visita in ampiezza** (Breadth-first search **BFS**)
  Consiste in una visita dei nodi per livelli, quindi prima si visita la radice, poi i nodi che sono a distanza 1 dalla radice, poi a distanza 2, etc.
  Questa visita può essere utile per calcolare il cammino più breve di un nodo rispetto alla radice.
- **Visita in profondità** (Depth-first search **DFS**)
  Consiste in una visita ricorsiva, quindi per ogni nodo adiacente a quello che stiamo visitando, visitiamo ricorsivamente anche questo nodo, visitando ricorsivamente tutti i nodi adiacenti a lui, etc.
  Questa visita può essere utile per l'ordinamento topologico e per trovare le componenti connesse (o fortemente connesse)

Un approccio ingenuo alla visita di un grafo è quello di scorrere tutti i nodi tramite l'insieme $G.V()$ e vedere tutti gli archi incidenti da ogni nodo. Però questo approccio non tiene in alcun modo conto della struttura del grafo e i nodi e gli archi vengono iterati tutti senza nessun criterio

```
visit(GRAPH G)
foreach u ∈ G.V() do
    { visita nodo u }
    foreach v ∈ G.adj(u) do
        { visita arco (u, v) }
```

Ecco un altro approccio che potrebbe sembrare più sensato, ispirato alla visita di un albero in cui trattiamo i nodi adiacenti come se fossero dei nodi figli:

```
BFSTraversal(GRAPH G, int r)
QUEUE Q = Queue()
Q.enqueue(r)
while not Q.isEmpty() do
    NODE u = Q.dequeue()
    { visita il nodo u }
    foreach v ∈ G.adj(u) do
        Q.enqueue(v)
```



Notiamo però che entriamo in un loop infinito visto che continuiamo a inserire i nodi adiacenti anche se sono nodi che abbiamo già visitato.

Di conseguenza ci serve **marcare** in qualche modo i nodi che abbiamo già visitato, ecco un algoritmo generico di attraversamento di un grafo che funziona:

```
graphTraversal(GRAPH G, NODE r)
SET S = Set()                                   % Insieme generico
S.insert(r)                                     % Da specificare
{ marca il nodo r }
while S.size() > 0 do
    NODE u = S.remove()                         % Da specificare
    { visita il nodo u }
    foreach v ∈ G.adj(u) do
        { visita l'arco (u, v) }
        if v non è ancora stato marcato then
            { marca il nodo v }
            S.insert(v)                         % Da specificare
```

Breadth-first search

Gli obiettivi della visita in **ampiezza** sono:

- Visitare i nodi a distanze crescenti dalla sorgente
  Quindi prima visitare tutti i nodi a distanza k e poi tutti quelli a $k+1$
- Calcolare il cammino più breve da r a tutti gli altri nodi
  Dove le distanze sono calcolate come il numero di archi attraversati
- Generare un **albero breadth-first**
  Ovvero generare un albero contenente tutti i nodi raggiungibili da r, tale per cui il cammino dalla radice r al nodo u nell'albero corrisponde al cammino più breve da r a u nel grafo

Ecco un'implementazione di una visita in ampiezza:

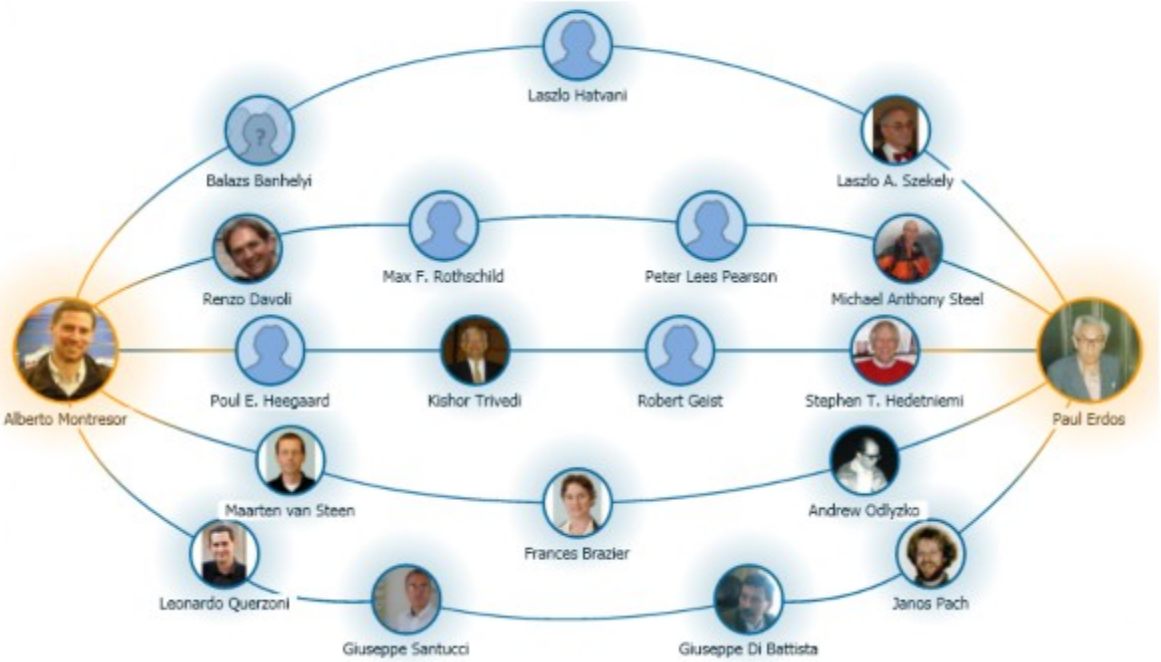
```
bfs(GRAPH G, NODE r)
QUEUE Q = Queue( )
S.enqueue(r)
boolean[ ] visited = new boolean[G.size()]
foreach u ∈ G.V() − {r} do
    visited[u] = false
visited[r] = true
while not Q.isEmpty() do
    NODE u = Q.dequeue()
    { visita il nodo u}
    foreach v ∈ G.adj(u) do
        { visita l'arco (u, v) }
        if not visited[v] then
            visited[v] = true
            Q.enqueue(v)
```

Un'applicazione di una visita BFS è quella di Paul Erdos, con il **numero di Erdos**:

- Erdos ha valore $erdos = 0$
- I co-autori di Erdos (ovvero le persone che hanno scritto un articolo con Erdos) hanno valore $erdos = 1$
- Se X è co-autore di qualcuno con $erdos = k$ e non è co-autore di qualcuno con $erdos < k$, allora X ha $erdos = k + 1$
- Le persone non raggiunte da questa definizione hanno $Erdos = +\infty$

Ecco un esempio col professore Montresor:

Alberto Montresor, $erdos = 4$



Ecco un'implementazione dell'algoritmo per calcolare il numero di Erdos:

```
distance(GRAPH G, NODE r, int[] distance)
QUEUE Q = Queue()
Q.enqueue(r)
foreach u ∈ G.V() − {r} do
    distance[u] = ∞
distance[r] = 0
while not Q.isEmpty() do
    NODE u = Q.dequeue()
    foreach v ∈ G.adj(u) do
        if distance[v] == ∞ then % Se il nodo v non è stato scoperto
            distance[v] = distance[u] + 1
            Q.enqueue(v)
```

Come abbiamo già detto la visita BFS può essere usata per ottenere **il cammino più breve** fra due nodi (misurato in numero di archi).
Possiamo quindi costruire un **albero di copertura** con radice r aggiungendo alla funzione di prima un vettore dei padri $parent$, che costruiamo durante la visita.

Così facendo è facile costruire un algoritmo che ci stampa il cammino dalla radice r ad un nodo s:

```
distance([...], NODE[] parent)
[...]
parent[r] = nil
while not S.isEmpty() do
    NODE u = S.dequeue()
    foreach v ∈ G.adj(u) do
        if distance[v] == ∞ then
            distance[v] =
              distance[u] + 1
            parent[v] = u
            S.enqueue(v)
```

```
printPath(NODE r, NODE s, NODE[] parent)
if r == s then
    print s
else if parent[s] == nil then
    print "error"
else
    printPath(r, parent[s], parent)
    print s
```

La complessità della visita BFS è $O(n+m)$.
Ognuno dei nodi n viene inserito nella coda al massimo una volta e ogni volta che un nodo viene estratto, tutti i suoi archi vengono visitati una sola volta. Di conseguenza il numero di archi analizzati è $m = \sum_{u \in V} d_{out}(u)$, dove $d_{out}(u)$ è l'out-degree del nodo u.

Depth-first-search

La visita in **profondità** è spesso una subroutine della soluzione di altri problemi.
Solitamente viene usata per esplorare tutto un grafo, e non solo le componenti connesse (come la BFS).

L'output di una visita in profondità è (invece di un albero come nella BFS) una **foresta depth-first** $G_f = (V, E_f)$, che è una collezione di alberi depth-first.

Per eseguire questa visita possiamo usare:

- Uno stack implicito (attraverso la ricorsione)
- Uno stack esplicito

Ecco un esempio di DFS con stack implicito (con ricorsione):

```
dfs(GRAPH G, NODE u, boolean[] visited)
visited[u] = true
{ visita il nodo u (pre-order) }
foreach v ∈ G.adj(u) do
    if not visited[v] then
        { visita l'arco (u, v) }
        dfs(G, v, visited)
{ visita il nodo u (post-order) }
```

Notiamo che anche qui la complessità è $O(n+m)$

Eseguire una DFS basata su chiamate ricorsive è rischioso, in quanto è possibile che, per grafi molto grandi, la profondità raggiunta sia troppo grande per la dimensione dello stack del linguaggio. In questi casi si preferisce usare una visita BFS oppure una DFS con stack esplicito, come questa:

```
dfs(GRAPH G, NODE r)
STACK S = Stack()
S.push(r)
boolean[] visited = new boolean[G.size()]
foreach u ∈ G.V() do
    visited[u] = false
while not S.isEmpty() do
    NODE u = S.pop()
    if not visited[u] then
        { visita il nodo u (pre-order) }
        visited[u] = true
        foreach v ∈ G.adj(u) do
            { visita l'arco (u, v) }
            S.push(v)
```

Note

- Un nodo può essere inserito nella pila più volte
- Il controllo se un nodo è già stato visitato viene fatto all'estrazione, non all'inserimento
- Complessità $O(m+n)$
  - $O(m)$ visite degli archi
  - $O(m)$ inserimenti, estrazioni
  - $O(n)$ visite dei nodi

Questa appena vista è una visita pre-order, visto che prima visitiamo il nodo e poi i suoi nodi adiacenti, altrimenti possiamo fare una visita post-order.

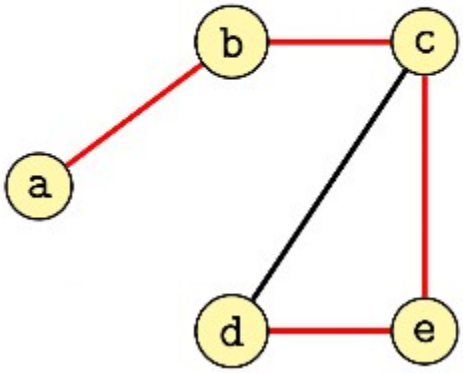
Per la visita post-order dobbiamo ingegnarci nel caso iterativo un po' di più visto che dobbiamo tener conto se un nodo è "nuovo" oppure "vecchio". Per far ciò aggiungiamo un tag *discovery* quando scopriamo un nodo, così quando estraiamo un nodo *discovery*, mettiamo il tag *finish* e inseriamo nello stack tutti i suoi nodi adiacenti, così quando i nodi adiacenti hanno finito, torniamo sul nostro nodo *finish* e sappiamo che possiamo visitarlo.

Parliamo ora di componenti connesse, visto che la maggior parte degli algoritmi sui grafi iniziano decomponendo il grafo nelle sue componenti connesse e vengono applicati ad ogni componente connessa, successivamente i risultati sono ricomposti assieme.

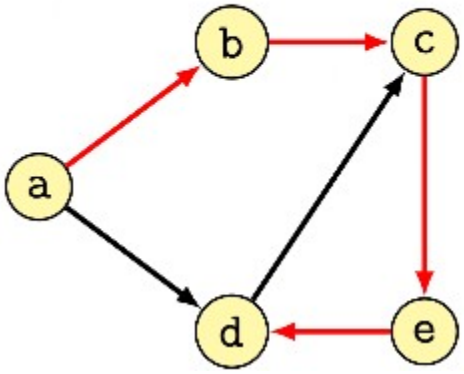
Le **componenti connesse** sono definite sui **grafi non orientati**
Le **componenti fortemente connesse** sono definite sui **grafi orientati**.

Un nodo *v* è **raggiungibile** da un nodo *u* se esiste almeno un cammino da *u* a *v*

Il nodo *d* è raggiungibile dal nodo *a* e viceversa

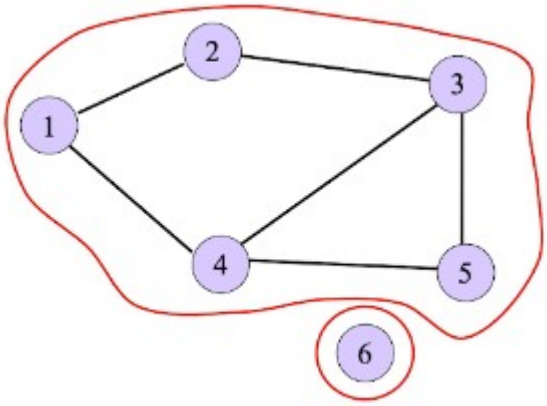


Il nodo *d* è raggiungibile dal nodo *a*, ma non viceversa



Un grafo **non orientato** $G=(V,E)$ è **connesso** se e solo se ogni suo nodo è raggiungibile da ogni altro suo nodo.
Un grafo $G'=(V',E')$ è una **componente connessa** di G se e solo se è un sottografo connesso e massimale di G:

- G' è un sottografo di G se e solo se $V' \subseteq V$ e $E' \subseteq E$
- G' è massimale se e solo se non esiste un altro sottografo G'' di G tale che G'' è connesso ed è più “grande” di G' (ovvero $G' \subseteq G'' \subseteq G$)

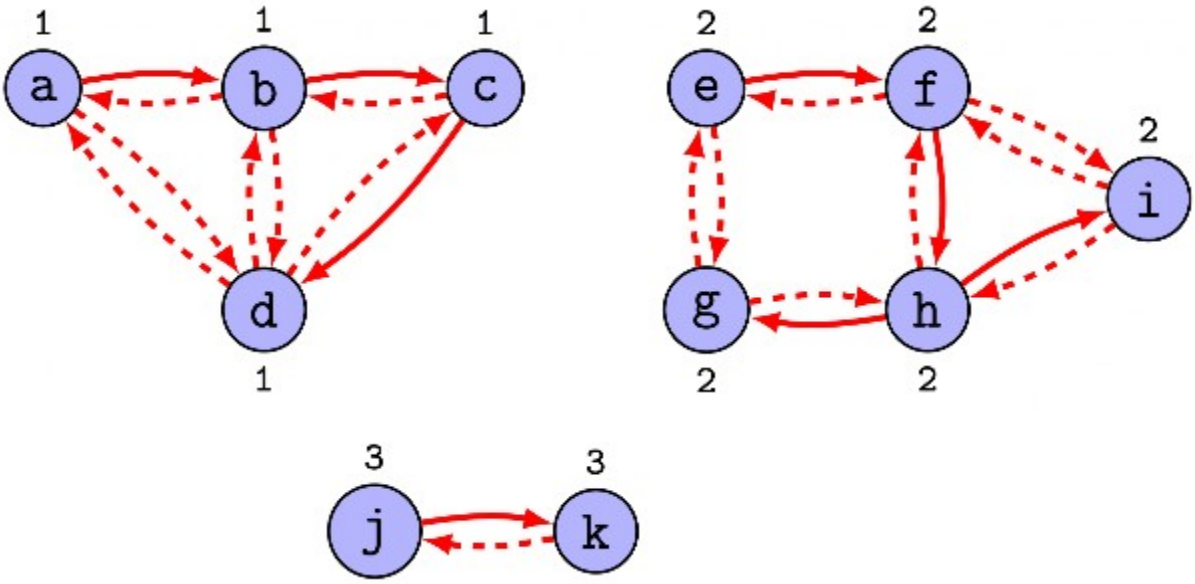


Vediamo ora come possiamo usare la visita DFS per trovare le componenti connesse di un grafo (e di conseguenza capire se è connesso, visto che basta controllare se ha una sola componente connessa).
Un grafo è **connesso** se, al termine della DFS partita da un nodo, tutti i nodi sono marcati. Altrimenti la visita deve ricominciare da capo da un nodo non marcato, identificando così una nuova componente del grafo.

Per identificare l'appartenenza ad una componente connessa usiamo un vettore id, che sarà popolato dagli identificativi delle componenti connesse. $id[u]$ è l'identificatore della CC a cui appartiene u.
Ecco l'implementazione:

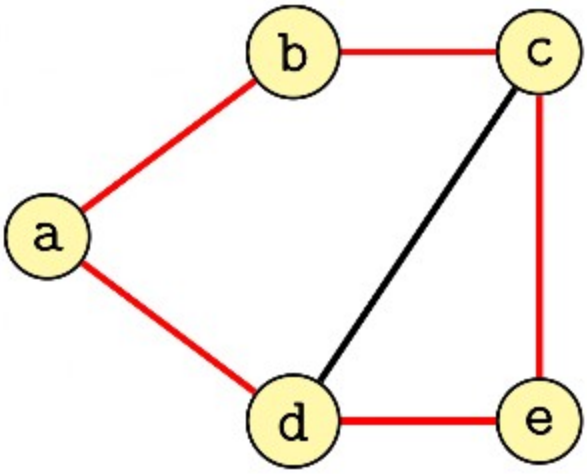
```
int[] cc(GRAPH G)
int[] id = new int[G.size()]
foreach u ∈ G.V() do
    id[u] = 0
int counter = 0
foreach u ∈ G.V() do
    if id[u] == 0 then
        counter = counter + 1
        ccdfs(G, counter, u, id)
return id
```

```
ccdfs(GRAPH G, int counter, NODE u, int[] id)
id[u] = counter
foreach v ∈ G.adj(u) do
    if id[v] == 0 then
        ccdfs(G, counter, v, id)
```



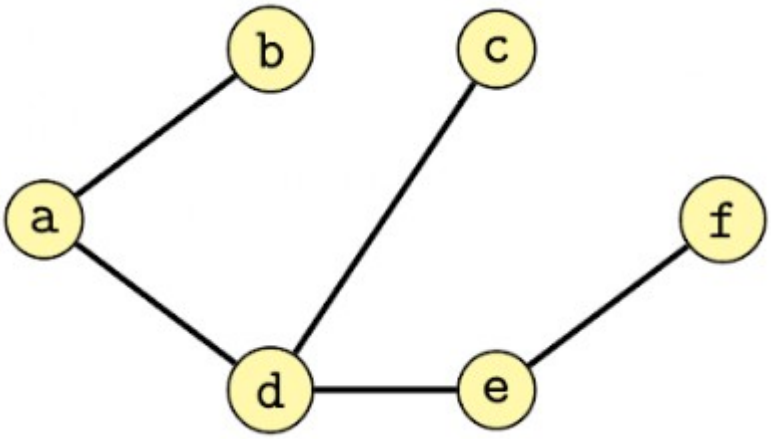
Questa procedura ha complessità $O(n+m)$.

Diamo ora la definizione di ciclo che ci servirà per altri scopi:
In un grafo **non orientato** $G=(V,E)$, un **ciclo** C di lunghezza $k>2$ è una sequenza di nodi $u_0,u_1,\ldots,u_k$ tale che $(u_i,u_{i+1}) \in E$ per $0 \le i \le k-1$ e $u_0=u_k$



$k > 2$ esclude cicli banali composti da coppie di archi (u, v) e (v, u), che sono onnipresenti nei grafi non orientati.

Un grafo non orientato che non contiene nessun ciclo è detto **grafo aciclico**

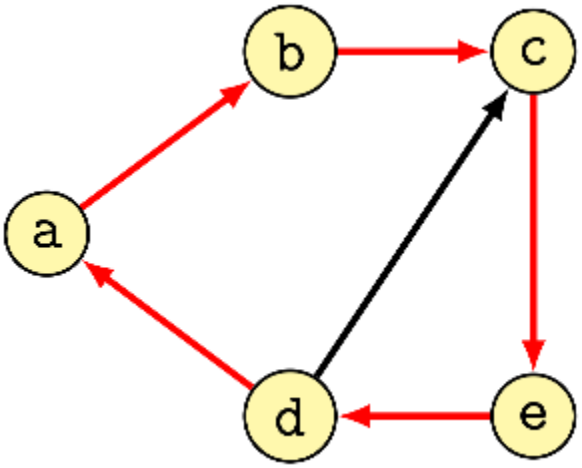


Vediamo ora un algoritmo che dato un grafo non orientato G, ci restituisca *true* se G contiene almeno un ciclo, altrimenti *false*.
L'idea è quella che se rivisito un nodo già precedentemente visitato (escludendo il "padre") allora esiste un ciclo, ovviamente devo fare questo per ogni componente connessa:

```
boolean hasCycleRec(GRAPH G, NODE u, NODE p, boolean[] visited)
visited[u] = true
foreach v ∈ G.adj(u) − {p} do
    if visited[v] then
        return true
    else if hasCycleRec(G, v, u, visited) then
        return true
return false
```

```
boolean hasCycle(GRAPH G)
boolean[] visited = new boolean[G.size()]
foreach u ∈ G.V() do
    visited[u] = false
foreach u ∈ G.V() do
    if not visited[u] then
        if hasCyclerec(G, u, null, visited) then
            return true
return false
```

In un grafo **orientato** $G=(V,E)$, un **ciclo** C di lunghezza $k>2$ è una sequenza di nodi $u_0,u_1,\ldots,u_k$ tale che $(u_i,u_{i+1}) \in E$ per $0 \le i \le k-1$ e $u_0=u_k$ (notiamo che non cambia nulla dalla definizione del ciclo per un grafo orientato, solo che qui cambia il concetto di arco)

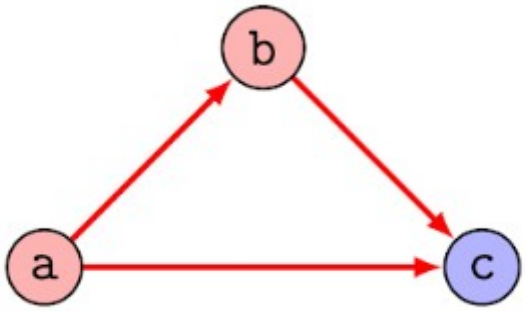


Esempio: a,b,c,e,d,a è un cammino nel grafo di lunghezza 5

Note: un ciclo è detto semplice se tutti i suoi nodi sono distinti (ad esclusione del primo e dell'ultimo)

Un grafo orientato e aciclico, quindi che non contiene cicli, è detto **DAG** (Directed Acyclic Graph). Se un grafo contiene cicli è detto **ciclico**.

In un grafo orientato, l'algoritmo che abbiamo visto per i grafi non orientati non va bene, perchè possiamo avere in risposta che esistono dei cicli anche quando non esistono, come nel seguente esempio:

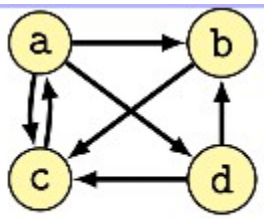


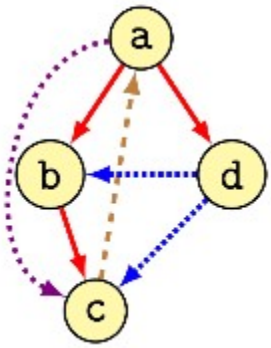
Per risolvere questo problema introduciamo il concetto di **albero di copertura DFS**.
Abbiamo quattro tipologie di archi in un albero di copertura DFS T:

- **Arco dell'albero**
  Quando l'arco va da un nodo marcato u ad un nodo non marcato v
- Archi non inclusi nell'albero (quando v è già marcato)
  - **Arco in avanti**
    Se u è un antenato di v in T
  - **Arco all'indietro**
    Se u è un discendente di v in T
  - **Arco di attraversamento**
    Se u non è né antenato né discendente di v in T

Gli archi (u, v) non inclusi nell'albero possono essere divisi in tre categorie

- Se u è un antenato di v in T, (u, v) è detto arco in avanti
- Se u è un discendente di v in T, (u, v) è detto arco all'indietro
- Altrimenti, viene detto arco di attraversamento





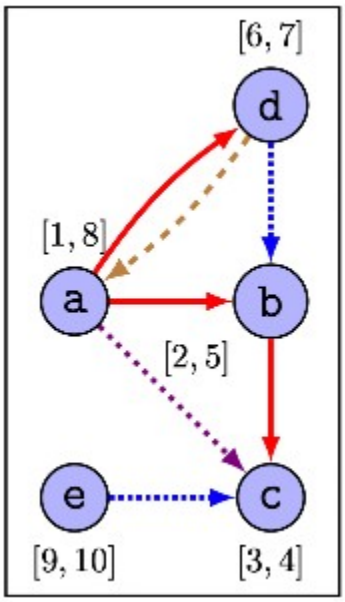
Per fare un algoritmo che costruisca questo albero denotando tutte le tipologie di archi, utilizziamo una variabile *time* che è un contatore, e due array che andranno ad indicare a che tempo abbiamo iniziato a visitare il nodo e a che tempo abbiamo finito (così da capire sia se il nodo è già stato visitato e sia per capire se è discendente, antenato)

```
dfs-schema(GRAPH G, NODE u, int &time, int[] dt, int[] ft)
{ visita il nodo u (pre-order) }
time = time + 1;   dt[u] = time
foreach v ∈ G.adj(u) do
    { visita l'arco (u, v) (qualsiasi) }
    if dt[v] == 0 then
        { visita l'arco (u, v) (albero) }
        dfs-schema(G, v, time, dt, ft)
    else if dt[u] > dt[v] and ft[v] == 0 then
        { visita l'arco (u, v) (indietro) }
    else if dt[u] < dt[v] and ft[v] ≠ 0 then
        { visita l'arco (u, v) (avanti) }
    else
        { visita l'arco (u, v) (attraversamento) }
{ visita il nodo u (post-order) }
time = time + 1;   ft[u] = time
```

- *time*: contatore
- *dt*: discovery time (tempo di scoperta)
- *ft*: finish time (tempo di fine)

Ecco un esempio di applicazione dell'algoritmo:

```
dfs-schema(GRAPH G, NODE u, int &time, int[] dt, int[] ft)
time = time + 1;   dt[u] = time
foreach v ∈ G.adj(u) do
    if dt[v] == 0 then
        { visita l'arco (u, v) (albero) }
        dfs-schema(G, v, time, dt, ft)
    else if dt[u] > dt[v] and ft[v] == 0 then
        { visita l'arco (u, v) (indietro) }
    else if dt[u] < dt[v] and ft[v] ≠ 0 then
        { visita l'arco (u, v) (avanti) }
    else
        { visita l'arco (u, v) (attraversamento) }
time = time + 1;   ft[u] = time
```



Potremmo dire che classificare gli archi che non fanno parte dell'albero è insensato visto che al momento non servono a niente. Però possiamo dimostrare delle proprietà del grafo proprio partendo dalle proprietà del tipo degli archi per costruire algoritmi migliori.

Teorema

Data una visita DFS di un grafo $G=(V,E)$, per ogni coppia di nodi $u, v \in V$, solo una delle seguenti condizioni è vera:

- Gli intervalli $[dt[u], ft[u]]$ e $[dt[v], ft[v]]$ sono **non sovrapposti**
  Allora u e v **non sono discendenti uno dell'altro** nella foresta DF
- L'intervallo $[dt[u], ft[u]]$ è contenuto in $[dt[v], ft[v]]$
  Allora u **è discendente** di v in un albero DF
- L'intervallo $[dt[u], ft[u]]$ contiene $[dt[v], ft[v]]$
  Allora u **è antenato** di v in un albero DF

Dimostrazione.

- $dt[u] < dt[v]$: Ci sono due sottocasi da considerare:
  - $dt[v] < ft[u]$: questo significa che v è stato scoperto dopo il discovery time di u, ma prima del finish time di u; il che significa che v è un discendente di u. Inoltre, poichè v è stato scoperto più recentemente di u, verrà completato prima del completamento di u; ne segue che i vari tempi coinvolti seguono quest'ordine:
    $$dt[u] < dt[v] < ft[v] < ft[u]$$
    quindi l'intervallo $[dt[v], ft[v]]$ è completamente contenuto in $[dt[u], ft[u]]$.
  - $ft[u] < dt[v]$: in questo caso i due intervalli sono completamente disgiunti, in quanto l'ordine ottenuto è il seguente:
    $$dt[u] < ft[u] < dt[v] < ft[v]$$
    Ne segue anche che u e v non sono discendenti l'uno dell'altro.
- $dt[u] > dt[v]$: questo caso è simmetrico, con il ruolo di u e v scambiati.

Possiamo quindi formulare il seguente teorema:
Un grafo orientato è aciclico SE E SOLO SE non esistono archi all'indietro nel grafo

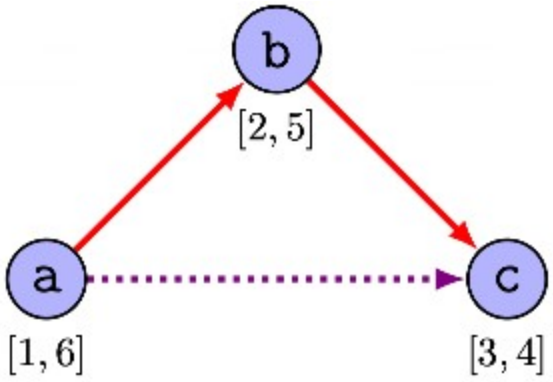
Dimostrazione

- **se**: Se esiste un ciclo, sia u il primo nodo del ciclo che viene visitato e sia (v, u) un arco del ciclo. Il cammino che connette u ad v verrà prima o poi visitato, e da v verrà scoperto l'arco all'indietro (v, u).
- **solo se**: Se esiste un arco all'indietro (u, v), dove v è un antenato di u, allora esiste un cammino da v a u e un arco da u a v, ovvero un ciclo.

Possiamo quindi costruire il seguente algoritmo, che non appena trova un arco all'indietro si ferma e ritorna true, ovvero che il grafo è ciclico:

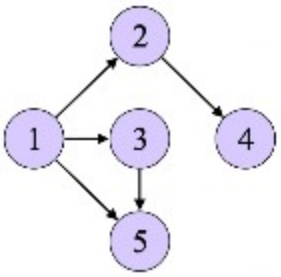
```
boolean hasCycleRec(GRAPH G, NODE u, int &time, int[] dt, int[] ft)
time = time + 1;   dt[u] = time
foreach v ∈ G.adj(u) do
    if dt[v] == 0 then
        if hasCycleRec(G, v, time, dt, ft) then
            return true
    else if dt[u] > dt[v] and ft[v] == 0 then
        return true
time = time + 1;   ft[u] = time
return false
```

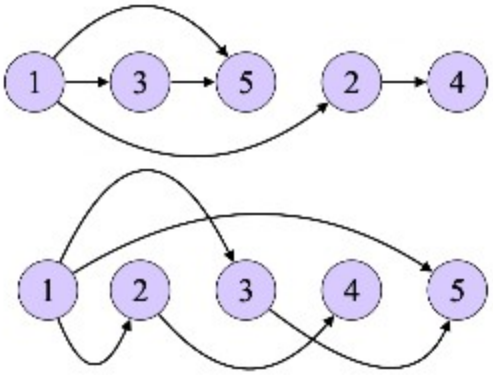
Notiamo che con questo algoritmo, il problema che avevamo con il vecchio algoritmo viene risolto:



Arco dell'albero	$dt[v] == 0$
Arco all'indietro:	$dt[u] > dt[v]$ **and** $ft[v] = 0$
Arco in avanti:	$dt[u] < dt[v]$ **and** $ft[v] \neq 0$
Arco attraversamento:	altrimenti

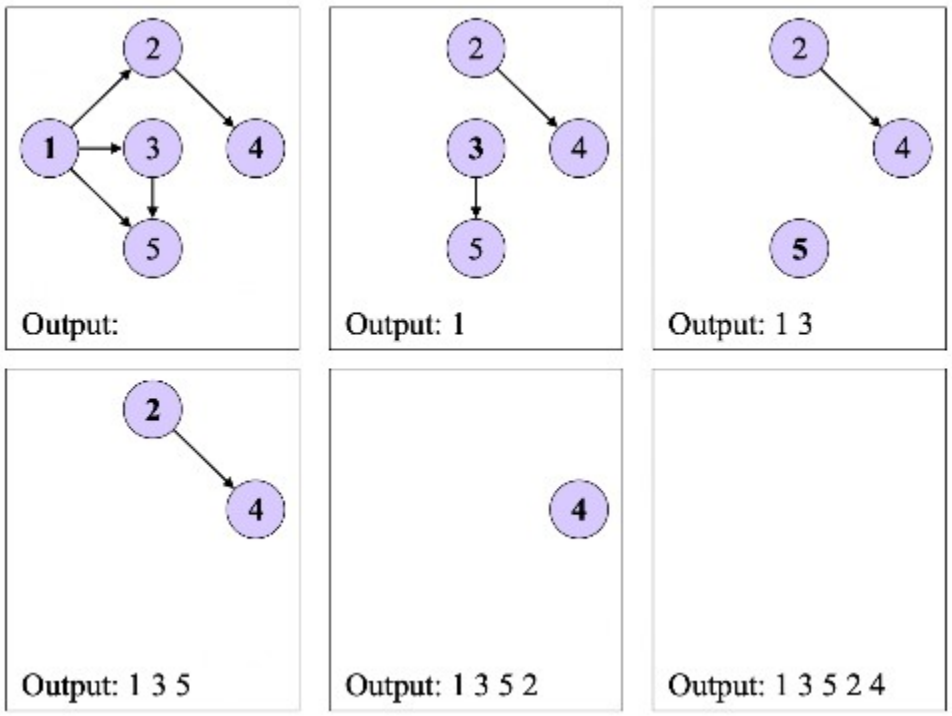
Vediamo ora un altro problema, l'**ordinamento topologico**, ovvero, dato un grafo DAG G, un ordinamento topologico di G è un ordinamento lineare dei suoi nodi tale che se $(u, v) \in E$, allora u appare prima di v nell'ordinamento.





Notiamo subito che esistono più algoritmi topologici per lo stesso grafo DAG, e che se un grafo contiene un ciclo, non esiste un ordinamento topologico.

Proviamo quindi a scrivere un algoritmo che ci ritorni un ordinamento topologico per un grafo DAG.
Un'idea potrebbe essere quella di trovare un **nodo senza archi entranti** (quindi che possa essere primo nell'ordinamento topologico). Aggiungere questo nodo alla lista rimuovendo tutti i suoi archi dal grafo e ripetere questa procedura fino a che tutti i nodi sono stati rimossi.

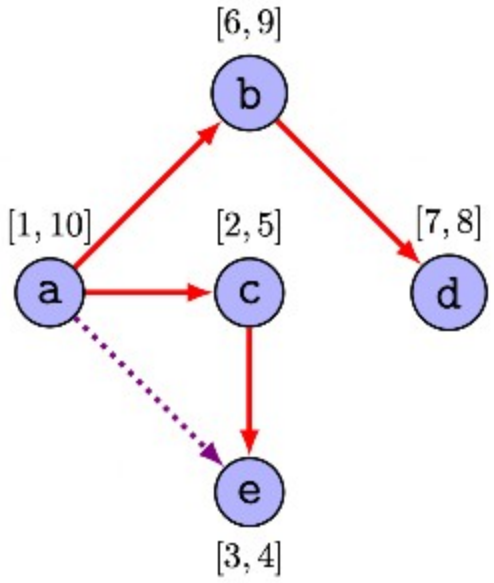


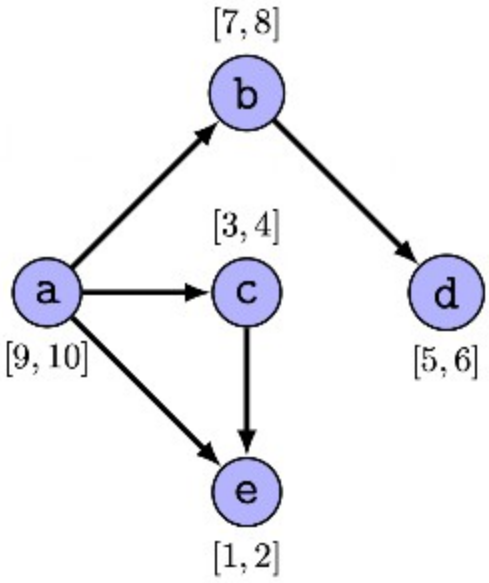
Possiamo però sfruttare la DFS per costruire l'ordinamento topologico, dove l'operazione di visita consiste nell'inserire il nodo in testa alla lista a "tempo di fine" (ovvero post-ordine). Questo algoritmo funziona perché quando un nodo è "finito", allora tutti i suoi discendenti saranno già nella lista, quindi aggiungendo il nodo verrà rispettato l'ordine.

```
STACK topSort(GRAPH G)
STACK S = Stack()
boolean[] visited = boolean[G.size()]
foreach u ∈ G.V() do visited[u] = false
foreach u ∈ G.V() do
    if not visited[u] then
        ts-dfs(G, u, visited, S)
return S
```

```
ts-dfs(GRAPH G, NODE u, boolean[] visited, STACK S)
visited[u] = true
foreach v ∈ G.adj(u) do
    if not visited[v] then
        ts-dfs(G, v, visited, S)
S.push(u)
```

Ecco un esempio di applicazione:





`Stack = { a, b, d, c, e }`

`Stack = { a, b, d, c, e }`

Questa procedura ha costo $O(n+m)$.

Lemma 3. L'algoritmo basato su DFS produce correttamente l'ordinamento topologico di un DAG G.

Dimostrazione. Dobbiamo provare che, dopo una DFS, se (u, v) appartiene ad E, allora $ft[v] < ft[u]$, e quindi u compare prima di v nella lista. Possono darsi tre casi, non essendoci archi all'indietro:

- (u, v) è un arco dell'albero, quindi v è un discendente di u nell'albero DFS e quindi (per il teorema delle parentesi) $[dt[v], ft[v]]$ è contenuta in $[dt[u], ft[u]]$, e quindi $ft[v] < ft[u]$;
- (u, v) è un arco in avanti, quindi di nuovo v è un discendente di u nell'albero DFS e $ft[v] < ft[u]$;
- (u, v) è un arco di attraversamento, significa che v è stato già completamento visitato al momento che u viene scoperto, e quindi $ft[v] < ft[u]$

L'ordinamento topologico è molto utile per quanto riguarda:

- Ordine di valutazione delle celle in uno spreadsheet
- Ordine di valutazione di un Makefile
- Risoluzione delle dipendenze nei linker
- Risoluzione delle dipendenze nei gestori di pacchetti software

Facciamo un ripasso delle componenti (fortemente) connesse:

Definizioni

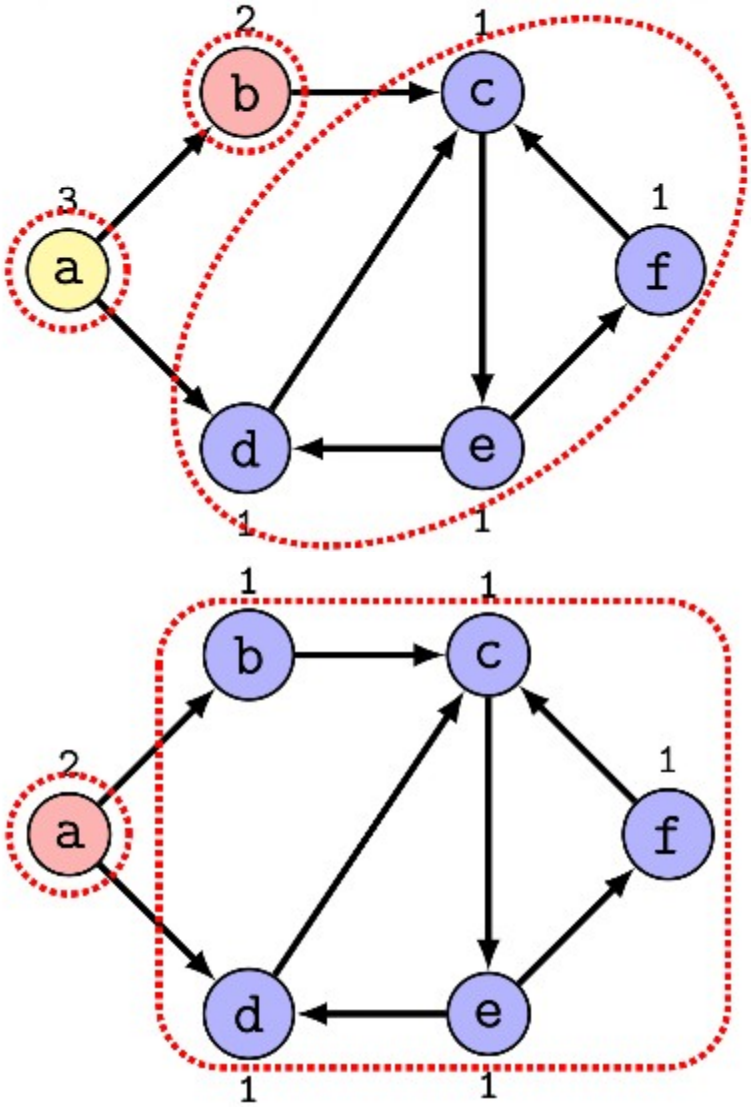
- Un grafo orientato $G = (V, E)$ è fortemente connesso $\Leftrightarrow$ ogni suo nodo è raggiungibile da ogni altro suo nodo
- Un grafo $G' = (V', E')$ è una componente fortemente connessa di $G \Leftrightarrow G'$ è un sottografo connesso e massimale di G

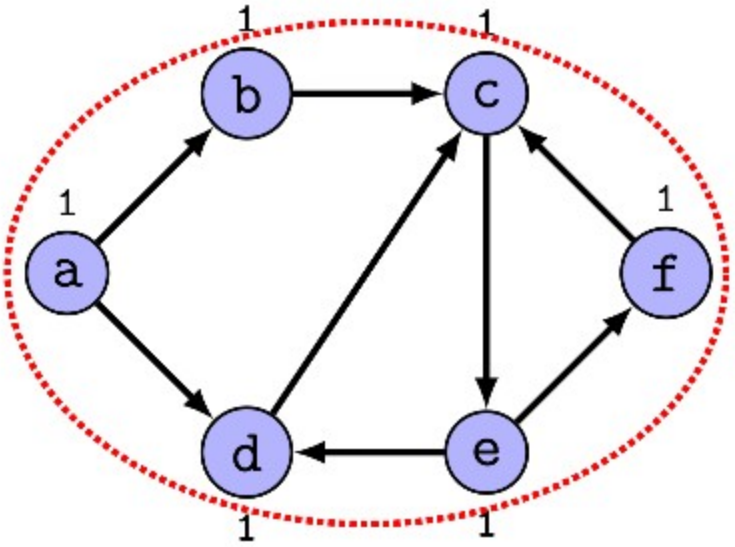
Repetita iuvant

- G' è un sottografo di G $(G' \subseteq G) \Leftrightarrow V' \subseteq V$ e $E' \subseteq E$
- G' è massimale $\Leftrightarrow \nexists$ un altro sottografo G'' di G tale che:
  - G'' è connesso
  - G'' è più grande di G' (i.e. $G' \subseteq G'' \subseteq G$)

Notiamo che in un grafo DAG, le componenti connesse variano in base al nodo da cui iniziamo la visita se usiamo l'algoritmo $cc()$, ecco un esempio:

- Si applica l'algoritmo cc() al grafo
- Purtroppo, il risultato dipende dal nodo di partenza





Notiamo che nessuno ci garantisce che queste componenti siano **fortemente connesse**.

Quindi venne inventato l'algoritmo di *Kosaraju*:

1. Effettuiamo un ordinamento topologico del grafo G
2. Calcola il grafo trasposto G_t di G
3. Esegue una visita DFS sul grafo G_t, utilizzando cc, esaminando i nodi nell'ordine inverso di tempo di fine della prima visita

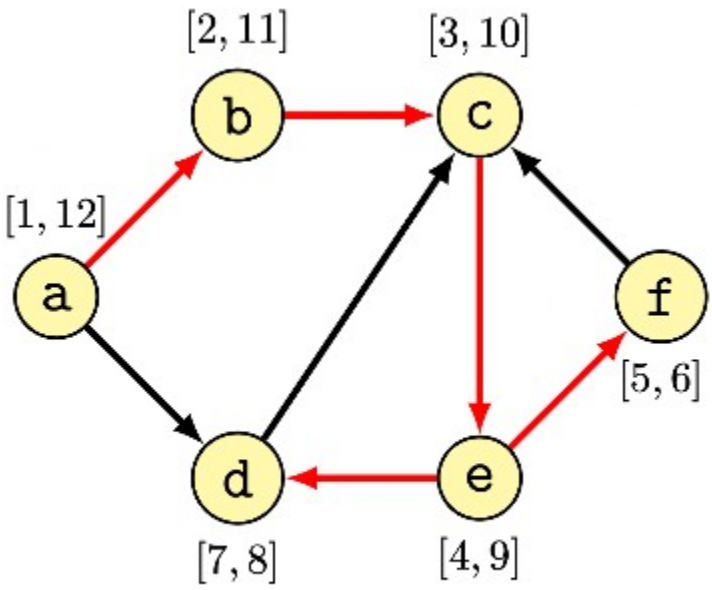
Le componenti connesse (e i relativi alberi DF) rappresentano le componenti fortemente connesse di G

```
int[] scc(GRAPH G)
STACK S = topSort(G)        % First visit
G^T = transpose(G)          % Graph transposal
return cc(G^T, S)           % Second visit
```

Per il primo punto, possiamo applicare l'ordinamento topologico in quanto:

- se un arco (u, v) non appartiene ad un ciclo, allora u viene listato prima di v nella sequenza ordinata
- gli archi di un ciclo vengono listati in qualche ordine, ininfluente

Possiamo quindi utilizzare $topsort()$ per ottenere i nodi in ordine decrescente di tempo di fine. Ecco l'applicazione al nostro esempio:



Stack = { a, b, c, e, d, f }

Ora dobbiamo per il secondo punto, calcolare il **grafo trasposto**.
Dato un grafo orientato $G=(V,E)$, il grafo trasposto $G_t=(V,E_t)$ ha gli stessi nodi e gli archi orientati in senso opposto:

$$E_t=\{(u,v) \mid (v,u) \in E\}$$

In sintesi “cambiamo” il verso di ogni arco del grafo.

Lemma 4. Un grafo G e il suo trasposto G^T hanno le stesse SCC.

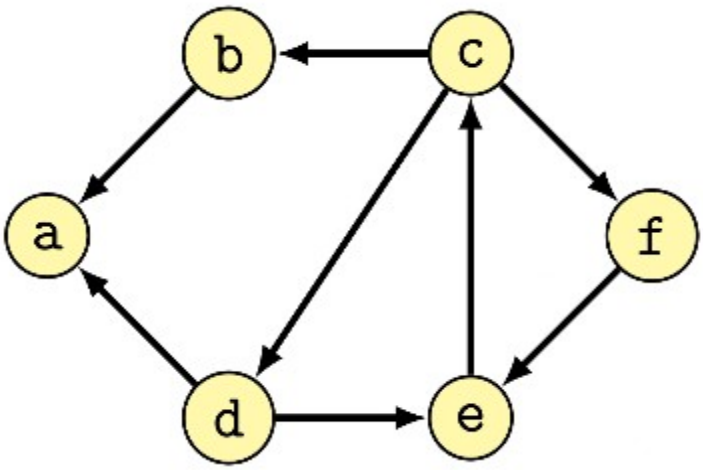
Dimostrazione. Se u, v appartengono alla stessa SCC, è possibile andare da u a v lungo il percorso p_{uv} e da v ad u lungo il percorso p_{vu}. Invertendo le direzioni degli archi, sarà possibile andare da v ad u lungo il cammino p^T_{uv} (l'inverso di p_{vu}), e da u a v lungo il cammino p^T_{vu}. □

```
GRAPH transpose(GRAPH G)
GRAPH G^T = Graph()
foreach u ∈ G.V() do
    G^T.insertNode(u)
foreach u ∈ G.V() do
    foreach v ∈ G.adj(u) do
        G^T.insertEdge(v, u)
return G^T
```

Costo computazionale: $O(m+n)$

- $O(n)$ nodi aggiunti
- $O(m)$ archi aggiunti
- Ogni operazione costa $O(1)$

Ecco l’applicazione al nostro esempio:

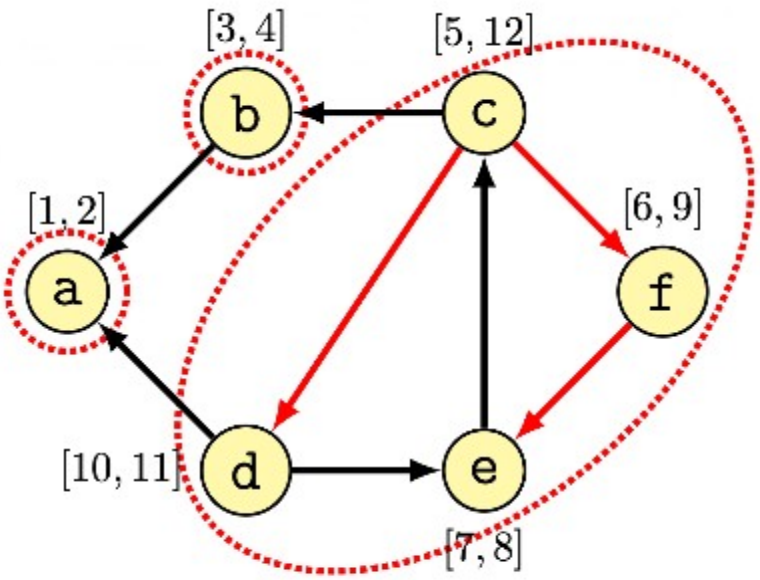


Ora facciamo il punto 3, trovando le componenti connesse però non in ordine casuale, ma nell'ordine dettato dall'ordinamento topologico del primo punto:

```
cc(GRAPH G, STACK S)
int[] id = new int[G.size()]
foreach u ∈ G.V() do
    id[u] = 0
int counter = 0
while not S.isEmpty() do
    u = S.pop()
    if id[u] == 0 then
        counter = counter + 1
        ccdfs(G, counter, u, id)
return id
```

```
ccdfs(GRAPH G, int counter, NODE u, int[] id)
id[u] = counter
foreach v ∈ G.adj(u) do
    if id[v] == 0 then
        ccdfs(G, counter, v, id)
```

Ecco l'applicazione al nostro esempio:




```
Stack = { a, b, c, e, d, f }
```

Notiamo che ogni punto richiede complessità $O(m+n)$, quindi l'algoritmo completo avrà complessità $3 \cdot O(m+n) = O(m+n)$.

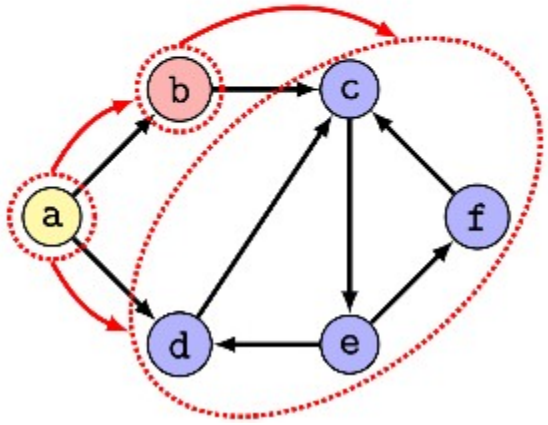
Notiamo che non importa da quale nodo partiamo nella prima visita, in ogni caso troveremo correttamente le componenti fortemente connesse.

Per dimostrare la correttezza definiamo il **grafo delle componenti**, che ha come vertici le componenti fortemente connesse e come archi gli archi che vanno da un nodo di una componente connessa a qualsiasi altro nodo di un'altra componente connessa:

Grafo delle componenti

$C(G) = (V_c, E_c)$

- $V_c = \{C_1, C_2, \ldots, C_k\}$, dove C_i è la i-esima SCC of G
- $E_c = \{(C_i, C_j) | \exists (u_i, u_j) \in E : u_i \in C_i \wedge u_j \in C_j\}$



Notiamo due proprietà:

- Il grafo delle componenti di G_t è uguale al grafo trasposto delle componenti di G
- Il grafo delle componenti è aciclico (altrimenti le componenti facenti parte del ciclo si unirebbero in una sola)

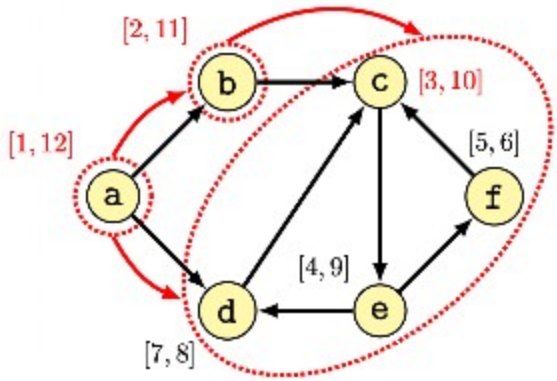
Abbiamo poi che

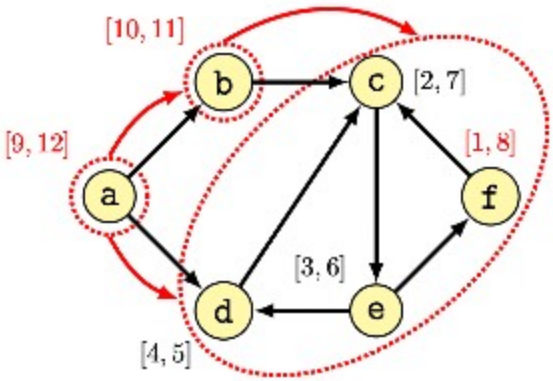
Discovery time e finish time del grafo delle componenti

$$dt(C) = \min\{dt(u) | u \in C\}$$
$$ft(C) = \max\{ft(u) | u \in C\}$$

Il che significa che discovery/finish time di una componente connessa corrisponde a discovery/finish time del primo nodo visitato di quella componente connessa.

Possiamo ora formulare il seguente teorema:
Siano C e C' due distinte SCCs nel grafo orientato $G=(V,E)$. Se esiste un arco $(C,C') \in E_c$, allora $ft(C) > ft(C')$





Lemma 6. Siano C e C' due SCC distinte nel grafo orientato $G = (V, E)$. Supponiamo che esista un arco $(u, v) \in E$, dove $u \in C$ e $v \in C'$. Allora $ft(C) > ft(C')$.

Dimostrazione. Ci possono essere due casi: viene scoperto prima un nodo di C oppure di C'.

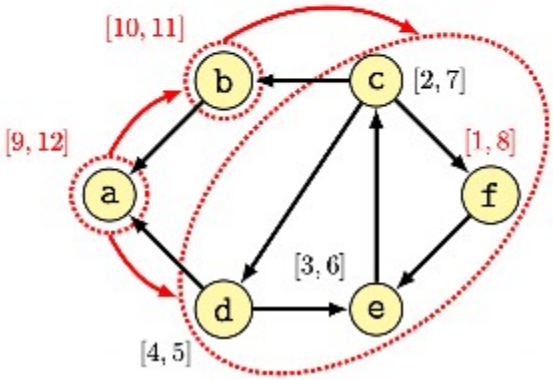
- $dt(C) < dt(C')$: al momento $dt(C)$ viene scoperto un nodo x; tutti i nodi di C e C' non erano ancora stati scoperti fino a quel momento. Poiché esiste un cammino fra x e tutti gli altri nodi di C e C', ne segue che sono tutti discendenti di x . Quindi ne segue che $ft(x)$ è maggiore di tutti gli ft sia di C che di C', quindi $ft(C) > ft(C')$.
- $dt(C') < dt(C)$: sia y il primo nodo scoperto in C'; tutti i nodi in C' sono raggiungibili da y (definizione di SCC) e tutti i nodi in C' non sono ancora stati scoperti; tutti i nodi in C' sono discendenti di y e quindi $ft(C') = ft(y)$. Inoltre, poichè esiste un cammino fra i nodi di C e C', non può esistere un cammino fra i nodi di C' e C. Quindi i nodi di C non sono raggiungibili da y, e verranno scoperti (e terminati) dopo. Quindi $ft(C) > ft(C')$.

Possiamo facilmente dimostrare che nel grafo trasposto vale il contrario:

Corollario

Siano C_x e C_y due SCC distinte nel grafo orientato $G = (V, E)$. Se esiste un arco $(x, y) \in E_t$ tale che $x \in C_x$ e $y \in C_y$, allora $ft(C_x) < ft(C_y)$.

$$(x, y) \in E_t \Rightarrow$$
$$(y, x) \in E \Rightarrow$$
$$(C_y, C_x) \in E_c \Rightarrow$$
$$ft(C_y) > ft(C_x) \Rightarrow$$
$$ft(C_x) < ft(C_y)$$



Questo corollario è fondamentale per capire la correttezza e il funzionamento dell'algoritmo di ricerca delle componenti fortemente connesse in un grafo non orientato.

Quindi, se la componente C_x e la componente C_y sono connesse da un arco $(x, y) \in E_t$, allora:

- Dal corollario abbiamo che $ft(C_x) < ft(C_y)$
- Dall'algoritmo, la visita di C_y inizierà prima della visita di C_x

Però non esistono cammini tra C_y e C_x in G_t (altrimenti il grafo delle componenti sarebbe ciclico), quindi la visita di C_y non raggiungerà C_x.
In conclusione, $cc()$ assegnerà correttamente gli identificatori delle componenti ai nodi.

Un'ultima osservazione è che quasi sempre un algoritmo che funziona per un grafo non orientato non può essere applicato paro paro ad un grafo orientato (come abbiamo visto), ma ci vuole qualche accorgimento in più.

Hashing

Introduzione

Prima di parlare di **hash**, ripassiamo il concetto di dizionario.
Un **dizionario** è una struttura dati utilizzata per memorizzare insiemi dinamici di **coppie** $<chiave, valore>$. Le coppie sono indicizzate in base alla **chiave** e il **valore** è un dato satellite.
Quindi associamo un valore ad una chiave univoca.
Delle applicazioni di un dizionario possono essere la tabella dei simboli di un compilatore e i dizionari di python.
Possiamo vedere gli array come dei dizionari con i numeri interi come chiavi.
Abbiamo le seguenti operazioni:

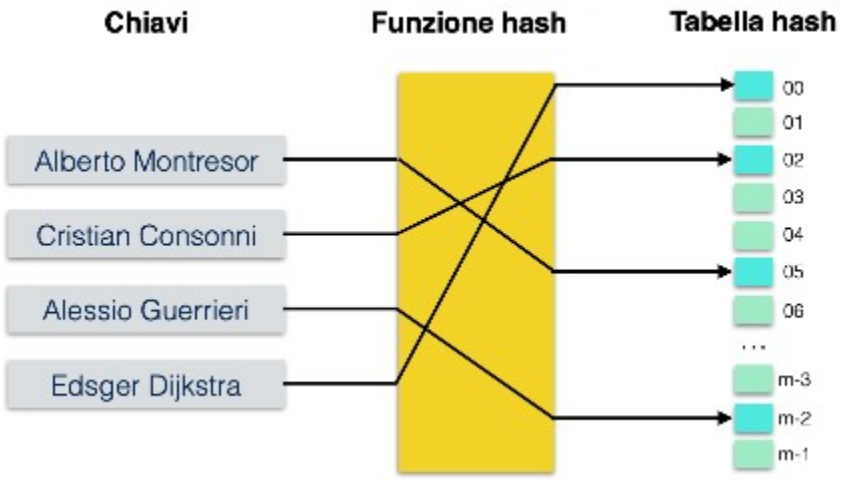
- $lookup(key) \rightarrow value$
- $insert(key, value)$
- $remove(key)$

Come vediamo nella seguente tabella, esistono varie implementazioni di un dizionario, ognuna però ha dei costi diversi per ogni operazione:

	Array non ordinato	**Array ordinato**	**Lista**	**Alberi RB**	**Implemen. ideale**
insert()	$O(1), O(n)$	$O(n)$	$O(1), O(n)$	$O(\log n)$	$O(1)$
lookup()	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(1)$
remove()	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$
foreach	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

L'implementazione ideale è fatta tramite **tabelle hash**.
L'insieme delle possibili chiavi è rappresentato dall'**insieme universo** U di dimensione $|U| = u$.
Le chiavi vengono memorizzate in un vettore $T[0 \ \ldots \ m-1]$ di dimensione m, che chiamiamo **tabella hash**.

Una **funzione hash** è definita come $h : U \rightarrow \{0 \ldots m-1\}$, di conseguenza la coppia chiave-valore $<k, v>$ viene memorizzata nella tabella hash nella posizione $h(k)$.



Con la precedente definizione, e con il fatto che praticamente sempre abbiamo $m < u$, due o più chiavi nel dizionario possono avere lo stesso valore hash, in questa situazione diciamo che è avvenuta una **collisione**.

La situazione ideale sarebbe avere funzioni hash senza collisioni, per farlo dovremmo prendere la funzione hash identità, ovvero che $h(k) = k$. Per ottenerla dobbiamo prendere $m = u$.
Il problema è che se u è molto grande, l'approccio non è praticabile, e anche nel caso in cui u non sia troppo grande, capita spesso di "registrare" un numero di chiavi molto inferiore di u, con un conseguente spreco di memoria.

	Array non ordinato	**Array ordinato**	**Lista**	**Alberi RB**	~~**Impl. ideale**~~ **Hash table**
insert()	$O(1), O(n)$	$O(n)$	$O(1), O(n)$	$O(\log n)$	$O(1)$
lookup()	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(1)$
remove()	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$
foreach	$O(n)$	$O(n)$	$O(n)$	$O(n)$	~~$O(n)$~~$O(m)$

Notiamo che il *foreach* si riduce a $O(m)$ in quanto non possiamo salvare più di m valori.

Funzioni hash

Riprendendo il concetto di prima, una funzione hash h si dice **perfetta** se è **iniettiva**, ovvero se non dà origine a collisioni:

$$\forall k_1, k_2 \in U \ : k_1 \neq k_2 \Rightarrow h(k_1) \neq h(k_2)$$

Ad esempio per mettere in un dizionario gli studenti di ASD dal 2005 al 2016 possiamo prendere tutti gli studenti dal 2005 al 2016, quindi con numero di matricola compreso

nell'intervallo $[100.090,\ 183.864]$, e possiamo definire h come $h(k) = k - 100.090$, ottenendo $m = 83.774$.

Notiamo che lo spazio delle chiavi è molto grande (tutti gli studenti di quel periodo), e rispetto alle chiavi che andremo effettivamente ad utilizzare (solo gli studenti di ASD), sprechiamo tantissimo spazio.
Quindi è spesso **impraticabile** ottenere **una funzione hash perfetta**.

Quindi se non possiamo evitare le collisioni, proviamo a cercare delle funzioni hash che le minimizzino il più possibile, questo significa trovare funzioni che **distribuiscano uniformemente** le chiavi negli indici $[0\ \dots\ m-1]$ della tabella hash.
Se per caso so qualche informazione sulla distribuzione dei valori, posso usarla per distribuire bene la funzione hash.
Sia $P(k)$ la probabilità che una chiave k sia inserita nella tabella hash.
Sia $Q(i)$ la probabilità che una chiave finisca nella cella i, ovvero:

$$Q(i) = \sum_{k \in U\ :\ h(k)=i} P(k)$$

Una funzione hash h gode di **uniformità semplice** se:

$$\forall\, i \in [0\ \dots\ m-1]\ :\ Q(i) = \frac{1}{m}$$

Ovviamente per poter ottenere una funzione hash con uniformità semplice, devo aver perfettamente nota la distribuzione delle probabilità P

Esempio

Se $\mathcal{U}$ è dato dai numeri reali in $[0, 1[$ e ogni chiave ha la stessa probabilità di essere scelta, allora $H(k) = \lfloor km \rfloor$ soddisfa la proprietà di uniformità semplice

Nella realtà però la distribuzione esatta non può essere (completamente) nota, si usano quindi **tecniche euristiche**.

Prima però, vediamo come possiamo sempre tradurre le chiavi in valori numerici (anche interpretando la loro rappresentazione in memoria come un numero):

Esempio: trasformazione stringhe

- $ord(c)$: valore ordinale binario del carattere c in qualche codifica
- $bin(k)$: rappresentazione binaria della chiave k, concatenando i valori binari dei caratteri che lo compongono
- $int(b)$: valore numerico associato al numero binario b
- $int(k) = int(bin(k))$

Nei prossimi esempi, utilizziamo codice ASCII a 8 bit

$$
\begin{aligned}
bin(\text{“DOG”}) &= ord(\text{“D”}) & ord(\text{“O”}) & & ord(\text{“G”}) \\
&= 01000100 & 01001111 & & 01000111 \\
int(\text{“DOG”}) &= 68 \cdot 256^2 + & 79 \cdot 256 + & & 71 \\
&= 4,476,743
\end{aligned}
$$

Estrazione

Ora che abbiamo visto come “tradurre” le chiavi in valori numerici, vediamo come trasformare questo numero in un valore compreso tra $[0, \ldots, m-1]$.

Prendiamo $m = 2^p$ (con p a scelta).

Definiamo la funzione hash $H(k) = \int(b)$, dove b è un sottoinsieme di p bit presi da $bin(k)$.

I problemi qui risiedono nel fatto che selezionare bit presi dal prefisso/suffisso della chiave può generare collisioni con alta probabilità (se le chiavi sono dei nomi, certe lettere sono più probabili). Le collisioni tuttavia possono esserci anche quando non selezioniamo bit presi da prefisso e suffisso.

Esempio 1

$m = 2^p = 2^{16} = 65536$; 16 bit meno significativi di $bin(k)$

$$bin(\text{“Alberto”}) = 01000001\ 01101100\ 01100010\ 01100101\ 01110010\ 01110100\ 01101111$$
$$bin(\text{“Roberto”}) = 01010010\ 01101111\ 01100010\ 01100101\ 01110010\ 01110100\ 01101111$$
$$H(\text{“Alberto”}) = int(0111010001101111) = 29.807$$
$$H(\text{“Roberto”}) = int(0111010001101111) = 29.807$$

Esempio 2

$m = 2^p = 2^{16} = 65536$; 16 bit presi all'interno di $bin(k)$

$$bin(\text{“Alberto”}) = 01000001\ 01101100\ 01100010\ 01100101\ 01110010\ 01110100\ 01101111$$
$$bin(\text{“Alessio”}) = 01000001\ 01101100\ 01100101\ 01110011\ 01110011\ 01101001\ 01101111$$
$$H(\text{“Alberto”}) = int(0001011011000110) = 5.830$$
$$H(\text{“Alessio”}) = int(0001011011000110) = 5.830$$

XOR

Prendiamo $m = 2^p$ (con p a scelta).

Definiamo la funzione hash $H(k) = \int (b)$, dove b è dato dalla somma modulo 2, effettuata bit a bit, di sottoinsiemi di p bit di $bin(k)$.

Il problema è che permutazioni (ovvero anagrammi) della stessa stringa possono generare lo stesso valore hash

Esempio

$m = 2^{16} = 65536$; 5 gruppi di 16 bit ottenuti con 8 zeri di "padding"

$bin(\text{“montresor”}) =$	$bin(\text{“sontremor”}) =$
01101101 01101111 $\oplus$	01110011 01101111 $\oplus$
01101110 01110100 $\oplus$	01101110 01110100 $\oplus$
01110010 01100101 $\oplus$	01110010 01100101 $\oplus$
01110011 01101111 $\oplus$	01101101 01101111 $\oplus$
01110010 00000000	01110010 00000000
$H(\text{“montresor”}) =$	$H(\text{“sontremor”}) =$
$int(01110000\ 00010001) =$	$int(01110000\ 00010001) =$
28.689	28.689

Metodo della divisione

Prendiamo m dispari (meglio se numero primo, o almeno coprimo con U).

Definiamo la funzione hash $H(k) = \int (k) \ \ mod \ \ m$

Esempio

$m = 383$

$$H(\text{“Alberto”}) = 18.415.043.350.787.183 \bmod 383 = 221$$
$$H(\text{“Alessio”}) = 18.415.056.470.632.815 \bmod 383 = 77$$
$$H(\text{“Cristian”}) = 4.860.062.892.481.405.294 \bmod 383 = 130$$

Non dobbiamo prendere m che sia uguale ad una potenza di 2 (se prendiamo $m = 2^p$ solo i p bit meno significativi vengono considerati) e neanche $m = 2^p - 1$.

Data una sequenza di caratteri presi da un alfabeto di dimensione 2^p, se si utilizza un valore di m pari a $2^p - 1$, due sequenze permutate danno origine allo stesso valore hash.

Dimostrazione

$$\begin{aligned} & [2^p a + b] \bmod 2^p - 1 \\ = \; & [(2^p - 1)a + a + b] \bmod 2^p - 1 \\ = \; & [a + b] \bmod 2^p - 1 \\ = \; & [(2^p - 1)b + b + a] \bmod 2^p - 1 \\ = \; & [2^p b + a] \bmod 2^p - 1 \end{aligned}$$

Quindi come *m* vanno bene numeri primi distanti da potenze di 2 e di 10.

Metodo della moltiplicazione (Knuth)

Prendiamo *m* qualsiasi (meglio se potenza di 2). Prendiamo una costante C reale con $0 < C < 1$.

Definiamo $i = \int (k)$

Definiamo la funzione hash $H(k) = \lfloor m \cdot (C \cdot i - \lfloor C \cdot i \rfloor) \rfloor$

Esempio

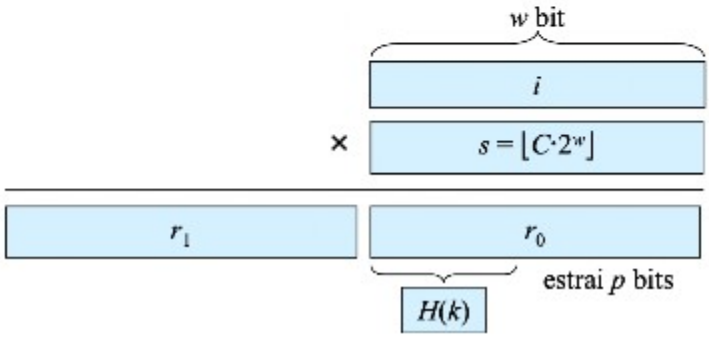
$m = 2^{16}$

$C = \frac{\sqrt{5}-1}{2}$

$$H(\text{“}\texttt{Alberto}\text{”}) = 65.536 \cdot 0.78732161432 = 51.598$$
$$H(\text{“}\texttt{Alessio}\text{”}) = 65.536 \cdot 0.51516739168 = 33.762$$
$$H(\text{“}\texttt{Cristian}\text{”}) = 65.536 \cdot 0.72143641000 = 47.280$$

Ecco l’implementazione di questo metodo:

- Si scelga un valore $m = 2^p$
- Sia w la dimensione in bit della parola di memoria: $i, m \leq 2^w$
- Sia $s = \lfloor C \cdot 2^w \rfloor$
- $i \cdot s$ può essere scritto come $r_1 \cdot 2^w + r_0$
  - r_1 contiene la parte intera di iC
  - r_0 contiene la parte frazionaria di iC
- Si restituiscano i p-bit più significativi di r_0



Il metodo della moltiplicazione però non fornisce hashing uniforme.

Gestione collisioni

Visto che abbiamo detto che le collisioni sono praticamente inevitabili, dobbiamo trovare un modo per gestirle efficacemente.

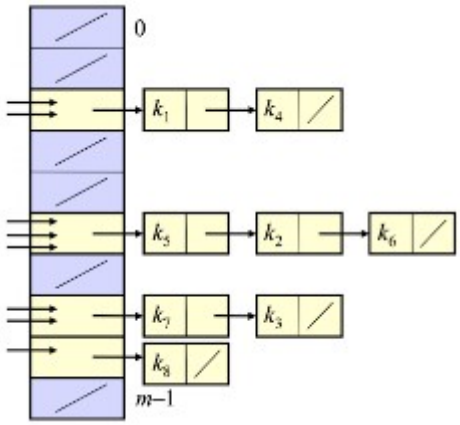
Dobbiamo quindi in qualche modo trovare delle posizioni alternative per le chiavi quando avviene una collisione, questo però aumenta la complessità della ricerca perché se una chiave non si trova nella posizione attesa, dobbiamo cercarla anche nelle posizioni alternative.
Questa ricerca **dovrebbe** costare $O(1)$ nel caso medio (quando la chiave è nella posizione in cui dovrebbe essere), ma può arrivare fino ad $O(n)$ nel caso pessimo.

Abbiamo due possibili tecniche per gestire le collisioni:

- **Lista di trabocco** (o memorizzazione esterna)
- **Indirizzamento aperto** (o memorizzazione interna)

Liste/vettori di trabocco

L'idea è quella che tutte le chiavi con lo stesso hash h vengono memorizzate in una **lista monodirezionale** o in un **vettore dinamico**.
Di conseguenza nella tabella hash dobbiamo memorizzare un puntatore alla testa della lista (o al vettore) nello slot $H(k)$-$esimo$



Quindi le operazioni della tabella hash possono essere viste così:

- $insert()$
  Inseriamo l'elemento in testa
- $lookup()$ e $remove()$
  Scansioniamo la lista per cercare la chiave

Vediamo quale è la complessità di questa implementazione, introducendo alcune "variabili":

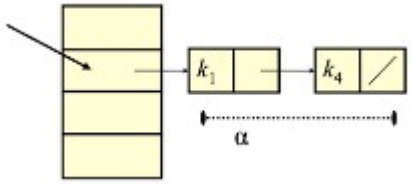
- n è il numero di chiavi effettivamente memorizzate nella tabella hash
- m è la capacità della tabella hash
- $\alpha = \frac{n}{m}$ è il **fattore di carico**
- $I(\alpha)$ è il numero medio di accessi alla tabella per la ricerca di una chiave non presente nella tabella (**ricerca con insuccesso**)
- $S(\alpha)$ è il numero medio di accessi alla tabella per la ricerca di una chiave presente nella tabella (**ricerca con successo**)

Nel **caso pessimo** tutte le chiavi sono collocate in un'unica lista (hanno tutte lo stesso hash), di conseguenza abbiamo:

- $insert()$: $\Theta(1)$
- $lookup()$ e $remove()$: $\Theta(n)$

Nel **caso medio** la complessità dipende da come le chiavi sono distribuite, assumiamo quindi un hashing semplice uniforme e che il costo del calcolo della funzione hash sia $\Theta(1)$.

Le liste/vettori dovrebbero essere in media lunghi α (ovvero $\frac{n}{m}$) visto che la distribuzione è uniforme.



Come prima cosa abbiamo il costo dell'hashing (per capire in quale degli m slot dovrà essere inserita la chiave) che sarà costante quindi $\Theta(1)$.
Nel caso di una **ricerca con insuccesso** dobbiamo scorrere tutta la lista corrispondente, quindi il costo atteso sarà $\Theta(1)+\alpha$
Nel caso di una **ricerca con successo**, in media troviamo la nostra chiave a metà della lista corrispondente, quindi il costo atteso sarà $\Theta(1)+\frac{\alpha}{2}$

Come abbiamo visto il fattore di carico influenza il costo computazionale delle operazioni sulle tabelle hash, infatti se la tabella è ben distribuita, ovvero $n=O(m)$ abbiamo $\alpha=O(1)$ e in questa situazione (ideale) ogni operazione ha costo $O(1)$.

Indirizzamento aperto

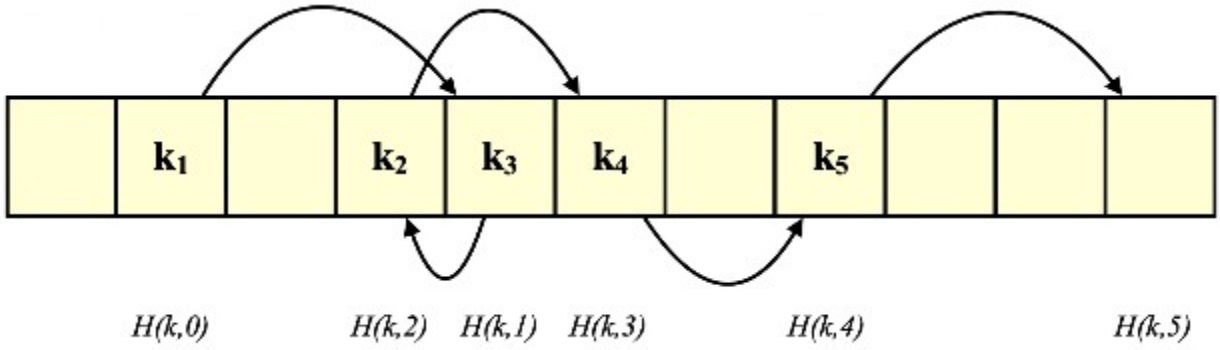
Il "problema" delle liste di trabocco è che la struttura dati è complessa (abbiamo liste, puntatori etc).

Esiste quindi una implementazione alternativa, ovvero l'**indirizzamento aperto**.
L'idea è quella di memorizzare tutte le chiavi direttamente nella tabella stessa, quindi ogni slot della tabella hash contiene o una chiave oppure nil.
Nel caso di una collisione nell'inserimento (ovvero quando lo slot "prescelto" è utilizzato) si cerca uno slot alternativo.
Nella ricerca si controlla prima lo slot prescelto, e poi si controlla negli slot "alternativi" fino a quando non si trova la chiave oppure nil.

Dobbiamo introdurre un altro concetto per capire quanti slot abbiamo controllato (altrimenti rischiamo di entrare in un loop infinito).
Un'**ispezione** è l'esame di uno slot durante la ricerca, introducendo l'ispezione, la funzione hash è estesa nel seguente modo:

$$H : U \times [0 \ \ldots \ m-1] \rightarrow [0 \ \ldots \ m-1]$$

Ovviamente una **sequenza di ispezione** (ovvero $[H(k,0), H(k,1) \ \ldots \ H(k,m-1)]$ deve essere una permutazione degli indici $[0 \ \ldots \ m-1]$, corrispondente all'ordine in cui vengono esaminati gli slot. Questo perché non vogliamo esaminare uno slot più di una volta e dobbiamo sapere quando fermarci per non continuare all'infinito.



In questo caso il fattore di carico α non può essere maggiore di 1 (n non può essere maggiore di m) quindi abbiamo $0 \leq \alpha \leq 1$. La tabella infatti in questa implementazione può andare in overflow (ovvero riempirsi completamente).

Generalizziamo il concetto di hashing uniforme semplice, introducendo l'hashing uniforme.
La situazione ideale prende il nome di **hashing uniforme**, in cui ogni chiave ha la stessa probabilità di avere come sequenza di ispezione una qualsiasi delle $m!$ permutazioni di $\langle 0 \ \ldots \ m-1 \rangle$.

Nella realtà è difficile implementare il vero hashing uniforme, ci si accontenta quindi di avere almeno una permutazione.
Vediamo alcune delle tecniche più diffuse delle funzioni hash con ispezione:

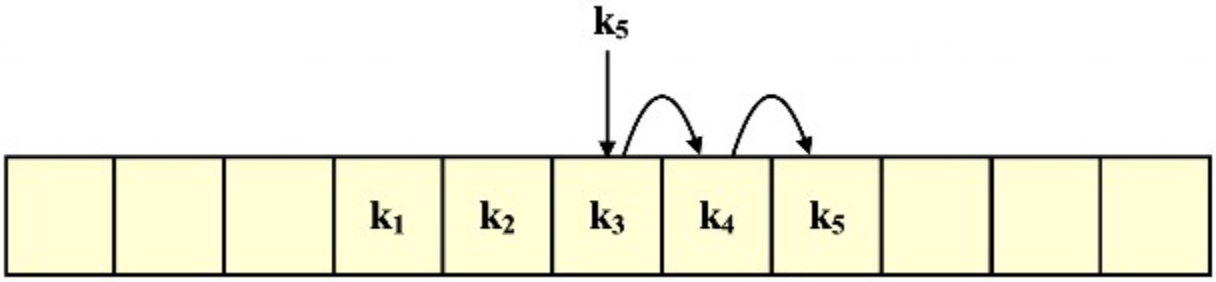
- **Ispezione lineare**
- **Ispezione quadratica**
- **Doppio hashing**

Nell'**ispezione lineare** ci spostiamo costantemente di tot posizioni in base al numero di ispezione (moltiplicato per un fattore costante h). La funzione di hash è la seguente:

$$H(k,i) = \left(H_1(k) + h \cdot i\right) \ mod \ m$$

La sequenza $H_1(k),\ H_1(k)+h,\ H_1(k)+2h,\ \ldots,\ H_1(k)+(m-1)h$ (tutto modulo m ovviamente) è determinata dal primo elemento, ovvero $H_1(k)$ (visto che poi l'incremento è lo stesso per ogni sequenza).
Con questa implementazione, sono possibili al massimo m sequenze di ispezione distinte (che variano grazie all'elemento di partenza appunto).

Il problema di questa ispezione è l'**agglomerazione primaria**, ovvero che è molto probabile che si formino delle lunghe sotto-sequenze occupate, le quali tendono a diventare sempre più lunghe.
Questo perché se abbiamo uno slot vuoto, preceduto da i slot pieni (ovviamente gli slot "precedenti" dipendono dal fattore h), questo slot avrà probabilità di essere riempito $\frac{i+1}{m}$, invece altri slot con lo slot precedente libero avranno probabilità $\frac{1}{m}$.
Di conseguenza i tempi medi di inserimento e cancellazione crescono, dovendo scorrere più posizioni.



Un altro problema è che elementi che hanno come posizione “vera” un determinato slot, potrebbero trovarlo occupato da elementi che sono finiti lì a causa di una collisione.

Nell’**ispezione quadratica** ci spostiamo in base a una funzione quadratica sul numero di ispezione (moltiplicato per un fattore costante h). La funzione di hash è la seguente:

$$H(k,i)=\left(H_1(k)+h\cdot i^2\right)\ \ mod\ \ m$$

Quindi dopo il primo elemento $H_1(k)$ (quello “vero”), le ispezioni successive hanno un offset che dipende da una funzione quadratica al numero di ispezione i.
Questo però **non garantisce** che la sequenza risultante sia una **permutazione**. Anche in questo caso sono possibili al massimo m sequenze distinte.

Il problema di questa implementazione è l’agglomerazione secondaria (che è meno grave della agglomerazione primaria), ovvero che se due chiavi hanno la stessa ispezione iniziale, allora le loro sequenze sono identiche.

Nel **doppio hashing** ci spostiamo in base a una seconda funzione di hash (che sarà costante ma solo in riferimento ad una specifica chiave) moltiplicata per il numero di ispezione. La funzione di hash è la seguente:

$$H(k,i)=\left(H_1(k)+i\cdot H_2(k)\right)\ \ mod\ \ m$$

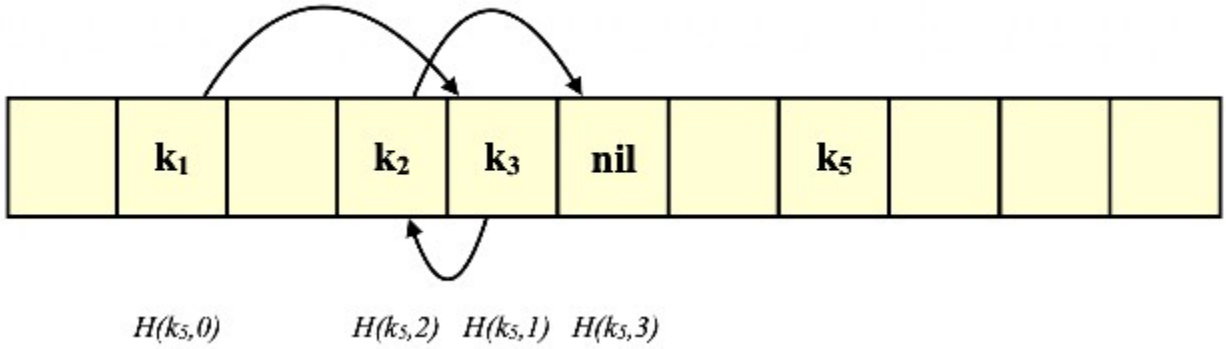
Abbiamo quindi due funzioni ausiliarie:

- $H_1(k)$ che fornisce la prima ispezione
- $H_2(k)$ che fornisce l’offset delle successive ispezioni

In questa implementazione possiamo avere al massimo m^2 sequenze di ispezione distinte possibili.
Per garantire una permutazione completa, $H_2(k)$ deve essere relativamente primo con m, quindi ad esempio possiamo:

- Scegliere $m=2^p$ e $H_2(k)$ deve restituire numeri dispari
- Scegliere m primo e $H_2(k)$ deve restituire numeri minori di m

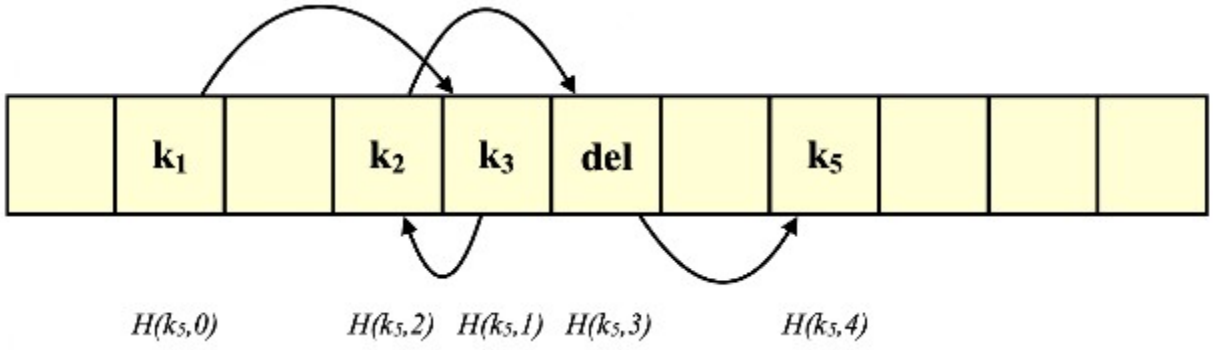
Nell’indirizzamento aperto abbiamo un problema nella cancellazione, questo perché quando cancelliamo una chiave, non possiamo semplicemente mettere nil, perché altrimenti un eventuale ricerca fatta successivamente si fermerebbe non appena troverebbe quel nil, anche nel caso in cui la chiave cercata è presente ma è stata inserita dopo il nodo che noi abbiamo poi cancellato.



Per risolvere questo problema utilizziamo un valore speciale *del* al posto di *nil* per marcare uno slot come vuoto dopo la cancellazione, così possiamo trattarli diversamente in base a quello che stiamo facendo:

- Nella **ricerca** i slot *del* sono trattati come slot “pieni”
- Nell’**inserimento** i slot *del* sono trattati come slot “vuoti”

Lo svantaggio è che il tempo di ricerca non dipende più da α, visto che i nodi rimossi continuano ad allungare la ricerca.



Vediamo ora un’implementazione dell’hashing doppio considerando tutti gli aspetti che abbiamo visto:

HASH

```
ITEM[] K                                   % Tabella delle chiavi
ITEM[] V                                   % Tabella dei valori
int m                                      % Dimensione della tabella

HASH Hash(int dim)
    HASH t = new HASH
    t.m = dim
    t.keys = new ITEM[0 ... dim − 1]
    t.values = new ITEM[0 ... dim − 1]
    for i = 0 to dim − 1 do
        t.keys[i] = nil
    return t

int scan(ITEM k, boolean insert)
    int delpos = m                         % Prima posizione deleted
    int i = 0                              % Numero di ispezione
    int j = H(k)                           % Posizione attuale
    while keys[j] ≠ k and keys[j] ≠ nil and i < m do
        if keys[j] == deleted and delpos == m then
            delpos = j
        j = (j + H'(k)) mod m
        i = i + 1
    if insert and keys[j] ≠ k and delpos < m then
        return delpos
    else
        return j
```

```
ITEM lookup(ITEM k)
    int i = scan(k, false)
    if keys[i] == k then
        return values[i]
    else
        return nil

insert(ITEM k, ITEM v)
    int i = scan(k, true)
    if keys[i] == nil or keys[i] == deleted or keys[i] == k then
        keys[i] = k
        values[i] = v
    else
        % Errore: tabella hash piena

remove(ITEM k)
    int i = scan(k, false)
    if keys[i] == k then
        keys[i] = deleted
        values[i] = nil
```

2 Domanda: Costo computazionale di una ricerca senza successo, in caso di indirizzamento aperto

Il numero atteso di ispezioni $I(\alpha)$ per una ricerca senza successo è pari a $\frac{1}{(1-\alpha)}$.
Abbiano un tabella di m celle, con n elementi già inseriti. Quindi $n/m = \alpha$.
In un ricerca senza successo, ogni ispezione, tranne l'ultima, trova una cella occupata; l'ultima trova una cella **nil**.

- Variabile casuale X: numero di ispezioni.
- Evento A_i: l'i-esima ispezione trova una cella occupata.

$$
\begin{aligned}
Pr(X \geq i) &= Pr(A_1 \wedge A_2 \wedge \ldots \wedge A_{i-1})) \\
&= Pr(A_1) \cdot Pr(A_2|A_1) \cdot Pr(A_3|A_1 \wedge A_2) \cdot \ldots \cdot Pr(A_{i-1}|A_1 \wedge \ldots \wedge A_{i-2}) \\
&= \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \cdot \ldots \cdot \frac{n-(i-2)}{m-(i-2)} \\
&\leq \left(\frac{n}{m}\right)^{i-1} \\
&\leq \alpha^{i-1}
\end{aligned}
$$

Questo perché alla prima ispezione, ci sono n elementi in m celle; alla seconda ispezione, non si può più passare per la prima cella, quindi ci sono $n-1$ elementi in $m-1$ celle; e così via.

$$\begin{aligned} E[X] &= \sum_{i=1}^{\infty} Pr(X \geq i) \\ &\leq \sum_{i=1}^{\infty} \alpha^{i-1} \\ &= \sum_{i=0}^{\infty} \alpha^{i} \\ &= \frac{1}{1-\alpha} \end{aligned}$$

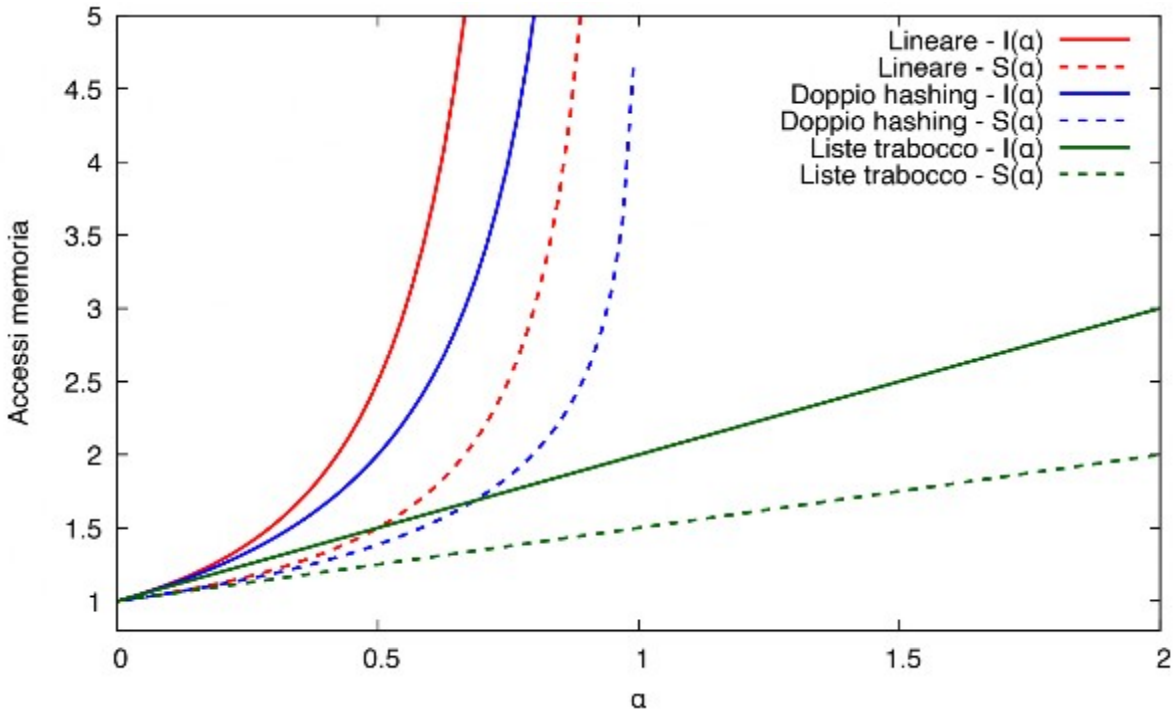
3 Domanda: Costo computazionale di una ricerca con successo, in caso di indirizzamento aperto

Una ricerca con successo per la chiave k segue la stessa sequenza di quando k è stato inserito.
Sia β il fattore di carico all'atto dell'inserzione ed α quello all'atto della ricerca con successo. Facendo la media sui possibili valori di β, che sono compresi tra 0 ed α, si ottiene:

$$S(\alpha) = \frac{1}{\alpha}\int_0^{\alpha} \frac{1}{1-\beta}\mathrm{d}\beta = \frac{1}{\alpha}\ln\frac{1}{1-\alpha} = -\frac{1}{\alpha}\ln(1-\alpha)$$

Finiamo riassumendo le complessità di tutte le tecniche che abbiamo visto (notando che le liste di trabocco son la tecnica migliore):

Metodo	α	$I(\alpha)$	$S(\alpha)$
Lineare	$0 \leq \alpha < 1$	$\frac{(1-\alpha)^2+1}{2(1-\alpha)^2}$	$\frac{1-\alpha/2}{1-\alpha}$
Hashing doppio	$0 \leq \alpha < 1$	$\frac{1}{1-\alpha}$	$-\frac{1}{\alpha}\ln(1-\alpha)$
Liste di trabocco	$\alpha \geq 0$	$1+\alpha$	$1+\alpha/2$



I “problemi” dell’hashing sono:

- La scarsa “locality of reference” (cache miss)
- Non è possibile ottenere le chiavi in ordine (come ad esempio in un albero binario di ricerca)

Insiemi e dizionari

Molte delle strutture dati che abbiamo visto finora possono essere utilizzate per implementare insiemi o dizionari, ognuna con i suoi vantaggi e svantaggi in base al tipo di operazioni che dovremo fare.

Gli **insiemi** sono una semplice collezione di oggetti.
I **dizionari** sono delle associazioni chiave-valore.

Per gli insiemi possiamo utilizzare dei **vettori booleani**. Ad esempio se dobbiamo memorizzare un sottoinsieme dei numeri interi da 1 a m, abbiamo una collezione di m oggetti memorizzati in un vettore.
Possiamo rappresentare l’insieme come vettore booleano di m elementi, mettendo a $true$ gli elementi che fanno parte dell’insieme.
I **vantaggi** sono che è una implementazione notevolmente semplice ed è molto efficiente per la verifica dell’appartenenza di un elemento all’insieme.
Gli **svantaggi** sono che si occupa tanta memoria, ovvero $O(m)$, che può essere molto grande indipendentemente dalle dimensioni effettive (ovvero dal numero di elementi facenti effettivamente parte dell’insieme). Infatti alcune operazioni sono molto inefficienti, ovvero $O(m)$ anche se l’insieme ha pochi elementi.

Ecco l'implementazione:

SET (vettore booleano)

```
boolean[] V
int size
int capacity

SET Set(int m)
    SET t = new SET
    t.size = 0
    t.capacity = m
    t.V = new int[1...m] = {false}
    return t

boolean contains(int x)
    if 1 ≤ x ≤ capacity then
        return V[x]
    else
        return false

int size()
    return size

insert(int x)
    if 1 ≤ x ≤ capacity then
        if not V[x] then
            size = size + 1
            V[x] = true

remove(int x)
    if 1 ≤ x ≤ capacity then
        if V[x] then
            size = size − 1
            V[x] = false
```

```
SET union(SET A, SET B)
    int newsize = max(A.capacity, B.capacity)
    SET C = Set(newsize)
    for i = 1 to A.capacity do
        if A.contains(i) then
            C.insert(i)
    for i = 1 to B.capacity do
        if B.contains(i) then
            C.insert(i)
    return C
```

```
SET difference(SET A, SET B)
    SET C = Set(A.capacity)
    for i = 1 to A.capacity do
        if A.contains(i) and not B.contains(i) then
            C.insert(i)
    return C
```

```
SET intersection(SET A, SET B)
    int newsize = min(A.capacity, B.capacity)
    SET C = Set(newsize)
    for i = 1 to min.capacity, B.capacity) do
        if A.contains(i) and B.contains(i) then
            C.insert(i)
    return C
```

Per gli insiemi possiamo utilizzare anche dei **vettori/liste non ordinati**.
In questo caso le operazioni di ricerca, inserimento e cancellazione costano $O(n)$ (perchè dobbiamo scorrere tutti gli elementi già presenti).
L'opzione di inserimento però, assumendo l'assenza (ovvero che l'elemento non sia già salvato) costa $O(1)$.
Le operazioni di unione, intersezione e differenza costano $O(nm)$ (per ogni elemento del primo insieme devo scorrere tutti gli elementi del secondo).
Ecco un esempio della differenza:

```
SET difference(SET A, SET B)
SET C = Set()
foreach s ∈ A do
    if not B.contains(s) then
        C.insert(s)
return C
```

Possiamo utilizzare anche dei **vettori/liste ordinati**.
In questo caso le operazioni di ricerca, inserimento e cancellazione costano $O(n)$ (perchè dobbiamo scorrere tutti gli elementi già presenti).
La ricerca nel caso dei vettori ordinati costa $O(\log\ n)$ (ma non nelle liste)
Le operazioni di unione, intersezione e differenza costano $O(n)$.

```
LIST intersection(LIST A, LIST B)
LIST C = Set()
POS pos_a = A.head()
POS pos_b = B.head()
while not A.finished(pos_a) and
      not B.finished(pos_b) do
    if A.read(pos_a) == B.read(pos_b) then
        C.insert(C.tail(), A.read(pos_a))
        pos_a = A.next(pos_a)
        pos_b = B.next(pos_b)
    else if A.read(pos_a) < B.read(pos_b)
     then
        pos_a = A.next(pos_a)
    else
        pos_b = B.next(pos_b)
return C
```

Come ultime implementazioni degli insiemi vediamo gli **alberi bilanciati** e le **tabelle hash**:

Alberi bilanciati

- Ricerca, inserimento, cancellazione: $O(\log n)$
- Iterazione: $O(n)$
- Con ordinamento
- Implementazioni:
  - Java `TreeSet`
  - Python `OrderedSet`
  - C++ STL `set`

Hash table

- Ricerca, inserimento, cancellazione: $O(1)$
- Iterazione: $O(m)$
- Senza ordinamento
- Implementazioni:
  - Java `HashSet`
  - Python `set`
  - C++ STL `unordered_set`

Ecco un riassunto generale per le implementazioni di un insieme:

	contains lookup	insert	remove	min	foreach (Memoria)	Ordine
Vettore boolcano	$O(1)$	$O(1)$	$O(1)$	$O(m)$	$O(m)$	Sì
Lista non ordinata	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	No
Lista ordinata	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	Sì
Vettore ordinato	$O(\log n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	Sì
Alberi bilanciati	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Sì
Hash (Mem. interna)	$O(1)$	$O(1)$	$O(1)$	$O(m)$	$O(m)$	No
Hash (Mem. esterna)	$O(1)$	$O(1)$	$O(1)$	$O(m+n)$	$O(m+n)$	No

$m \equiv$ dimensione del vettore o della tabella hash

Divide-et-impera

Introduzione

Per risolvere un problema (ovvero scrivere un algoritmo) non esiste una sequenza predefinita di passi da fare per trovare la soluzione, ma esistono varie tecniche e modi, però si possono evidenziare quattro fasi che vengono eseguite la maggior parte delle volte:

- **Classificazione del problema**
- **Caratterizzazione della soluzione**
- **Tecnica di progetto**
- **Utilizzo di strutture dati**

Queste fasi però non sono necessariamente sequenziali, nel senso che può essere che per certi problemi sia meglio iniziare da una fase piuttosto che un'altra.

A grandi linee si possono identificare **3 tipologie** di problemi principali:

- **Problemi decisionali**
  Ovvero se il dato in ingresso soddisfa una certa proprietà o meno.
  La soluzione è una risposta: sì oppure no
  Un esempio è stabilire se un grafo è connesso oppure no.
- **Problemi di ricerca**
  Abbiamo uno spazio di ricerca (ovvero dove dovremo cercare) che è l'insieme delle soluzioni possibili.
  Una soluzione ammissibile è una soluzione che rispetta certi vincoli (imposti dalla ricerca)
  Un esempio è trovare la posizione di una sottostringa in una stringa.
- **Problemi di ottimizzazione**
  Ci sono tante soluzioni e ogni soluzione ha associata una funzione di costo.

Vogliamo trovare la soluzione di costo minimo.
Un esempio è trovare il cammino più breve tra due nodi

È fondamentale definire bene il problema in modo matematico e formale, perché senza capire bene di cosa stiamo parlando e cosa vogliamo ottenere è impossibile trovare un algoritmo per risolvere il problema.

Spesso la formulazione è banale, ma aiuta e può suggerire una prima idea di soluzione.
Ad esempio:
Data una sequenza di n elementi, una permutazione ordinata è data dal minimo seguito da una permutazione ordinata dei restanti $n-1$ elementi (Selection Sort)
Qui vediamo che la formulazione matematica da subito un'idea di soluzione (ricorsiva).

- Sottostruttura ottima $\rightarrow$ Programmazione dinamica
- Proprictà greedy $\rightarrow$ Tecnica greedy

Esistono varie tecniche standard per la risoluzione dei problemi, le principali sono:

- **Divide-et-impera**
  Il problema viene suddiviso in sotto-problemi indipendenti, che vengono risolti ricorsivamente (top-down)
  Si usa per problemi di **decisione** e **ricerca**
- **Programmazione dinamica**
  La soluzione viene costruita (bottom-up) a partire da un insieme di sotto-problemi potenzialmente ripetuti
  Si usa per problemi di **ottimizzazione**
- **Memoization (o annotazione)**
  Versione top-down della programmazione dinamica (fondendolo con un approccio divide-et-impera)
- **Tecnica greedy**
  Approccio "ingordo", ovvero si fa sempre la scelta localmente ottima
- **Backtrack**
  Si procede per "tentativi", tornando ogni tanto sui nostri passi
- **Ricerca locale**
  La soluzione ottima viene trovata "migliorando" via via soluzioni già esistenti
- **Algoritmi probabilistici**
  Meglio scegliere con giudizio (ma in maniera costosa) o scegliere a caso (in maniera gratuita)

Il metodo **divide-et-impera** è composto da 3 fasi:

1. **Divide**
   Dividi il problema in sotto-problemi più piccoli e indipendenti
2. **Impera**
   Risolvi i sotto-problemi in maniera ricorsiva

3. **Combina**
   “Unisci” le soluzioni dei sottoproblemi

Non esiste una ricetta unica per la tecnica divide-et-impera, nel senso che le varie fasi possono essere più o meno complesse. (Ad esempio MergeSort ha un “divide” banale ma un “combina” complesso, invece QuickSort ha un “divide” complesso ma un “combina” banale).

Un esempio di algoritmo divide-et-impera è la ricerca del minimo:

```
int minrec(int[] A, int i, int j)
if i == j then
    return A[i]
else
    m = ⌊(i + j)/2⌋
    return min(minrec(A, i, m), minrec(A, m + 1, j))
```

Vediamo che la complessità è $\Theta(n)$:

Complessità

$$T(n) = \begin{cases} 2T(n/2) + 1 & n > 1 \\ 1 & n = 1 \end{cases}$$

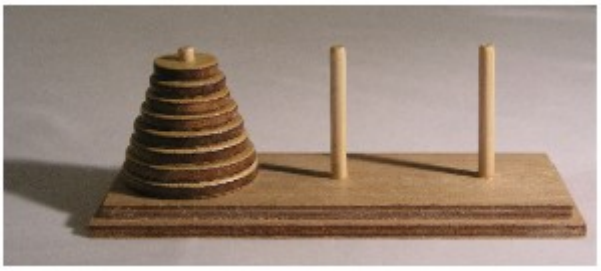
$T(n) = \Theta(n)$ – Non ne vale la pena!

Visto che la complessità è la stessa di un algoritmo lineare (che abbiamo già visto), non ha senso usare divide-et-impera, in quanto rendiamo la soluzione complessa (con ricorsione) senza guadagnare nulla.

Torre di Hanoi

La torre di Hanoi è un gioco matematico composto dalle seguenti regole:

- Ci sono **3 pioli**
- Ci sono n dischi di dimensione diverse, inizialmente i dischi sono impilati in ordine decrescente nel piolo di sinistra
- Si può muovere al massimo un disco alla volta
- Non si può mai impilare un disco più grande su uno più piccolo
- Lo scopo è impilare in ordine decrescente i dischi sul piolo di destra utilizzando il piolo centrale come appoggio



Vediamo direttamente l'implementazione dell'algoritmo e poi discutiamolo:

```
hanoi(int n, int src, int dest, int middle)
if n == 1 then
    print src → dest
else
    hanoi(n − 1, src, middle, dest)
    print src → dest
    hanoi(n − 1, middle, dest, src)
```

Come possiamo vedere, se dobbiamo spostare un solo disco da sorgente a destinazione, basta spostarlo. Se invece abbiamo più dischi, spostiamo prima tutti i dischi "sopra" sull'appoggio, poi spostiamo il nostro disco sulla destinazione, e poi spostiamo i dischi che abbiamo messo sull'appoggio sulla destinazione.
Il costo computazionale è facile da calcolare:

Costo computazionale

- Ricorrenza: $T(n) = 2T(n-1) + 1$
- Costo computazionale? $O(2^n)$
- Questa soluzione è ottima (si può dimostrare)

1 Domanda: Complessità Torri di Hanoi

$$\begin{aligned} T(n) &= 2T(n-1)+1 \\ &= 4T(n-2)+2+1 \\ &= 2^i T(n-i) + \sum_{j=0}^{i-1} 2^j \\ &= \sum_{j=0}^{n-1} 2^j + T(1) \\ &= 2^n \end{aligned}$$

Quicksort

Il Quicksort è un algoritmo di ordinamento basato sulla tecnica divide-et-impera.
La complessità di Quicksort è la seguente:

- Caso medio: $O(n \ \log \ n)$
- Caso pessimo: $O(n^2)$

Il fattore costante di QuickSort è migliore di MergeSort (che ha sempre complessità $O(n \ \log \ n)$) e in più non utilizza memoria addizionale come MergeSort. Tuttavia nei casi pessimi è inefficiente, ma esistono delle tecniche "euristiche" per evitare il caso pessimo, quindi QuickSort è spesso preferito ad altri algoritmi.
Vediamo ora l'algoritmo.
In input viene dato un vettore $A[1 \ \ldots \ n]$ e due indici *lo* e *hi* tali che $1 \le lo \le hi \le n$:

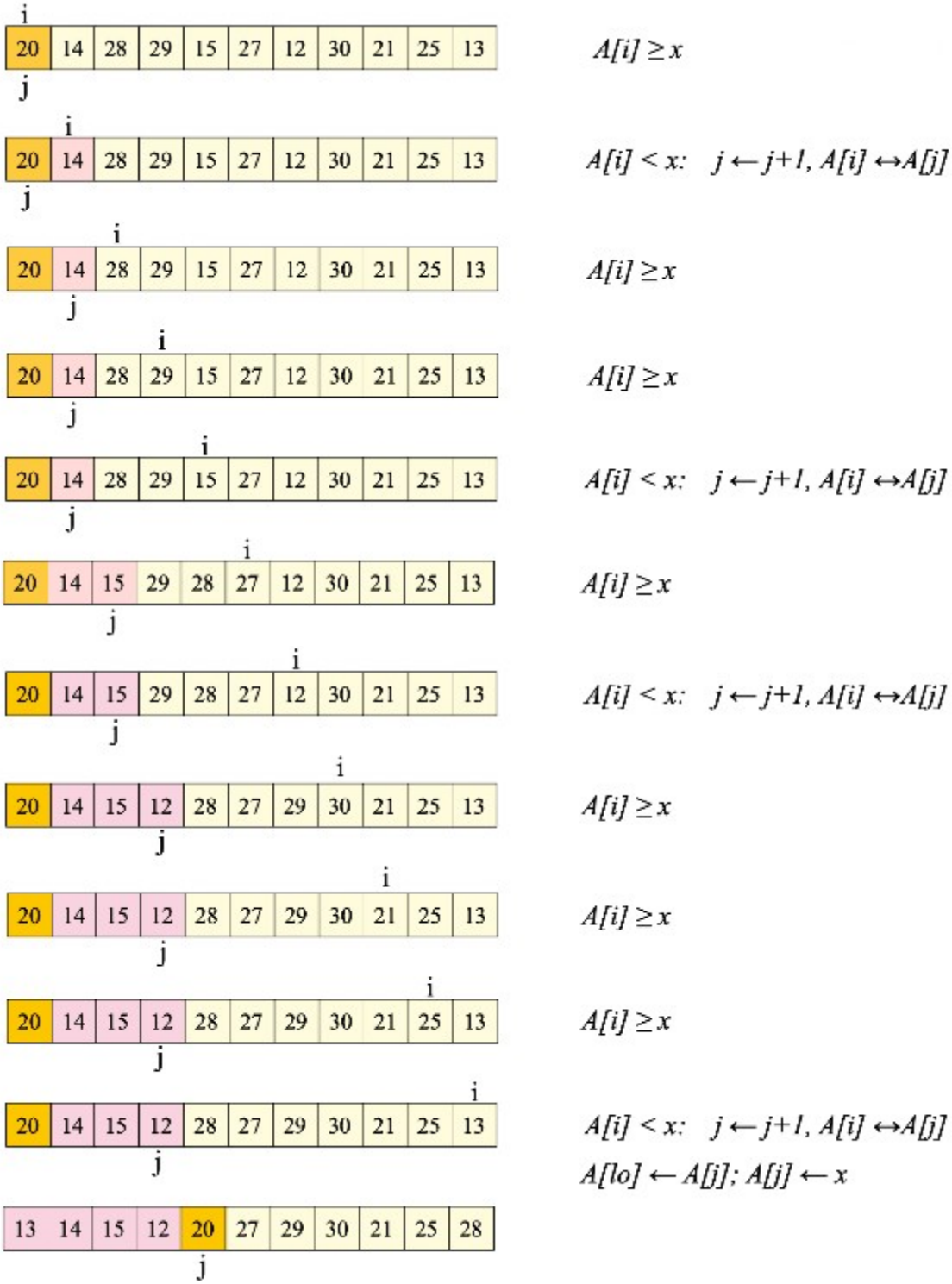
- **Divide**
  Viene scelto un valore p (detto **perno** o **pivot**) tale che $p \in A[lo \ \ldots \ hi]$
  Successivamente dobbiamo spostare gli elementi del vettore $A[lo \ \ldots \ hi]$ in modo che, denotando con j la nuova posizione del perno, per ogni $i \in [lo \ \ldots \ j-1]$ deve valere $A[i] < A[j]$, deve valere $p = A[j]$, e per ogni $i \in [j+1 \ \ldots \ hi]$ deve valere $A[i] > A[j]$
- **Impera**
  Ordina i due sottovettori $A[lo \ \ldots \ j-1]$ e $A[j+1 \ \ldots \ hi]$, richiamando ricorsivamente QuickSort su di essi.
- **Combina**
  In questo caso non deve fare nulla in quanto $A[lo \ \ldots \ j-1]$ è ordinato, j è maggiore di tutti gli elementi del primo sottovettore e minore di tutti gli elementi del secondo, $A[j+1 \ \ldots \ hi]$ è ordinato.
  Quindi il vettore è già ordinato.

Vediamo ora l'implementazione, vedendo prima la funzione che "trova" il perno, posiziona gli elementi come richiesto da "divide" e ritorna la posizione del perno:

```
int pivot(ITEM[] A, int lo, int hi)
ITEM pivot = A[lo]
int j = lo
for i = lo + 1 to hi do
    if A[i] < pivot then
        j = j + 1
        swap(A, i, j)
A[lo] = A[j]
A[j] = pivot
return j
```

```
swap(ITEM[] A, int i, int j)
ITEM temp = A[i]
A[i] = A[j]
A[j] = A[temp]
```

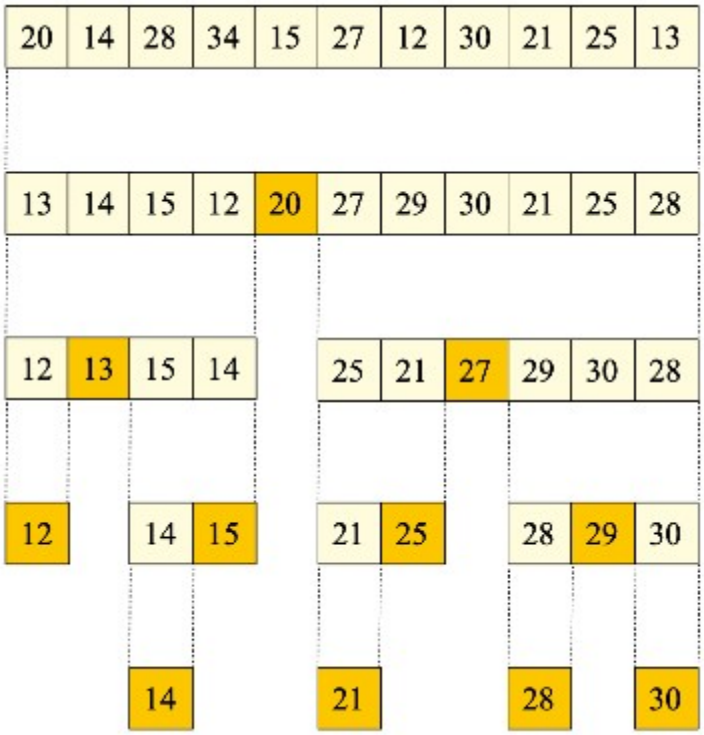
Vediamo un esempio di applicazione di questo algoritmo per capirne il funzionamento:



Ora vediamo la funzione principale, che ottiene il perno e chiama ricorsivamente se stessa:

```
QuickSort(ITEM[] A, int lo, int hi)
if lo < hi then
    int j = pivot(A, lo, hi)
    QuickSort(A, lo, j − 1)
    QuickSort(A, j + 1, hi)
```

Vediamo ora l'algoritmo completo sull' esempio di prima:



Vediamo ora la complessità:

- Il costo di $pivot()$ è $\Theta(n)$ in quanto scorriamo tutto il vettore una volta sola.
- Il costo di $QuickSort()$
    - Nel caso pessimo
      Il vettore di dimensione n viene diviso in due vettori da 0 e $n-1$ elementi, quindi
      $T(n)=T(n-1)+T(0)+\Theta(n)=T(n-1)+\Theta(n)=\Theta(n^2)$
    - Nel caso ottimo

      Il vettore di dimensione n viene diviso in due vettori, entrambi da $\frac{n}{2}$ elementi,

      quindi $T(n)=2T\left(\frac{n}{2}\right)+\Theta(n)=\Theta(n \log n)$

Solitamente i partizionamenti sono parzialmente bilanciati, ovvero nel caso medio QuickSort è molto più vicino al caso ottimo che al caso pessimo.

- Esempio: Partizionamento 9-a-1:

$$T(n) = T(n/10) + T(9n/10) + cn = \Theta(n \log n)$$

- Esempio: Partizionamento 99-a-1:

$$T(n) = T(n/100) + T(99n/100) + cn = \Theta(n \log n)$$

In maniera “euristica” possiamo dire che i partizionamenti parzialmente bilanciati “dominano” quelli pessimi.

Possiamo pensare ad un altro modo per scegliere il perno, invece di prendere il primo elemento, prendiamo il valore mediano tra il primo elemento, l’ultimo elemento e l’elemento nella posizione centrale. Così abbiamo più possibilità di evitare il caso pessimo:

```
int pivot(ITEM[] A, int lo, int hi)
int m = ⌊(lo + hi)/2⌋
if A[lo] > A[hi] then        % Sposta il massimo in ultima posizione
    swap(A, lo, hi)
if A[m] > A[hi] then         % Sposta il massimo in ultima posizione
    swap(A, m, hi)
if A[m] > A[lo] then         % Sposta il mediano in prima posizione
    swap(A, m, lo)
ITEM pivot = A[lo]
[...]
```

Algoritmo di Strassen

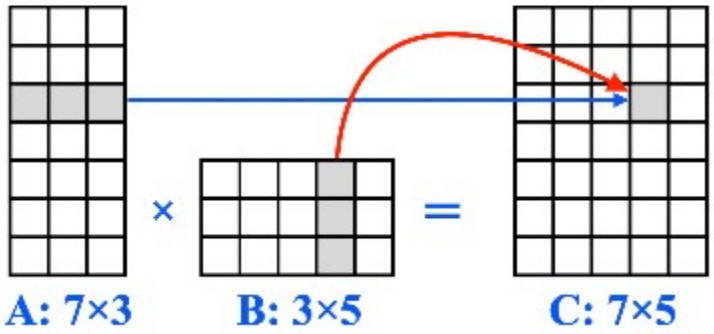
L’algoritmo di Strassen viene utilizzato per migliorare la complessità della moltiplicazione tra matrici.
L’algoritmo “base” per la moltiplicazione matriciale è il seguente:

$$C = A \times B$$

$$c_{i,j} = \sum_{k=1}^{n_k} a_{i,k} \cdot b_{k,j}$$

$$T(n) = \Theta(n_i \cdot n_k \cdot n_j) = \Theta(n^3)$$

```
matrixProduct(float[][] A, B, C, int n_i, n_k, n_j)
for i = 1 to n_i do                          % Righe
    for j = 1 to n_j do                      % Colonne
        C[i, j] = 0
        for k = 1 to n_k do
            C[i, j] = C[i, j] + A[i, k] · B[k, j]
```



La complessità di questo algoritmo, come possiamo vedere, (assumendo siano matrici quadrate) è $\Theta(n^3)$.

Proviamo ad applicare l'approccio divide-et-impera, quindi suddividiamo le matrici $n \times n$ in quattro matrici $\frac{n}{2} \times \frac{n}{2}$

$$A = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix} \qquad B = \begin{bmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{bmatrix}$$

Eseguiamo quindi il calcolo del prodotto matrice:

$$C = \begin{bmatrix} A_{1,1} \times B_{1,1} + A_{1,2} \times B_{2,1} & A_{1,1} \times B_{1,2} + A_{1,2} \times B_{2,2} \\ A_{2,1} \times B_{1,1} + A_{2,2} \times B_{2,1} & A_{2,1} \times B_{1,2} + A_{2,2} \times B_{2,2} \end{bmatrix}$$

L'equazione di ricorrenza di questo algoritmo è il seguente (fermandoci quando $n = 1$ visto che il prodotto di matriciale è se stesso)

$$T(n) = \begin{cases} 1 & n = 1 \\ 8T(n/2) + n^2 & n > 1 \end{cases}$$

Vediamo ora come possiamo migliorare la complessità di questo algoritmo, considerando il calcolo dei seguenti elementi intermedi:

Calcolo elementi intermedi

$$X_1 = (A_{11} + A_{22}) \times (B_{11} + B_{22})$$
$$X_2 = (A_{21} + A_{22}) \times B_{11}$$
$$X_3 = A_{11} \times (B_{12} - B_{22})$$
$$X_4 = A_{22} \times (B_{21} - B_{11})$$
$$X_5 = (A_{11} + A_{12}) \times B_{22}$$
$$X_6 = (A_{21} - A_{11}) \times (B_{11} + B_{12})$$
$$X_7 = (A_{12} - A_{22}) \times (B_{21} + B_{22}).$$

Equazione di ricorrenza

$$T(n) = \begin{cases} 1 & n = 1 \\ 7T(n/2) + n^2 & n > 1 \end{cases}$$
$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$$

Il calcolo della matrice finale sarà fatto nel seguente modo:

$$C = \begin{bmatrix} X_1 + X_4 - X_5 + X_7 & X_3 + X_5 \\ X_2 + X_4 & X_1 + X_3 - X_2 + X_6 \end{bmatrix}$$

In realtà esistono algoritmi ancora migliori (ad esempio $O\left(n^{2.37}\right)$ è il migliore attualmente) ma il limite inferiore al problema è $\Omega\left(n^2\right)$.

Conclusioni

È vantaggioso applicare la tecnica divide-et-impera (quando possibile) nei casi in cui i passi "divide" ed "combina" sono semplici.
Ovviamente l'algoritmo divide-et-impera deve avere costi minori rispetto al corrispondente algoritmo iterativo (per il sorting è vantaggioso, per la ricerca del minimo no).

Altri vantaggi di questa tecnica sono la **facile parallelizzazione** e un utilizzo ottimale della cache

Esercizio

Gap

In un vettore V contenente $n \geq 2$ interi, un gap è un indice i, $1 < i \leq n$, tale che $V[i-1] < V[i]$.

- Dimostrare che se $n \geq 2$ e $V[1] < V[n]$, allora V contiene almeno un gap
- Progettare un algoritmo che, dato un vettore V contenente $n \geq 2$ interi e tale che $V[1] < V[n]$, restituisca la posizione di un gap nel vettore.

Per assurdo:

- Supponiamo che non ci sia un gap nel vettore
- Allora $V[1] \geq V[2] \geq V[3] \geq \ldots \geq V[n-1] \geq V[n]$
- Assurdo, perché $V[1] < V[n]$.

Gap – Dimostrazione per induzione

Proviamo a riformulare la proprietà tenendo conto di due indici:

- Sia V un vettore di dimensione n
- Siano i, j due indici tali che $1 \leq i < j \leq n$ e $V[i] < V[j]$

In altre parole, ci sono più di due elementi nel sottovettore $V[i \ldots j]$ e il primo elemento $V[i]$ è più piccolo dell'ultimo elemento $V[j]$.

Vogliamo provare per induzione sulla dimensione n del sottovettore che il sottovettore contiene un gap.

- Caso base: $n = j - i + 1 = 2$, i.e. $j = i + 1$:
  $V[i] < V[j]$ implica che $V[i] < V[i+1]$, che è un gap.
- Ipotesi induttiva: dato un qualunque (sotto)vettore $V[h \ldots k]$ di dimensione $n' < n$, tale che $V[h] < V[k]$, allora $V[h \ldots k]$ contiene un gap
- Passo induttivo: consideriamo un qualunque elemento m tale che $i < m < j$. Almeno uno dei due casi seguenti è vero:
  - Se $V[m] < V[j]$, allora esiste un gap in $V[m \ldots j]$, per ipotesi induttiva
  - Se $V[i] < V[m]$, allora esiste un gap in $V[i \ldots m]$, per ipotesi induttiva

```
int gap(int[] V, int n)
return gapRec(V, 1, n)
```

```
int gapRec(int[] V, int i, int j)
if j == i + 1 then
    return j
int m = ⌊(i + j)/2⌋
if V[m] < V[j] then
    return gapRec(V, m, j)
else
    return gapRec(V, i, m)
```

Strutture dati speciali

Fino ad ora abbiamo visto delle strutture dati (ad esclusione dei grafi) che implementano i seguenti tipi di dato astratto:

- Sequenze
- Insiemi
- Dizionari

Le **strutture dati speciali** si usano quando non tutte le operazioni sono necessarie, quindi è possibile realizzare strutture dati più efficienti, “specializzate” appunto per particolari compiti; Oppure quando le operazioni base (come inserimento, cancellazione, lettura, etc) possono non essere sufficienti, a volte servono delle **operazioni speciali**.

Esempi

- Code con priorità
- Insiemi disgiunti
- Interval/segment tree
- K-D tree
- Trie
- Fenvick Tree
- Merkle tree
- Secondo Wikipedia, 338 pagine nella categoria strutture dati...

Code con priorità

Una **coda con priorità** (o priority queue) è una struttura dati astratta, simile a una coda, in cui ogni elemento possiede una sua "**priorità**".
Esistono due versioni di coda con priorità:

- **Min-priority queue**: estrazione per valori crescenti di priorità
- **Max-priority queue**: estrazione per valori decrescenti di priorità

Le operazioni permesse in questa struttura dati sono:

- Inserimento in coda
- Estrazione dell'elemento con priorità di valore min/max
- Modifica priorità (decremento/incremento) di un elemento inserito

Ecco la specifica:

MINPRIORITYQUEUE

```
% Crea una coda con priorità con capacità n, vuota
PRIORITYQUEUE PriorityQueue(int n)

% Restituisce true se la coda con priorità è vuota
boolean isEmpty()

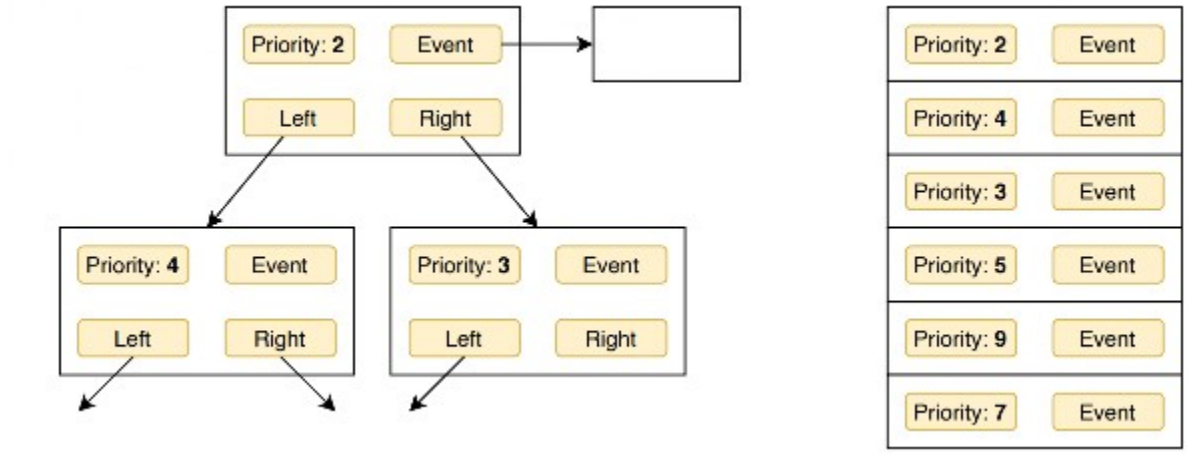
% Restituisce l'elemento minimo di una coda con priorità non vuota
ITEM min()

% Rimuove e restituisce il minimo da una coda con priorità non vuota
ITEM deleteMin()

% Inserisce l'elemento x con priorità p nella coda con priorità. Restituisce
% un oggetto PRIORITYITEM che identifica x all'interno della coda
PRIORITYITEM insert(ITEM x, int p)

% Diminuisce la priorità dell'oggetto identificato da y portandola a p
decrease(PRIORITYITEM y, int p)
```

Un reality check di una coda con priorità è un simulatore event-driven, in cui ogni evento ha associato un timestamp di esecuzione e ogni evento può generare nuovi eventi con timestamp diversi. In questa situazione si può usare una coda con priorità per eseguire gli eventi in ordine di timestamp



La coda con priorità verrà utilizzata in algoritmi che vedremo prossimamente come:

- Algoritmo di Dijkstra
- Codifica di Huffmann
- Algoritmo di Prim per gli alberi di copertura di peso minimo

Ecco le implementazioni con i vari costi di una coda con priorità:

Metodo	Lista/vettore non ordinato	Lista Ordinata	Vettore Ordinato	Albero RB
min()	$O(n)$	$O(1)$	$O(1)$	$O(\log n)$
deleteMin()	$O(n)$	$O(1)$	$O(n)$	$O(\log n)$
insert()	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$
decrease()	$O(n)$	$O(n)$	$O(\log n)$	$O(\log n)$

Vettore heap

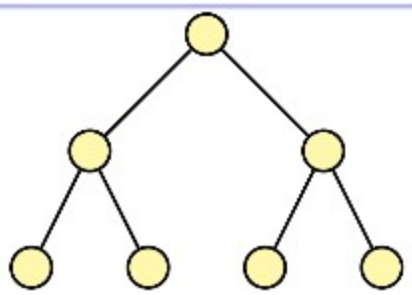
L'**heap** è una struttura dati speciale che associa i **vantaggi di un albero** (l'esecuzione in tempo $O(\log\ n)$) e i **vantaggi di un vettore** (la memorizzazione efficiente).
È stata inventata nel 1964 da J. Williams.
È stata inventata per implementare un nuovo algoritmo di ordinamento: l'**HeapSort**, ma Williams intuì subito che poteva esser utilizzata per altri scopi (come la coda con priorità per appunto).
Seguiamo quindi l'approccio storico nel presentare gli heap:

- Vediamo prima l'HeapSort
- Poi le code con priorità tramite heap

Riprendiamo alcune definizioni fatte in precedenza:

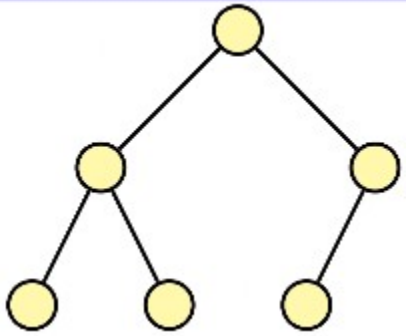
Albero binario perfetto

- Tutte le foglie hanno la stessa profondità h
- Nodi interni hanno tutti grado 2
- Dato il numero di nodi n, ha altezza $h = \lfloor \log n \rfloor$
- Dato l'altezza h, ha numeri di nodi $n = 2^{h+1} - 1$

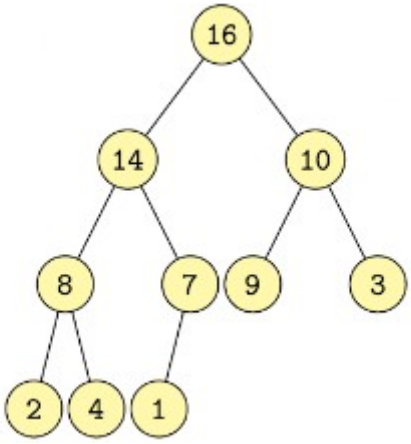


Albero binario completo

- Tutte le foglie hanno profondità h o $h-1$
- Tutti i nodi a livello h sono "accatastati" a sinistra
- Tutti i nodi interni hanno grado 2, eccetto al più uno
- Dato il numero di nodi n, ha altezza $h = \lfloor \log n \rfloor$



Un albero **max-heap** (o **min-heap**) è un albero binario completo tale che il valore memorizzato in ogni nodo è **maggiore** (o **minore**) dei valori memorizzati nei suoi figli.
(Le definizioni e gli algoritmi per gli alberi max-heap sono simmetrici rispetto agli algoritmi per gli alberi min-heap).



Un albero heap **non impone** una relazione di **ordinamento totale** fra i figli di un nodo.
Un albero heap è un **ordinamento parziale**:

- Riflessivo
  Ogni nodo è $\geq$ di se stesso
- Antisimmetrico
  Se $n \geq m$ e $m \geq n$, allora $m = n$
- Transitivo
  Se $n \geq m$ e $m \geq r$, allora $n \geq r$

La nozione di ordinamento parziale è più debole di quella di ordinamento totale, però è più semplice da costruire.

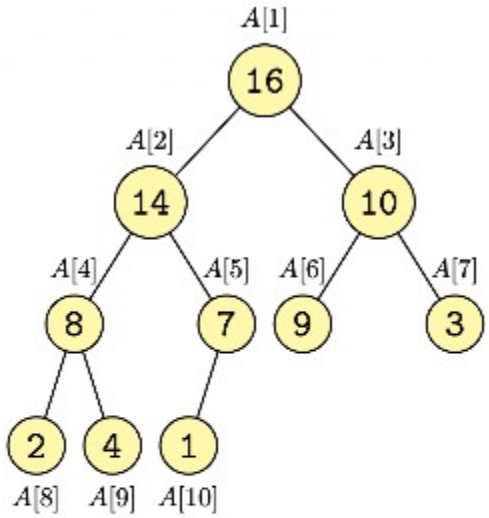
Un albero heap può essere rappresentato mediante un **vettore heap**.
Si può usare la notazione degli array che parte da 1:

Vettore heap

Un albero heap può essere rappresentato tramite un vettore heap

Memorizzazione ($A[1 \dots n]$)

Radice	$root() = 1$
Padre nodo i	$p(i) = \lfloor i/2 \rfloor$
Figlio sx nodo i	$l(i) = 2i$
Figlio dx nodo i	$r(i) = 2i + 1$



A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]	A[10]
16	14	10	8	7	9	3	2	4	1

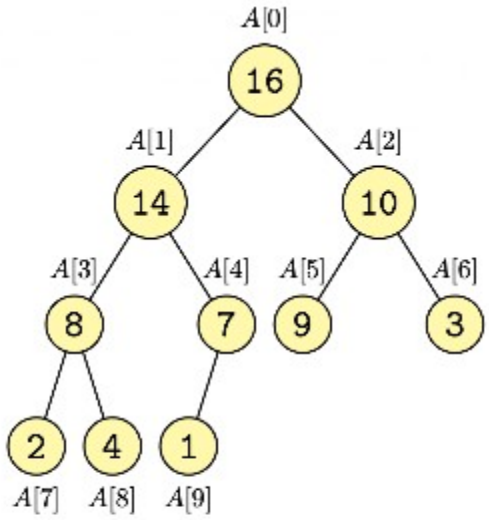
O la notazione degli array che parte da 0:

Vettore heap

Un albero heap può essere rappresentato tramite un vettore heap

Memorizzazione ($A[0 \dots n-1]$)

Radice	$root() = 0$
Padre nodo i	$p(i) = \lfloor (i-1)/2 \rfloor$
Figlio sx nodo i	$l(i) = 2i + 1$
Figlio dx nodo i	$r(i) = 2i + 2$



$A[0]$	$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$	$A[6]$	$A[7]$	$A[8]$	$A[9]$
16	14	10	8	7	9	3	2	4	1

Le proprietà di max-heap su un vettore sono le seguenti:

$$A[i] \geq A[l(i)],\ A[i] \geq A[r(i)]$$

Invece le proprietà di min-heap su un vettore sono le seguenti:

$$A[i] \leq A[l(i)],\ A[i] \leq A[r(i)]$$

HeapSort

Vediamo ora l'HeapSort.

Come prima cosa vediamo come organizzare l'HeapSort, ovvero come ottenere una struttura heap su cui poter applicare l'HeapSort.

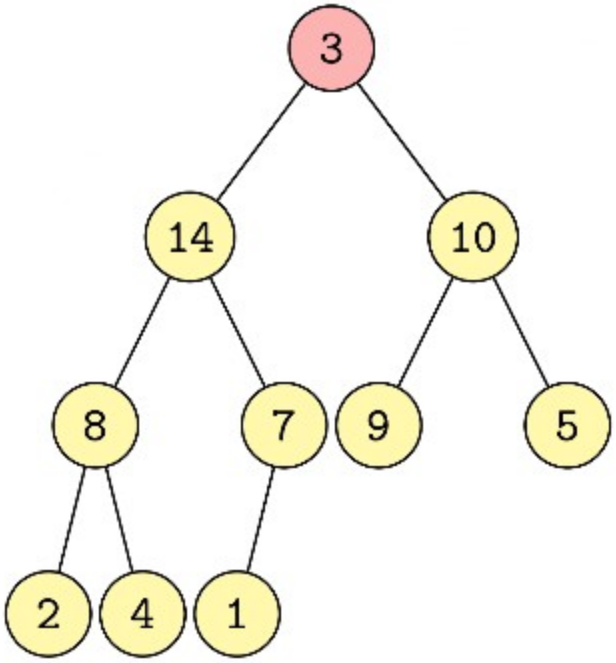
Abbiamo due funzioni per costruire un vettore heap:

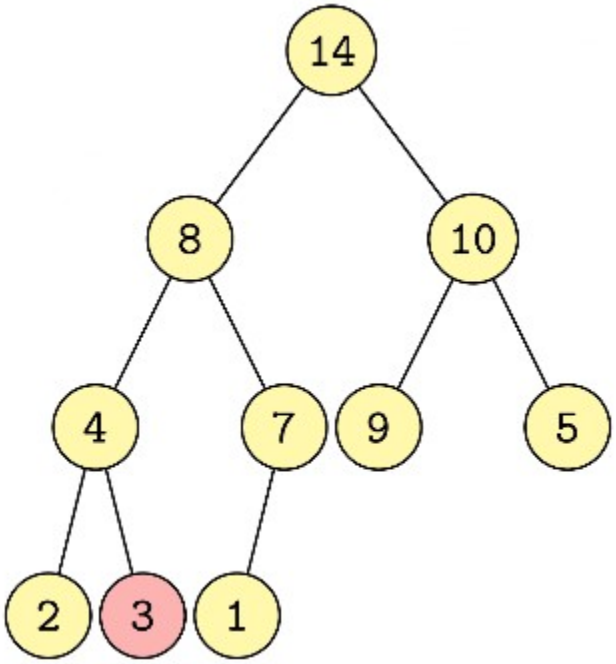
- $heapBuild()$
  Costruisce un max-heap a partire da un vettore non ordinato.
- $maxHeapRestore()$
  Ripristina la proprietà max-heap avendo avendo i due figli che sono già dei max-heap.

Vediamo in dettaglio $maxHeapRestore()$:

- **Input**
  Un vettore A e un indice i, tale per cui gli alberi binari con radici $l(i)$ e $r(i)$ sono già max-heap. (Non sappiamo nulla sulla relaziona tra i, $l(i)$ e $r(i)$, ovvero non è detto che il sottoalbero con radice i sia un max-heap)
- **Output**
  Modificare in-place (ovvero sullo stesso vettore dell'input senza uso di strutture ausiliarie) il vettore A in modo tale che l'albero binario con radice i sia un max-heap.

Vediamo un esempio:





Il codice della funzione *maxHeapRestore*() è il seguente:

```
maxHeapRestore(ITEM[] A, int i, int dim)
int max = i
if l(i) ≤ dim and A[l(i)] > A[max] then
    max = l(i)
if r(i) ≤ dim and A[r(i)] > A[max] then
    max = r(i)
if i ≠ max then
    swap(A, i, max)
    maxHeapRestore(A, max, dim)
```

Quale è la complessità computazionale di *maxHeapRestore*()?

Ad ogni chiamata vengono eseguiti $O(1)$ confronti (un numero costante di confronti, 2), nel caso peggiore si chiama ricorsivamente *maxHeapRestore*() su uno dei figli, l'esecuzione termina quando si raggiunge una foglia nel caso peggiore e abbiamo un'altezza dell'albero pari a $\lfloor \log n \rfloor$

Di conseguenza la complessità è:

$$T(n) = O(\log\ n)$$

Per dimostrare la correttezza procediamo per induzione sull'altezza h dell'albero.

Il teorema è che al termine dell'esecuzione, l'albero radicato in $A[i]$ rispetta la proprietà max-heap.

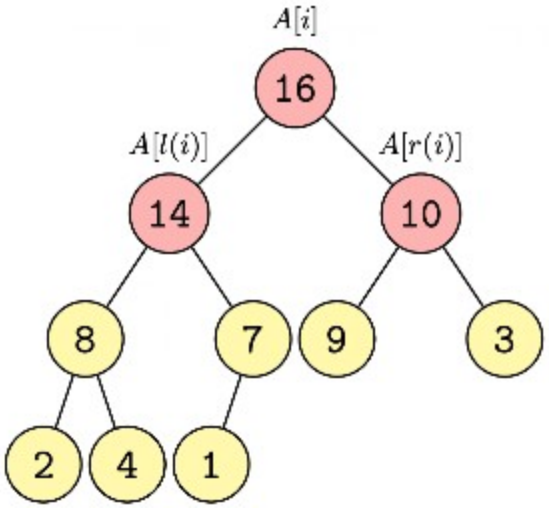
Il caso base è rispettato in quanto per $h = 0$, l'albero è composto da un solo nodo che rispetta sempre la proprietà heap.

L'ipotesi induttiva è: l'algoritmo funziona correttamente su tutti gli alberi di altezza minore di h.

Induzione - Altezza h - Caso 1

$A[i] \geq A[l(i)], A[i] \geq A[r(i)]$:

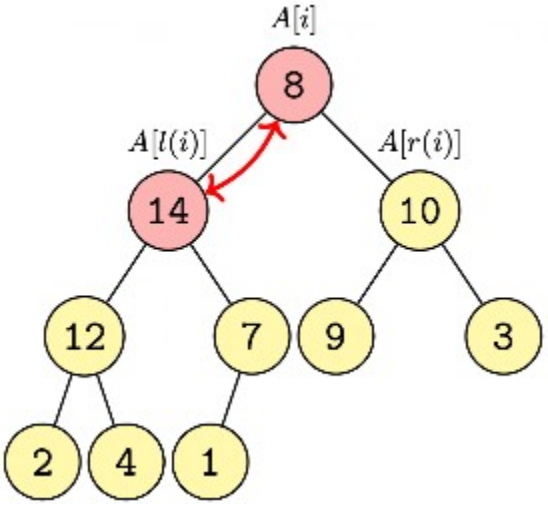
- L'albero radicato in $A[i]$ rispetta la proprietà max-heap
- L'algoritmo termina (CVD)

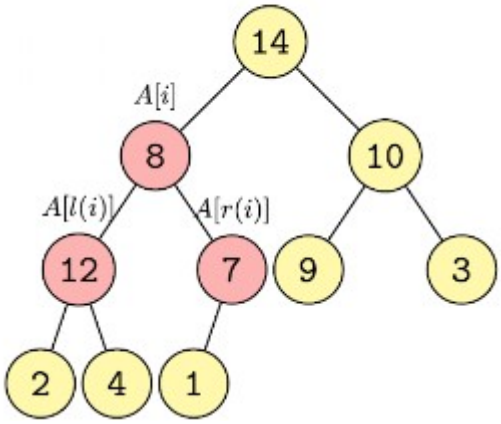


Induzione - Altezza h - Caso 2

$A[l(i)] > A[i], A[l(i)] > A[r(i)]$:

- Viene fatto uno scambio $A[i] \leftrightarrow A[l(i)]$
- Dopo lo scambio, $A[i] \geq A[l(i)], A[i] \geq A[r(i)]$
- Il sottoalbero $A[r(i)]$ è inalterato e rispetta la proprietà heap





- Il sottoalbero $A[l(i)]$ può aver perso la proprietà heap
- Si applica maxHeapRestore() ricorsivamente su di $A[l(i)]$, che ha altezza minore di h

Il passo induttivo del caso 3 è simmetrico a quello del caso 2.

Vediamo ora la funzione *heapBuild*$()$. Sia $A[1 \ldots n]$ un vettore da ordinare secondo max-heap. Notiamo che tutti i nodi $A[\lfloor n/2 \rfloor + 1 \ldots n]$ sono foglie dell'albero, e quindi possiamo considerarli come alberi max-heap contenenti 1 elemento.
La procedura *heapBuild*$()$ consiste quindi nell'attraversare i restanti nodi dell'albero, partendo da $\lfloor n/2 \rfloor$ fino ad 1, ed eseguire *maxHeapRestore*$()$ su ognuno di essi.

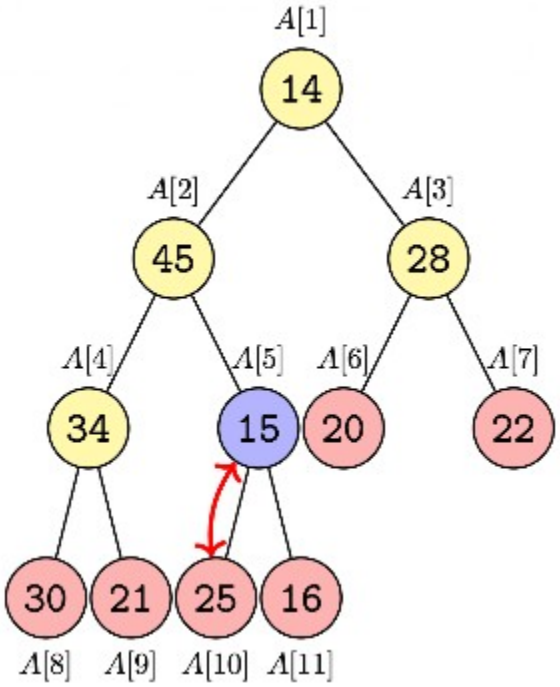
Il codice sarà il seguente:

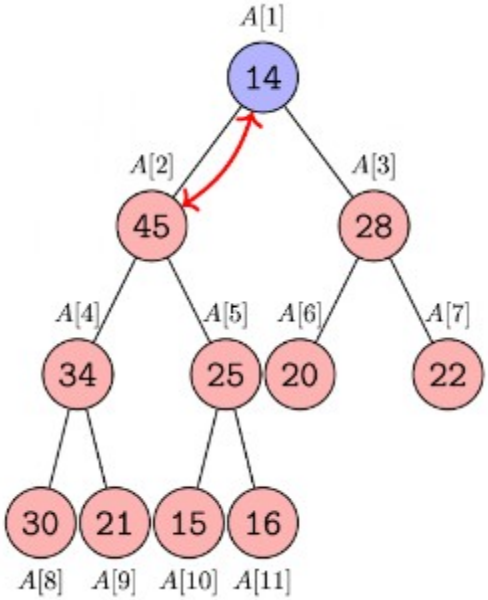
```
heapBuild(ITEM[] A, int n)

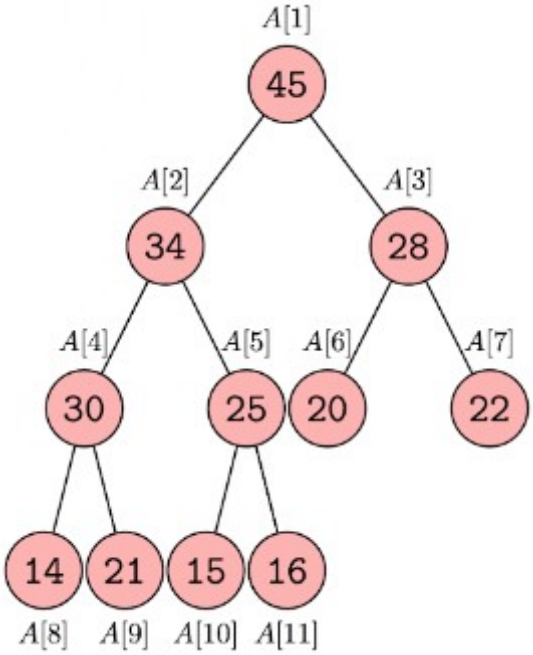
for i = ⌊n/2⌋ downto 1 do
    maxHeapRestore(A, i, n)
```

Ecco un esempio pratico:

- I nodi $A[\lfloor n/2 \rfloor + 1 \ldots n]$ sono foglie dell'albero e quindi heap di un elemento
- Per ogni posizione da $\lfloor n/2 \rfloor$ fino ad 1, si esegue maxHeapRestore()







Verifichiamo la correttezza di *heapBuild*().
Dobbiamo dimostrare l'**invariante di ciclo** che ci dice:
All'inizio di ogni iterazione del ciclo **for**, i nodi $[i+1,\ldots,n]$ sono radice di uno heap.
Se la dimostriamo, allora alla fine del ciclo avremo che ogni nodo è radice di uno heap.
Ricordiamo che $l(i)=2i$

Dimostrazione – Inizializzazione

- All'inizio, $i = \lfloor n/2 \rfloor$.
- Supponiamo che $\lfloor n/2 \rfloor + 1$ non sia una foglia
- Quindi ha almeno il figlio sinistro: $2\lfloor n/2 \rfloor + 2$
- Questo ha indice $n + 1$ oppure $n + 2$, assurdo perché n è la dimensione massima
- La dimostrazione vale per tutti gli indici successivi

Quindi l'invariante di ciclo è valida al primo ciclo, ora verifichiamo la conservazione.

Dimostrazione – Conservazione

- È possibile applicare maxHeapRestore al nodo i, perché $2i < 2i + 1 \leq n$ sono entrambi radici di heap
- Al termine dell'iterazione, tutti i nodi $[i \dots n]$ sono radici di heap

Dimostrazione – Conclusione

- Al termine, $i = 0$. Quindi il nodo 1 è radice di uno heap.

Vediamo ora la complessità di *heapBuild*()

Qual è la complessità di HEAPBUILD()?

- Limite superiore: $T(n) = O(n \log n)$
- Limite inferiore: $T(n) = \Omega(n \log n)$?

Osserviamo quindi che dando in pasto a *heapBuild*() un vettore che ha già la proprietà di max-heap, allora la complessità è $\Omega(n)$, in quanto basta scorrere tutti i nodi e basta.

Le operazioni maxHeapRestore() vengono eseguite un numero decrescente di volte su heap di altezza crescente

Altezza	**# Volte**
0	$\lfloor n/2 \rfloor$
1	$\lfloor n/4 \rfloor$
2	$\lfloor n/8 \rfloor$
...	...
h	$\lfloor n/2^{h+1} \rfloor$

$$
\begin{aligned}
T(n) &\leq \sum_{h=1}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} h \\
&= n \sum_{h=1}^{\lfloor \log n \rfloor} \left(\frac{1}{2}\right)^{h+1} h \\
&= n/2 \sum_{h=1}^{\lfloor \log n \rfloor} \left(\frac{1}{2}\right)^{h} h \\
&\leq n/2 \sum_{h=1}^{+\infty} \left(\frac{1}{2}\right)^{h} h \qquad = n = O(n)
\end{aligned}
$$

Formula: $\sum_{h=1}^{+\infty} h x^h = \frac{x}{(1-x)^2}, \text{per } x < 1$

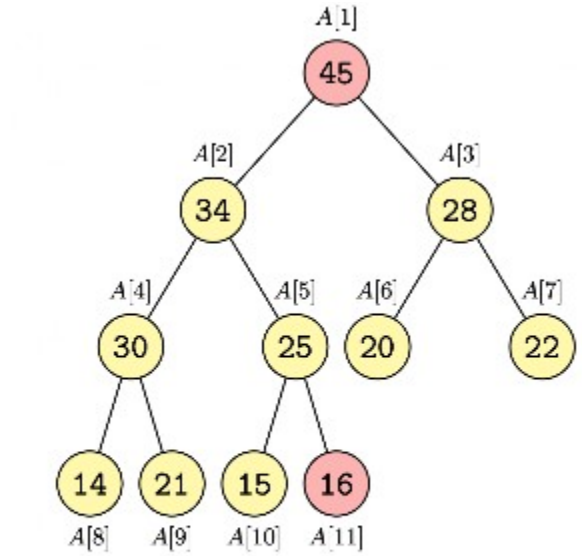
Quindi all'altezza 0 (dal basso) chiamo $maxHeapRestore()$ $\frac{n}{2}$ volte, anche se in realtà non faccio nulla e infatti moltiplico per l'altezza 0.

All'altezza 1 chiamo $maxHeapRestore()$ $\frac{n}{4}$ volte, e farà al massimo uno scambio.

All'altezza 2 chiamo $maxHeapRestore()$ $\frac{n}{8}$ volte, e farà al massimo due scambi. E cosi via.

Otteniamo quindi che $heapBuild()$ ha complessità $\Theta(n)$

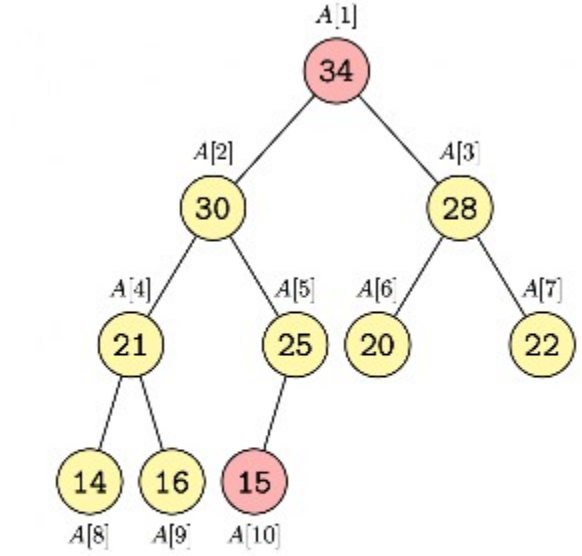
Vediamo ora l'$heapSort()$.
Ovviamente partiamo da una struttura max-heap quindi sappiamo che **l'elemento in prima posizione contiene il massimo**.
Collochiamo in fondo questo elemento, scambiandolo con l'elemento a cui "ruba la posizione".
Ora chiamiamo $maxHeapRestore()$ diminuendo di uno la posizione dell'array (per lasciare fuori l'ultimo elemento che vogliamo resti fisso lì) e ricominciamo il ciclo.



A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]	A[10]	A[11]
45	34	28	30	25	20	22	14	21	15	16

HEAPSORT(ITEM[] A, **int** n)

heapBuild(A, n)
for $i = n$ **downto** 2 **do**
  swap$(A, 1, i)$
  maxHeapRestore$(A, 1, i-1)$



A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]	A[10]	A[11]
34	30	28	21	25	20	22	14	16	15	45

HEAPSORT(ITEM[] A, **int** n)

heapBuild(A, n)
for $i = n$ **downto** 2 **do**
  swap$(A, 1, i)$
  maxHeapRestore$(A, 1, i-1)$

2 Correttezza di heapsort()

Invariante di ciclo: Al passo i, il sottovettore $A[i+1 \dots n]$ è ordinato; $A[1 \dots i] \leq A[i+1 \dots n]$ e $A[1]$ è la radice di un vettore heap di dimensione i.

Dimostrazione. Per induzione su i.

- **Inizializzazione**: Al passo $i = n$, il sottovettore $A[n+1 \dots n]$ è vuoto. Dopo aver eseguito heapBuild(), $A[1]$ è la radice di un vettore heap di dimensione n.
- **Conservazione**: Ad ogni passo i, il nodo in posizione $A[1]$ è il massimo fra i nodi rimanenti $A[1 \dots i]$ ed è minore di tutti i nodi in $A[i+1 \dots n]$.
- **Conclusione**: L'algoritmo termina quando $i = 1$. Questo significa che il vettore è dato dai nodi $A[2 \dots n]$ è ordinato, e $A[1] \leq A[2 \dots n]$. Quindi il vettore è ordinato.

Complessità

- heapBuild() costa $\Theta(n)$
- maxHeapRestore() costa $\Theta(\log i)$ in un heap con i elementi
- Viene eseguita con i che varia da 2 a n

$$T(n) = \sum_{i=2}^{n} \log i + \Theta(n) = \Theta(n \log n)$$

3 Complessità di heapsort()

Vengono compiuti n passi, ognuno dei quali richiede $O(\log n)$ passi. Quindi il costo totale è $O(n \log n)$. Notare la differenza con l'analisi di complessità di heapBuild(): poiché heapBuild() funziona dal basso, solo una volta viene costruito uno heap di altezza $\log n$. In questo caso, invece la funzionalità di maxHeapRestore() viene eseguita $n/2$ volte con alberi dell'altezza $h = O(\log n)$. Quindi il costo totale è $O(n \log n)$.

Implementazione code con priorità

Vedremo come si implementa una min-priority queue, in quanto negli esempi che vedremo in seguito daremo la precedenza a elementi con priorità minore.
Vedremo:

- Come strutturare un vettore che memorizza coppie
- Come implementare *minHeapRestore*()
- Come implementare i singoli metodi

Memorizzazione

```
PRIORITYITEM
int priority                          % Priorità
ITEM value                            % Elemento
int pos                               % Posizione nel vettore heap
```

```
swap(PRIORITYITEM[] H, int i, int j)
PRIORITYITEM temp = H[i]
H[i] = H[j]
H[j] = temp
H[i].pos = i
H[j].pos = j
```

Inizializzazione

```
PRIORITYQUEUE
int capacity          % Numero massimo di elementi nella coda
int dim               % Numero attuale di elementi nella coda
PRIORITYITEM[] H                      % Vettore heap

PRIORITYQUEUE PriorityQueue(int n)
    PRIORITYQUEUE t = new PRIORITYQUEUE
    t.capacity = n
    t.dim = 0
    t.H = new PRIORITYITEM[1...n]
    return t
```

Inserimento

```
PriorityItem insert(Item x, int p)
precondition: dim < capacity
dim = dim + 1
H[dim] = new PriorityItem()
H[dim].value = x
H[dim].priority = p
H[dim].pos = dim
int i = dim
while i > 1 and H[i].priority < H[p(i)].priority do
    swap(H, i, p(i))
    i = p(i)
return H[i]
```

minHeapRestore()

```
minHeapRestore(PriorityItem[] A, int i, int dim)
int min = i
if l(i) ≤ dim and A[l(i)].priority < A[min].priority then
    min = l(i)
if r(i) ≤ dim and A[r(i)].priority < A[min].priority then
    min = r(i)
if i ≠ min then
    swap(A, i, min)
    minHeapRestore(A, min, dim)
```

Cancellazione / lettura minimo

ITEM deleteMin()

---

precondition: $dim > 0$

swap$(H, 1, dim)$
$dim = dim - 1$
minHeapRestore$(H, 1, dim)$
return $H[dim + 1].value$

ITEM min()

---

precondition: $dim > 0$

return $H[1].value$

Decremento priorità

---

decrease(PRIORITYITEM x, **int** p)

---

precondition: $p < x.priority$

$x.priority = p$
int $i = x.pos$
while $i > 1$ **and** $H[i].priority < H[p(i)].priority$ **do**
    swap$(H, i, p(i))$
    $i = p(i)$

Tutte le operazioni che modificano gli heap, sistemano la proprietà heap:

- Lungo un cammino radice-foglia (come *deleteMin*$()$)
- Lungo un cammino nodo-radice (come *insert*$()$ e *decrease*$()$)

Poiché l'altezza è $\lfloor \log n \rfloor$, il costo di tali operazioni è $O(\log\ n)$

Operazione	Costo
insert()	$O(\log n)$
deleteMin()	$O(\log n)$
min()	$\Theta(1)$
decrease()	$O(\log n)$